

UNIVERSIDADE CIDADE DE SÃO PAULO

Aurora EduTech

SisGESC – ERP de Gestão Educacional

São Paulo
2026

Projeto de desenvolvimento de ERP de Gestão Escolar, apresentado ao curso de Análise e Desenvolvimento de Sistemas e Gestão da Tecnologia da Informação da Universidade Cidade

de São Paulo, sob orientação do Prof. Clovis Jose Ramos Ferraro, para matéria “Banco de Dados”.

Sumário

1. Diagrama Entidade-Relacionamento	
2. Dicionário de Dados	
Bloco 1 — Pessoa e Contato	
Bloco 2 — Recursos Humanos	
Bloco 3 — Folha de Pagamento	
Bloco 4 — Ano Letivo e Calendário	
Bloco 5 — Professores	
Bloco 6 — Alunos e Responsáveis	
Bloco 7 — Turmas e Matrículas	
Bloco 8 — Grade Curricular	
Bloco 9 — Atividades e Notas	
Bloco 10 — Frequência	
Bloco 11 — Boletim	
Bloco 12 — Financeiro Alunos	
Bloco 13 — Alerta de Evasão	
3. Script SQL	
3.1 Script SQL SEED	
4. Regras de Negócio e Analítica	
4.1 Regras de Negócio – Módulo Acadêmico	
4.2 Regras de Negócio – Módulo RH	
4.3 Regras de Negócio – Módulo Financeiro	
4.4 Regras de Negócio – Alerta de Evasão Escolar	
4.5 Observação sobre uso de dados para BI/IA	
5. Modelo Analítico OLAP – Star Schema	
5.1 Diagrama Star Schema	
5.2 Dicionário de Dados OLAP	
Convenções	
Dimensões Conformadas	
Módulo Acadêmico	
Módulo Financeiro	
Módulo RH	
Módulo Gestão	
6. Problemas Identificados e Correções Implementadas	
7. DER	

Referências

Introdução:

O projeto Aurora EduTech tem como objetivo desenvolver um banco de dados relacional para um Sistema de Gestão Escolar (ERP Educacional), integrando os processos das áreas acadêmica, financeira e de Recursos Humanos de uma instituição de ensino privada. O cenário adotado é uma escola de Ensino Fundamental I, atendendo alunos do 1º ao 5º ano, com foco nas necessidades específicas dessa etapa.

A proposta é construir uma base de dados sólida, bem-organizada e integrada, capaz de suportar operações do dia a dia (OLTP) como registro de frequência, notas e pagamentos e de subsidiar análises estratégicas, como indicadores de desempenho acadêmico, controle de inadimplência e gestão de pessoal.

Link do github: <https://github.com/IrisPfister/AuroraEduTech>

1. Diagrama Entidade-Relacionamento:



Imagem anexada do diagrama Entidade-Relacionamento completo com a cardinalidade em evidência.

```
Enum enum_tipo_dia {  
    util  
    feriado  
    recesso  
}
```

```
Enum enum_natureza_verba {  
    credito  
    debito  
}
```

```
Enum enum_status_contrato {  
    ativo  
    encerrado  
    suspenso  
}
```

```
Enum enum_periodo {  
    manha  
    tarde  
}
```

```
Enum enum_tipo_folha {  
    folha_mensal  
    adiantamento  
    decimo_terceiro  
    rescisao  
    ferias  
}
```

```
Enum enum_status_folha {  
    em_processamento  
    pendente  
    pago  
    cancelado  
    estornado  
}
```

```
Enum enum_dia_semana {  
    segunda  
    terca  
    quarta  
    quinta  
    sexta  
}
```

```
Enum enum_sexo {  
    masculino  
    feminino  
}
```

```
Enum enum_nacionalidade {  
    brasileiro  
    estrangeiro  
}
```

```
Enum enum_raca {  
    branca  
    negra  
    parda  
    amarela  
}
```

```
Enum enum_categoria {  
    celular  
    residencial  
    emergencia  
}
```

Relação dos tipos enumerados (ENUMs) utilizados no modelo de dados do SisGESC-AuroraEduTech.

2. Dicionário de dados:

Bloco 1 — Pessoa e contato:

Nome da Tabela: tb_pessoa				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_pessoa	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da pessoa no sistema.	
cpf	CHAR(11)	UNIQUE, NOT NULL	CPF da pessoa — 11 dígitos numéricos, único no sistema	
RG	VARCHAR(11)	NULL	Registro Geral (documento de identidade). Campo opcional.	
Nome	VARCHAR(30)	NOT NULL	Primeiro nome da pessoa.	
Sobrenome	VARCHAR(40)	NOT NULL	Sobrenome da pessoa.	
sexo	ENUM('masculino', 'feminino')	NOT NULL, DEFAULT 'feminino'	Gênero biológico declarado (masculino ou feminino).	
data_nascimento	DATE	NOT NULL	Data de nascimento da pessoa.	
nacionalidade	ENUM('brasileiro', 'estrangeiro')	NOT NULL, DEFAULT 'brasileiro'	Indica se a pessoa é brasileira ou estrangeira.	
raca	ENUM('branca', 'negra', 'parda', 'amarela')	NOT NULL, DEFAULT 'parda'	Autodeclaração de raça/cor conforme classificação IBGE.	

criado_em	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data e hora de criação do registro.	
atualizado_em	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Data e hora da última atualização do registro.	

2.1.2. Chaves alternativas

Nome_chave	Colunas	Descrição	Observação
uq_pessoa_cpf	cpf	Permite que não haja duplicidade.	

2.2. Tabela tb_endereco

Nome da Tabela: tb_endereco				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_endereco	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do endereço	
fk_pessoa	INT	FK, NOT NULL	Referência à pessoa vinculada	FK → tb_pessoa.pk_pessoa; ON DELETE RESTRICT
cep	CHAR(8)	NOT NULL	CEP (somente dígitos)	
numero	VARCHAR(10)	NULL	Número do logradouro	
complemento	VARCHAR(80)	NULL	Complemento do endereço	
principal	BOOL	NOT NULL, DEFAULT FALSE	Indica se é o endereço principal	
criado_em	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data e hora da criação	
atualizado_em	TIMESTAMP	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Data e hora da última atualização	

2.3. Tabela tb_email

Nome da Tabela: tb_email				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_email	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do e-mail	
fk_pessoa	INT	FK, NOT NULL	Referência à pessoa vinculada	FK → tb_pessoa.pk_pessoa; ON DELETE RESTRICT
email	VARCHAR(100)	UNIQUE, NOT NULL	Endereço de e-mail	Chave alternativa — UNIQUE
principal	BOOL	NOT NULL, DEFAULT FALSE	Indica se é o e-mail principal	

2.3.1.

Nome chave	Colunas	Descrição	Observação
uq_email	email	Permite que não haja duplicidade.	

2.4.

Nome da Tabela: tb_telefone				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_telefone	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do telefone	
fk_pessoa	INT	FK, NOT NULL	Referência à pessoa vinculada	FK → tb_pessoa.pk_pessoa
categoria	ENUM	NOT NULL	Tipo do telefone	celular residencial emergencial
ddd	CHAR(2)	NOT NULL	Código DDD da cidade	
numero	VARCHAR(9)	NOT NULL	Número do telefone (sem DDD)	
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o telefone está ativo	

principal	BOOL	NOT NULL, DEFAULT FALSE	Indica se é o telefone principal	
-----------	------	----------------------------	-------------------------------------	--

2.4.1.

Nome_chave	Colunas	Descrição	Observação
uq_ddd_telefone	ddd, numero	Permite que os números de telefone não se repitam.	

Bloco 2 — Recursos Humanos:

Nome da Tabela: tb_cargo				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_cargo	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do cargo	
nome_cargo	VARCHAR(60)	UNIQUE, NOT NULL	Nome do cargo	Chave alternativa — UNIQUE
descricao	VARCHAR(200)	NULL	Descrição das atribuições do cargo	
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o cargo está disponível	

2.5.1.

Nome_chave	Colunas	Descrição	Observação
uq_cargo_nome	nome_cargo	Permite que não haja duplicidades no cargo de cada funcionário.	O RH não poderá cadastrar o mesmo cargo para um funcionário mais de uma vez.

2.6.

Nome da Tabela: tb_tipo_contrato				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_contrato	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo	
descricao	VARCHAR(30)	UNIQUE, NOT NULL	Descrição do tipo de contrato	Ex: CLT, PJ, Estágio,

				Temporário.
--	--	--	--	-------------

Nome da Tabela: tb_funcionario				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_funcionario	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do funcionário	
fk_pessoa	INT	FK, UNIQUE, NOT NULL	Referência aos dados pessoais	FK → tb_pessoa.pk_pessoa; relação 1:1
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o funcionário está ativo	

Nome da Tabela: tb_contrato				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_contrato	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do contrato	
fk_funcionario	INT	FK, NOT NULL	Referência ao funcionário	FK → tb_funcionario.pk_funcionario
fk_tipo_contrato	INT	FK, NOT NULL	Tipo de contratação	FK → tb_tipo_contrato.pk_tipo_contrato
status_contrato	ENUM('ativo','encerrado','suspensão')	NOT NULL DEFAULT 'ativo'	Status atual do contrato	Trigger valida consistência com data_fim
carga_horaria_semanal	TINYINT UNSIGNED	NOT NULL	Carga horária semanal contratada (horas)	
data_inicio	DATE	NOT NULL	Data de início do contrato	
data_fim	DATE	NULL	Data de encerramento	NULL = prazo indeterminado; CHECK data_fim >= data_inicio

Nome da Tabela: tb_historico_salario				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_historico_salario	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do histórico	
fk_contrato	INT	FK, NOT NULL	Referência ao contrato	FK → tb_contrato.pk_contrato
salario	DECIMAL(10,2)	NOT NULL	Salário vigente no período	CHECK salario > 0
data_inicio	DATE	NOT NULL	Início da vigência do salário	UQ: fk_contrato + data_inicio
data_fim	DATE	NULL	Fim da vigência do salário	NULL = salário atual; CHECK data_fim >= data_inicio
motivo_reajuste	VARCHAR(100)	NULL	Justificativa do reajuste salarial	

Nome da Tabela: tb_historico_cargo				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_historico_cargo	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do histórico	
fk_contrato	INT	FK, NOT NULL	Referência ao contrato	FK → tb_contrato.pk_contrato
fk_cargo	INT	FK, NOT NULL	Referência ao cargo exercido	FK → tb_cargo.pk_cargo; UQ: fk_contrato + fk_cargo
data_inicio	DATE	NOT NULL	Início da vigência no cargo	
data_fim	DATE	NULL	Fim da vigência no cargo	NULL = cargo atual; CHECK data_fim >= data_inicio
motivo_mudanca	VARCHAR(100)	NULL	Justificativa para a mudança de cargo	

Bloco 3 — Folha de Pagamento:

Nome da Tabela: tb_folha_pagamento				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_folha_pagamento	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da folha	
fk_contrato	INT	FK, NOT NULL	Referência ao contrato do funcionário	FK → tb_contrato.pk_contrato
status_folha	ENUM('em_processamento', 'pendente', 'pagado', 'cancelado', 'estornado')	NOT NULL DEFAULT 'em_processamento'	Status do processamento da folha	
tipo_folha	ENUM('folha_mensal', 'adiantamento', 'decimo_terceiro', 'rescisao', 'ferias')	NOT NULL	Tipo de folha de pagamento	
mes	TINYINT UNSIGNED	NOT NULL	Mês de referência (1-12)	CHECK mes BETWEEN 1 AND 12; UQ: fk_contrato + ano + mes + tipo_folha
ano	YEAR	NOT NULL	Ano de referência	
valor	DECIMAL(10,2)	NOT NULL	Valor total líquido da folha	
data_emiss	DATE	NOT NULL	Data de	

ao			emissão da folha	
criado_em	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Data e hora de criação	
atualizado_em	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Data e hora da última atualização	

Nome da Tabela:				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_verba	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo de verba	
descricao	VARCHAR(60)	UNIQUE, NOT NULL	Descrição da verba	Ex: Salário Base, INSS, Vale-transporte
natureza_verba	ENUM	NOT NULL	Natureza financeira da verba	credito = acréscimo debito = desconto
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se a verba está ativa	

Nome da Tabela:				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
fk_folha_pagamento	INT	FK, NOT NULL	Referência à folha de pagamento	FK → tb_folha_pagamento.pk_folha_pagamento; UQ: fk_folha + fk_tipo_verba
fk_tipo_verba	INT	FK, NOT NULL	Referência ao tipo de verba	FK → tb_tipo_verba.pk_tipo_verba
valor	DECIMAL(10,	NOT	Valor da	CHECK valor >= 0

	2)	NULL	verba no período	
referencia	VARCHAR(80)	NULL	Informação complementar da verba	Ex: '20 dias trabalhados'

Bloco 4 — Ano Letivo e Calendário:

Nome da Tabela:				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_ano_letivo	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do ano letivo	
ano	YEAR	UNIQUE, NOT NULL	Ano de vigência	Chave alternativa — UNIQUE
data_inicio	DATE	NOT NULL	Data de início do ano letivo	
data_fim	DATE	NOT NULL	Data de término do ano letivo	CHECK data_fim >= data_inicio
ativo	BOOL	NOT NULL, DEFAULT FALSE	Indica se o ano letivo está ativo	Somente um registro deve ser TRUE por vez

Nome da Tabela:				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_calendario	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do dia	
fk_ano_letivo	INT	FK, NOT NULL	Referência ao ano letivo	FK → tb_ano_letivo.pk_ano_letivo; UQ: fk_ano_letivo + data_evento
data_evento	DATE	NOT NULL	Data do dia registrado no calendário	
tipo_dia	ENUM	NOT NULL, DEFAULT 'util'	Classificação do dia	util feriado recesso
letivo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o dia conta presença	

Nome da Tabela:				
-----------------	--	--	--	--

tb_evento_escolar				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_evento	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do evento	
fk_calendario_letivo	INT	FK, NOT NULL	Referência ao calendário letivo	FK → tb_calendario_letivo.pk_calendario
titulo_evento	VARCHAR(100)	NOT NULL	Título/nome do evento	
descricao	TEXT	NULL	Descrição detalhada do evento	
hora_inicio	TIME	NULL	Horário de início do evento	
hora_fim	TIME	NULL	Horário de término do evento	CHECK hora_fim > hora_inicio (quando preenchido)

Bloco 5 — Professores:

Nome da Tabela: tb_tipo_aula				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_aula	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo de aula	
descricao	VARCHAR(40)	UNIQUE, NOT NULL	Metodologia de ensino predominante	Ex: lab de informatica

Nome da Tabela: tb_professor				
Nome da Coluna:	Tipo de	Restrição:	Descrição:	Observação:

	Dado:			
pk_professor	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do professor	
fk_funcionario	INT	FK, UNIQUE, NOT NULL	Referência ao funcionário	FK → tb_funcionario.pk_funcionario; relação 1:1
fk_tipo_aula	INT	FK, NULL	Metodologia de ensino predominante	FK → tb_tipo_aula.pk_tipo_aula; opcional
utiliza_rubrica_avaliativa	BOOLEAN	NOT NULL, DEFAULT FALSE	Usa rubricas na avaliação	
atende_educacao_especial	BOOLEAN	NOT NULL, DEFAULT FALSE	Habilitado para atender educação especial	

Nome da Tabela: tb_nivel_formacao				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_nivel_formacao	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do nível	
descricao	VARCHAR(40)	UNIQUE, NOT NULL	Nível de escolaridade	Ex: Graduação, Mestrado, Doutorado

Nome da Tabela: tb_formacao_professor				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_formacao	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da formação	
fk_professor	INT	FK, NOT NULL	Referência ao professor	FK → tb_professor.pk_professor
fk_nivel_formacao	INT	FK, NOT NULL	Nível de escolaridade da	FK → tb_nivel_formacao.pk_nivel_formacao

			formação	
curso	VARCHAR(100)	NOT NULL	Nome do curso concluído	
instituicao	VARCHAR(100)	NOT NULL	Instituição onde concluiu	
ano_conclusao	YEAR	NOT NULL	Ano de conclusão do curso	

Bloco 6 — Alunos e Responsáveis:

Nome da Tabela:				
tb_tipo_escola_anterior				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_escola_anterior	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo	
descricao	VARCHAR(30)	UNIQUE, NOT NULL	Tipo de escola de origem	Ex: Pública, Privada, Nenhuma

Nome da Tabela:				
tb_aluno				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_aluno	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do aluno	
fk_pessoa	INT	FK, UNIQUE, NOT NULL	Referência aos dados pessoais	FK → tb_pessoa.pk_pessoa; relação 1:1
fk_tipo_escola_anterior	INT	FK, NULL	Tipo de escola anterior ao ingresso	FK → tb_tipo_escola_anterior.pk_tipo_escola_anterior

data_ingresso	DATE	NOT NULL	Data de ingresso na escola	
faz_atividade_extracurricular	BOOL	NOT NULL, DEFAULT FALSE	Indica se realiza atividade extracurricular	
ra	VARCHAR (15)	UNIQUE, NOT NULL	Registro do Aluno (RA)	Chave alternativa alfanumérica; CHECK ra REGEXP '[0-9]+\$'

Nome da Tabela: tb_responsavel				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_responsavel	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do responsável	
fk_pessoa	INT	FK, UNIQUE, NOT NULL	Referência aos dados pessoais	FK → tb_pessoa.pk_pessoa; relação 1:1
criado_em	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data e hora da criação	
atualizado_em	TIMESTAMP	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Data e hora da última atualização	

Nome da Tabela: tb_responsavel_aluno				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
fk_aluno	INT	PK, FK, NOT NULL	Aluno vinculado	FK → tb_aluno.pk_aluno; PK composta com fk_responsavel
fk_responsavel	INT	PK, FK, NOT NULL	Referência ao responsável	FK → tb_responsavel.pk_responsavel; PK composta com fk_aluno
responsavel_principal	BOOL	NOT NULL, DEFAULT FALSE	É o responsável principal do aluno	
responsavel_financeiro	BOOL	NOT NULL, DEFAULT FALSE	É o responsável pelos pagamentos	
pode_retirar	BOOL	NOT NULL, DEFAULT	Autorizado a retirar o	

		FALSE	aluno na escola	
--	--	-------	-----------------	--

2.6 Tabela tb_profissao

Nome da Tabela: tb_profissao				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_profissao	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da profissão	
profissao	VARCHAR(30)	UNIQUE, NOT NULL	Nome da profissão	
descricao	VARCHAR(100)	NOT NULL	Descrição da profissão	
criado_em	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Data e hora de criação	

2.7 Tabela tb_profissao_responsavel

Nome da Tabela: tb_profissao_responsavel				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
fk_responsavel	INT	PK, FK, NOT NULL	Referência ao responsável	
fk_profissao	INT	PK, FK, NOT NULL	Referência à profissão	
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o vínculo está ativo	
criado_em	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Data e hora de criação	

Bloco 7 — Turmas e Matrículas:

Nome da Tabela: tb_turma				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_turma	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da turma	
fk_ano_letivo	INT	FK, NOT NULL	Referência ao ano letivo	FK → tb_ano_letivo.pk_ano_letivo
nome_turma	VARCHAR(10)	NOT NULL	Nome da turma (ex: '1A')	UQ: fk_ano_letivo + nome_turma + periodo
ano_escolar	TINYINT UNSIGNED	NOT NULL	Ano ou série escolar	CHECK ano_escolar BETWEEN 1 AND 5
periodo	ENUM	NOT NULL	Turno de	manha tarde

			funcionament o	
capacidade_maxim a	TINYINT UNSIGNED	NULL	Capacidade máxima de alunos na turma	

Nome da Tabela: tb_status_matricula				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_status_matricula	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do status	
descricao	VARCHAR(30)	UNIQUE, NOT NULL	Descrição do status	Ex: ativa, trancada, transferida, encerrada

Nome da Tabela: tb_tipo_ingresso				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_ingresso	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo	
descricao	VARCHAR(60)	UNIQUE, NOT NULL	Nome do tipo de ingresso	Ex: Matrícula nova, Rematrícula, Transferência
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o tipo está disponível	

Nome da Tabela: tb_matricula				
Nome da Coluna:	Tipo de Dado :	Restrição:	Descrição:	Observação:

pk_matricula	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da matrícula	
fk_aluno	INT	FK, NOT NULL	Aluno matriculado	FK → tb_aluno.pk_aluno; UQ: fk_aluno + fk_turma
fk_turma	INT	FK, NOT NULL	Turma em que o aluno está matriculado	FK → tb_turma.pk_turma
fk_tipo_ingresso	INT	FK, NOT NULL	Tipo de ingresso utilizado	FK → tb_tipo_ingresso.pk_tipo_ingresso
fk_status_matricula	INT	FK, NOT NULL	Status atual da matrícula	FK → tb_status_matricula.pk_status_matricula
data_matricula	DATE	NOT NULL	Data de efetivação da matrícula	

Bloco 8 — Grade Curricular:

Nome da Tabela: tb_area_conhecimento				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_area_conhecimento	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da área	
descricao	VARCHAR(40)	UNIQUE, NOT NULL	Nome da área do conhecimento	Ex: Linguagens, Exatas, Ciências

Nome da Tabela: tb_materia				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_materia	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da matéria	
fk_area_conhecimento	INT	FK, NOT NULL	Área do conhecimento da disciplina	FK → tb_area_conhecimento.pk_area_conhecimento
nome_materia	VARCHAR(60)	UNIQUE, NOT NULL	Nome da disciplina	Chave alternativa — UNIQUE
ativo	BOOL	NOT NULL,	Indica se a	Exclusão lógica

		DEFAULT TRUE	matéria está ativa	
--	--	-----------------	-----------------------	--

Nome da Tabela: tb_grade_curricular				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_grade	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da grade	Entidade associativa turma–matéria–professor
fk_turma	INT	FK, NOT NULL	Referência à turma	FK → tb_turma.pk_turma
fk_materia	INT	FK, NOT NULL	Referência à matéria	FK → tb_materia.pk_materia; UQ: fk_turma + fk_materia + data_inicio
fk_professor	INT	FK, NOT NULL	Professor responsável pela disciplina	FK → tb_professor.pk_professor
data_inicio	DATE	NOT NULL	Início da vigência da grade	
data_fim	DATE	NULL	Fim da vigência da grade	

Nome da Tabela: tb_horario_aula				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_horario_aula	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do horário	
fk_grade	INT	FK, NOT NULL	Referência à grade curricular	FK → tb_grade_curricular.pk_grade
dia_semana	ENUM	NOT NULL	Dia da semana da aula	segunda terça quarta quinta sexta
hora_inicio	TIME	NOT NULL	Horário de início da aula	UQ: fk_grade + dia_semana + hora_inicio
hora_fim	TIME	NOT NULL	Horário de término da	CHECK hora_fim > hora_inicio

			aula	
sala	VARCHAR(10)	NULL	Identificação da sala de aula	

Bloco 9 — Atividades e Notas:

Nome da Tabela:				
tb_tipo_atividade				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_atividade	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo	
descricao	VARCHAR(40)	UNIQUE, NOT NULL	Tipo de atividade avaliativa	Ex: atividade em grupo

Nome da Tabela:				
tb_atividade_professor				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_atividade_professor	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da atividade	
fk_grade	INT	FK, NOT NULL	Grade curricular à qual pertence	FK → tb_grade_curricular.pk_grade
fk_tipo_atividade	INT	FK, NOT NULL	Tipo de atividade	FK → tb_tipo_atividade.pk_tipo_atividade
nome_atividade	VARCHAR(100)	NOT NULL	Nome/título da atividade	
bimestre	TINYINT UNSIGNED	NOT NULL	Bimestre letivo (1–4)	CHECK bimestre BETWEEN 1 AND 4
data_prevista	DATE	NOT NULL	Data prevista para realização	
peso	DECIMAL(4,2)	NOT NULL, DEFAULT 1.00	Peso da atividade no cálculo da nota	CHECK peso > 0
valor_maximo	DECIMAL(4,2)	NOT NULL, DEFAULT 10.00	Pontuação máxima	CHECK valor_maximo > 0

			possível	
--	--	--	----------	--

Nome da Tabela: tb_nota_atividade				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
fk_matricula	INT	FK, NOT NULL	Referência à matrícula	FK → tb_matricula.pk_matricula; PK composta com fk_atividade_professor
fk_atividade_professor	INT	FK, NOT NULL	Referência à atividade avaliativa do professor	FK → tb_atividade_professor.pk_atividade_professor; PK composta com fk_matricula
valor_nota	DECIMAL(4,2)	NULL	Nota obtida pelo aluno na atividade	NULL = não realizada/lançada; CHECK 0 <= valor_nota <= 10
entregue_no_prazo	BOOL	NOT NULL, DEFAULT TRUE	Indica a presença do aluno (TRUE = presente; FALSE = ausente)	
criado_em	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Data e hora de criação do registro	
atualizado_em	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Data e hora da última	

		TAMP ON UPDATE CURRENT_TIMES TAMP	atualiza ção	
--	--	---	-----------------	--

Bloco 10 — Frequência:

Nome da Tabela: tb_presenca_aluno				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_presenca	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do registro	
fk_matricula	INT	FK, NOT NULL	Referência à matrícula	FK → tb_matricula.pk_matricula
fk_calendario_letivo	INT	FK, NOT NULL	Referência ao calendário letivo	FK → tb_calendario_letivo.pk_calendario; UQ: fk_matricula + fk_calendario_letivo + fk_horario_aula
fk_horario_aula	INT	FK, NOT NULL	Referência ao horário da aula	FK → tb_horario_aula.pk_horario_aula
presenca	BOOL	NOT NULL	TRUE = presente; FALSE = ausente	
justificativa	VARCHAR(255)	NULL	Justificativa da falta, quando houver	

Bloco 11 — Boletim:

Nome da Tabela: tb_situacao_boletim				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_situacao_boletim	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único da situação	
descricao	VARCHAR(30)	UNIQUE, NOT NULL	Situação do aluno no bimestre	Ex: Em curso, Aprovado, Reprovado, Risco
ativo	BOOL	NOT NULL,	Indica se o registro	Exclusão

		DEFAULT TRUE	está ativo	lógica
--	--	--------------	------------	--------

Nome da Tabela: tb_boletim				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_boletim	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do boletim	
fk_matricula	INT	FK, NOT NULL	Referência à matrícula	FK → tb_matricula.pk_matricula; UQ: fk_matricula + bimestre
fk_situacao_boletim	INT	FK, NOT NULL	Situação acadêmica no bimestre	FK → tb_situacao_boletim.pk_situacao_boletim
bimestre	TINYINT UNSIGNED	NOT NULL	Bimestre de referência (1–4)	CHECK bimestre BETWEEN 1 AND 4

Bloco 12 — Financeiro alunos:

Nome da Tabela: tb_contrato_financeiro				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_contrato_financeiro	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do contrato financeiro	
fk_matricula	INT	FK, UNIQUE, NOT NULL	Referência à matrícula	FK → tb_matricula.pk_matricula; relação 1:1
valor_matricula	DECIMAL(10,2)	NOT NULL	Taxa de matrícula cobrada	CHECK valor_matricula >= 0
valor_mensalidade	DECIMAL(10,2)	NOT NULL	Valor padrão da mensalidade	CHECK valor_mensalidade > 0
dia_vencimento	TINYINT	NOT NULL	Dia do mês para vencimento	CHECK dia_vencimento BETWEEN 1 AND 28
data_inicio	DATE	NOT NULL	Início de vigência do contrato	
data_fim	DATE	NULL	Fim de	NULL = contrato ativo;

			vigência do contrato	CHECK data_fim >= data_inicio
--	--	--	----------------------	-------------------------------

Nome da Tabela: tb_status_pagamento				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_status_pagamento	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do status	
descricao	VARCHAR(30)	UNIQUE, NOT NULL	Descrição do status do pagamento	Ex: pendente, pago, atrasado, cancelado, estornado
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o registro está ativo	Exclusão lógica

Nome da Tabela: tb_pagamento				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_pagamento	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do pagamento	
fk_contrato_financeiro	INT	FK, NOT NULL	Referência ao contrato financeiro	FK → tb_contrato_financeiro.pk_contrato_financeiro; UQ: fk_contrato_financeiro + mes_referencia + ano_referencia
fk_status_pagamento	INT	FK, NOT NULL	Status do pagamento	FK → tb_status_pagamento.pk_status_pagamento
valor_original	DECIMAL(10,2)	NOT NULL	Valor original da cobrança	CHECK valor_cobrado > 0

valor_pago	DECIMAL(10,2)	NULL	Valor efetivamente pago	NULL = não pago; CHECK valor_pago >= 0
data_vencimento	DATE	NOT NULL	Data de vencimento da parcela	NULL = não pago ainda
data_pagamento	DATE	NULL	Data em que o pagamento foi realizado	
mes_referencia	TINYINT UNSIGNED	NOT NULL	Mês de referência (1-12)	CHECK mes_referencia BETWEEN 1 AND 12
ano_referencia	YEAR	NOT NULL	Ano de referência	

Bloco 13 — Alerta de Evasão (Análise preditiva):

Nome da Tabela:				
tb_nivel_risco_evasao				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_nivel_risco	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do nível de risco	
descricao	VARCHAR(30)	UNIQUE, NOT NULL	Descrição do nível de risco	Ex: Baixo, Médio, Alto, Crítico
ordem	TINYINT	UNIQUE, NOT NULL	Ordem de criticidade do nível	Permite ordenar os níveis do menos ao mais crítico

Nome da Tabela:				
tb_status_alerta				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_status_alerta	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do status	
descricao	VARCHAR(40)	UNIQUE, NOT NULL	Descrição do status do alerta	Ex: aberto, em acompanhamento, resolvido, evadiu

ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o registro está ativo	Exclusão lógica
-------	------	---------------------------	------------------------------------	-----------------

Nome da Tabela: tb_tipo_risco_evasao				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_tipo_risco_evasao	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do tipo de risco	
descricao	VARCHAR(60)	UNIQUE, NOT NULL	Descrição do fator de risco identificado	Ex: Faltas excessivas, Inadimplência, Queda de desempenho
ativo	BOOL	NOT NULL, DEFAULT TRUE	Indica se o tipo de risco está ativo	Exclusão lógica

Nome da Tabela: tb_alerta_evasao				
Nome da Coluna:	Tipo de Dado:	Restrição:	Descrição:	Observação:
pk_alerta	INT	PK, NOT NULL, AUTO_INCREMENT	Identificador único do alerta	
fk_matricula	INT	FK, NOT NULL	Matrícula do aluno em risco	FK → tb_matricula.pk_matricula
fk_responsavel_alerta	INT	FK, NOT NULL	Funcionário responsável pelo acompanham ento	FK → tb_funcionario.pk_funcionario
fk_tipo_risco_evasao	INT	FK, NOT NULL	Tipo de risco identificado	FK → tb_tipo_risco_evasao.pk_tipo_risco_evasao
fk_nivel_risco	INT	FK, NOT NULL	Nível de criticidade do risco	FK → tb_nivel_risco_evasao.pk_nivel_risco
fk_status_alerta	INT	FK, NOT NULL	Status de	FK →

			acompanham ento do alerta	tb_status_alerta.pk_status_alerta
data_alerta	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Data e hora de emissão do alerta	
descricao	TEXT	NULL	Descrição do contexto do alerta	
data_resolucao	DATETIME	NULL	Data e hora de resolução do alerta	NULL = alerta em aberto
observacao_resolucao	TEXT	NULL	Detalhes sobre como o alerta foi resolvido	Preenchido ao resolver ou encerrar o alerta

3. SCRIPT SQL:

-- SISTEMA AURORA EDUTECH

-- Created by Redgate Data Modeler (<https://datamodeler.redgate-platform.com>)
-- Last modification date: 2026-05-09 17:03:31.092

```

CREATE DATABASE sistema_aurora_edutech; USE sistema_aurora_edutech; -- tables --
Table: tb_alerta_evasao CREATE TABLE tb_alerta_evasao ( pk_alerta int NOT NULL
AUTO_INCREMENT, fk_matricula int NOT NULL, fk_responsavel_alerta int NOT NULL,
fk_tipo_risco_evasao int NOT NULL, fk_nivel_risco int NOT NULL, fk_status_alerta int
NOT NULL, data_alerta datetime NOT NULL DEFAULT current_timestamp, descricao text NULL,
data_resolucao datetime NULL, observacao_resolucao text NULL, atualizado_em timestamp
NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX
uq_alerta_matricula (fk_matricula,fk_tipo_risco_evasao,data_alerta), CHECK
(data_resolucao IS NULL OR data_resolucao >= data_alerta), CHECK (data_alerta BETWEEN
'2000-01-01 00:00:00' AND '2099-12-31 23:59:59'), CHECK ( data_resolucao IS NULL OR
(observacao_resolucao IS NOT NULL AND CHAR_LENGTH(TRIM(observacao_resolucao)) > 0)),
CONSTRAINT pk_alerta_evasao PRIMARY KEY (pk_alerta) );

-- Table: tb_aluno CREATE TABLE tb_aluno ( pk_aluno int NOT NULL AUTO_INCREMENT,
fk_pessoa int NOT NULL, fk_tipo_escola_anterior int NULL, data_ingresso date NOT NULL,
faz_atividade_extracurricular bool NOT NULL DEFAULT false, ra varchar(15) NOT NULL,
UNIQUE INDEX uq_aluno_pessoa (fk_pessoa), UNIQUE INDEX uq_aluno (ra), CHECK (ra REGEXP
'^[0-9]+$' ), CONSTRAINT pk_aluno PRIMARY KEY (pk_aluno) );

-- Table: tb_ano_letivo CREATE TABLE tb_ano_letivo ( pk_ano_letivo int NOT NULL
AUTO_INCREMENT, ano year NOT NULL, data_inicio date NOT NULL, data_fim date NOT NULL,
ativo bool NOT NULL DEFAULT false, UNIQUE INDEX uq_ano_letivo_ano (ano), CHECK (ano
BETWEEN 2000 AND 2099), CHECK (data_inicio BETWEEN '2000-01-01' AND '2099-12-31'),
CHECK (data_fim >= data_inicio), CONSTRAINT pk_ano_letivo PRIMARY KEY
(pk_ano_letivo) );

-- Table: tb_area_conhecimento CREATE TABLE tb_area_conhecimento
( pk_area_conhecimento int NOT NULL AUTO_INCREMENT, descricao varchar(40) NOT NULL,
UNIQUE INDEX uq_area_conhecimento_desc (descricao), CONSTRAINT pk_area_conhecimento
PRIMARY KEY (pk_area_conhecimento) );

-- Table: tb_atividade_professor CREATE TABLE tb_atividade_professor
( pk_atividade_professor int NOT NULL AUTO_INCREMENT, fk_grade int NOT NULL,
fk_tipo_atividade int NOT NULL, nome_atividade varchar(100) NOT NULL, bimestre tinyint
UNSIGNED NOT NULL, data_prevista date NOT NULL, peso decimal(4,2) NOT NULL DEFAULT
1.00, valor_maximo decimal(4,2) NOT NULL DEFAULT 10.00, UNIQUE INDEX
uq_atividade_grade_nome_bimestre (bimestre,fk_grade,nome_atividade), CHECK (( bimestre
BETWEEN 1 AND 4 )), CHECK (peso > 0 AND peso <= 10), CHECK (valor_maximo > 0 AND
valor_maximo <= 10), CHECK (data_prevista >= '2020-01-01'), CONSTRAINT
pk_atividade_professor PRIMARY KEY (pk_atividade_professor) );

-- Table: tb_boletim CREATE TABLE tb_boletim ( pk_boletim int NOT NULL AUTO_INCREMENT,
fk_matricula int NOT NULL, fk_situacao_boletim int NOT NULL, bimestre tinyint UNSIGNED
NOT NULL, UNIQUE INDEX uq_boletim_bimestre (fk_matricula,bimestre), CHECK (( bimestre
BETWEEN 1 AND 4 )), CONSTRAINT pk_boletim PRIMARY KEY (pk_boletim) );

-- Table: tb_calendario_letivo CREATE TABLE tb_calendario_letivo ( pk_calendario int
NOT NULL AUTO_INCREMENT, fk_ano_letivo int NOT NULL, data_evento date NOT NULL,
tipo_dia enum('util','feriado','recesso') NOT NULL DEFAULT 'util', letivo bool NOT
NULL DEFAULT true, UNIQUE INDEX uq_calendario_dia (fk_ano_letivo,data_evento), CHECK
(data_evento BETWEEN '2000-01-01' AND '2099-12-31'), CHECK (tipo_dia != 'util' OR
letivo = true), CONSTRAINT pk_calendario PRIMARY KEY (pk_calendario) );

```

```
-- Table: tb_cargo CREATE TABLE tb_cargo ( pk_cargo int NOT NULL AUTO_INCREMENT,
nome_cargo varchar(60) NOT NULL, descricao varchar(200) NULL, ativo bool NOT NULL
DEFAULT true, UNIQUE INDEX uq_cargo_nome (nome_cargo), CONSTRAINT pk_cargo PRIMARY KEY
(pk_cargo) );
```

```
-- Table: tb_contrato CREATE TABLE tb_contrato ( pk_contrato int NOT NULL
AUTO_INCREMENT, fk_funcionario int NOT NULL, fk_tipo_contrato int NOT NULL,
status_contrato enum('ativo','encerrado','suspensao') NOT NULL DEFAULT 'ativo',
carga_horaria_semanal tinyint UNSIGNED NOT NULL, data_inicio date NOT NULL, data_fim
date NULL, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, atualizado_em
timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, CHECK
(( data_fim IS NULL OR data_fim >= data_inicio )), CHECK (carga_horaria_semanal
BETWEEN 1 AND 60), CONSTRAINT pk_contrato PRIMARY KEY (pk_contrato) );
```

```
-- Table: tb_contrato_financeiro CREATE TABLE tb_contrato_financeiro
( pk_contrato_financeiro int NOT NULL AUTO_INCREMENT, fk_matricula int NOT NULL,
valor_matricula decimal(10,2) NOT NULL, valor_mensalidade decimal(10,2) NOT NULL,
dia_vencimento tinyint UNSIGNED NOT NULL, data_inicio date NOT NULL, data_fim date
NULL, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, atualizado_em timestamp
NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX
uq_contrato_financeiro_matricula (fk_matricula), CHECK (( valor_mensalidade > 0 )),
CHECK (( valor_matricula >= 0 )), CHECK (( dia_vencimento BETWEEN 1 AND 28 )), CHECK
(( data_fim IS NULL OR data_fim >= data_inicio )), CHECK (data_inicio BETWEEN '2000-
01-01' AND '2099-12-31'), CONSTRAINT pk_contrato_financeiro PRIMARY KEY
(pk_contrato_financeiro) );
```

```
-- Table: tb_email CREATE TABLE tb_email ( pk_email int NOT NULL AUTO_INCREMENT,
fk_pessoa int NOT NULL, email varchar(100) NOT NULL, principal bool NOT NULL DEFAULT
false, UNIQUE INDEX uq_email (email), CHECK (email REGEXP '^[A-Za-z0-9._%~]+@[A-Za-z0-
9.-]+\.[A-Za-z]{2,4}$'), CONSTRAINT pk_email PRIMARY KEY (pk_email) );
```

```
-- Table: tb_endereco CREATE TABLE tb_endereco ( pk_endereco int NOT NULL
AUTO_INCREMENT, fk_pessoa int NOT NULL, cep char(8) NOT NULL, numero varchar(10) NULL,
complemento varchar(80) NULL, principal bool NOT NULL DEFAULT false, criado_em
timestamp NOT NULL DEFAULT current_timestamp, atualizado_em timestamp NOT NULL DEFAULT
current_timestamp ON UPDATE CURRENT_TIMESTAMP, CHECK (CHAR_LENGTH(cep) = 8 AND cep
REGEXP '^[0-9]+$'), CONSTRAINT pk_endereco PRIMARY KEY (pk_endereco) );
```

```
-- Table: tb_evento_escolar CREATE TABLE tb_evento_escolar ( pk_evento int NOT NULL
AUTO_INCREMENT, fk_calendario int NOT NULL, titulo_evento varchar(100) NOT NULL,
descricao text NULL, hora_inicio time NULL, hora_fim time NULL, CHECK (( hora_fim IS
NULL OR hora_fim > hora_inicio )), CHECK (CHAR_LENGTH(titulo_evento) > 0), CHECK
(hora_fim IS NULL OR hora_inicio IS NOT NULL), CONSTRAINT pk_evento_escolar PRIMARY
KEY (pk_evento) );
```

```
-- Table: tb_folha_pagamento CREATE TABLE tb_folha_pagamento ( pk_folha_pagamento int
NOT NULL AUTO_INCREMENT, fk_contrato int NOT NULL, tipo_folha
enum('folha_mensal','adiantamento','decimo_terceiro','rescisao','ferias') NOT NULL,
valor decimal(10,2) NOT NULL, ano year NOT NULL, mes tinyint UNSIGNED NOT NULL,
data_emissao date NOT NULL, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
atualizado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
status_folha enum('em_processamento','pendente','pago','cancelado','estornado') NOT
NULL DEFAULT 'em_processamento', UNIQUE INDEX uq_contrato_ano_mes
(fk_contrato,ano,mes,tipo_folha), CHECK (valor >= 0), CHECK (mes BETWEEN 1 AND 12),
```

```
CHECK (YEAR(data_emissao) = ano AND MONTH(data_emissao) = mes), CONSTRAINT
pk_folha_pagamento PRIMARY KEY (pk_folha_pagamento) );
```

```
-- Table: tb_formacao_professor CREATE TABLE tb_formacao_professor ( pk_formacao int
NOT NULL AUTO_INCREMENT, fk_professor int NOT NULL, fk_nivel_formacao int NOT NULL,
curso varchar(100) NOT NULL, instituicao varchar(100) NOT NULL, ano_conclusao year NOT
NULL,
UNIQUE INDEX tb_formacao_professor_curso
(ano_conclusao,fk_nivel_formacao,fk_professor,curso,instituicao), CHECK (ano_conclusao
BETWEEN 1970 AND 2099), CONSTRAINT pk_formacao_professor PRIMARY KEY (pk_formacao) );
```

```
-- Table: tb_funcionario CREATE TABLE tb_funcionario ( pk_funcionario int NOT NULL
AUTO_INCREMENT, fk_pessoa int NOT NULL, ativo bool NOT NULL DEFAULT true, criado_em
timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, atualizado_em timestamp NOT NULL DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX uq_funcionario_pessoa
(fk_pessoa), CONSTRAINT pk_funcionario PRIMARY KEY (pk_funcionario) );
```

```
-- Table: tb_grade_curricular CREATE TABLE tb_grade_curricular ( pk_grade int NOT NULL
AUTO_INCREMENT, fk_turma int NOT NULL, fk_materia int NOT NULL, fk_professor int NOT
NULL, data_inicio date NOT NULL, data_fim date NULL, UNIQUE INDEX
uq_grade_turma_materia (fk_turma,fk_materia,data_inicio), CHECK (( data_fim IS NULL OR
data_fim >= data_inicio )), CONSTRAINT pk_grade_curricular PRIMARY KEY (pk_grade) );
```

```
-- Table: tb_historico_cargo CREATE TABLE tb_historico_cargo ( pk_historico_cargo int
NOT NULL AUTO_INCREMENT, fk_contrato int NOT NULL, fk_cargo int NOT NULL, data_inicio
date NOT NULL, data_fim date NULL, motivo_mudanca varchar(100) NULL, criado_em
timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, atualizado_em timestamp NOT NULL DEFAULT
CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX uq_hist_cargo_vigencia
(fk_contrato,fk_cargo,data_inicio), CHECK (( data_fim IS NULL OR data_fim >=
data_inicio )), CONSTRAINT pk_historico_cargo PRIMARY KEY (pk_historico_cargo) );
```

```
-- Table: tb_historico_salario CREATE TABLE tb_historico_salario
( pk_historico_salario int NOT NULL AUTO_INCREMENT, fk_contrato int NOT NULL, salario
decimal(10,2) NOT NULL, data_inicio date NOT NULL, data_fim date NULL, motivo_reajuste
varchar(100) NULL, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
atualizado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE INDEX uq_hist_salario_vigencia (fk_contrato,data_inicio), CHECK (( salario >
0 )), CHECK (( data_fim IS NULL OR data_fim >= data_inicio )), CONSTRAINT
pk_historico_salario PRIMARY KEY (pk_historico_salario) );
```

```
-- Table: tb_horario_aula CREATE TABLE tb_horario_aula ( pk_horario_aula int NOT NULL
AUTO_INCREMENT, fk_grade int NOT NULL, dia_semana
enum('segunda','terca','quarta','quinta','sexta') NOT NULL, hora_inicio time NOT NULL,
hora_fim time NOT NULL, sala varchar(10) NULL, UNIQUE INDEX uq_horario_grade_dia_hora
(fk_grade,dia_semana,hora_inicio), CHECK (( hora_fim > hora_inicio )), CHECK (sala IS
NULL OR CHAR_LENGTH(sala) > 0), CONSTRAINT pk_horario_aula PRIMARY KEY
(pk_horario_aula) );
```

```
-- Table: tb_materia CREATE TABLE tb_materia ( pk_materia int NOT NULL AUTO_INCREMENT,
fk_area_conhecimento int NOT NULL, nome_materia varchar(60) NOT NULL, ativo bool NOT
NULL DEFAULT true, UNIQUE INDEX uq_materia_nome (nome_materia), CONSTRAINT pk_materia
PRIMARY KEY (pk_materia) );
```

```
-- Table: tb_matricula CREATE TABLE tb_matricula ( pk_matricula int NOT NULL
AUTO_INCREMENT, fk_turma int NOT NULL, fk_tipo_ingresso int NOT NULL,
```

```
fk_status_matricula int NOT NULL, data_matricula date NOT NULL, fk_aluno int NOT NULL,
criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, atualizado_em timestamp NOT
NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX
uq_matricula_aluno_turma (fk_aluno,fk_turma), CHECK (data_matricula BETWEEN '2000-01-
01' AND '2099-12-31'), CONSTRAINT pk_matricula PRIMARY KEY (pk_matricula) );
```

```
-- Table: tb_nivel_formacao CREATE TABLE tb_nivel_formacao ( pk_nivel_formacao int NOT
NULL AUTO_INCREMENT, descricao varchar(40) NOT NULL, UNIQUE INDEX
uq_nivel_formacao_desc (descricao), CONSTRAINT pk_nivel_formacao PRIMARY KEY
(pk_nivel_formacao) );
```

```
-- Table: tb_nivel_risco_evasao CREATE TABLE tb_nivel_risco_evasao ( pk_nivel_risco
int NOT NULL AUTO_INCREMENT, descricao varchar(30) NOT NULL, ordem tinyint UNSIGNED
NOT NULL, UNIQUE INDEX uq_nivel_risco_desc (descricao), UNIQUE INDEX
uq_nivel_risco_ordem (ordem), CHECK (CHAR_LENGTH(descricao) > 0), CONSTRAINT
pk_nivel_risco_evasao PRIMARY KEY (pk_nivel_risco) );
```

```
-- Table: tb_nota_atividade CREATE TABLE tb_nota_atividade ( fk_matricula int NOT NULL,
fk_atividade_professor int NOT NULL, valor_nota decimal(4,2) NULL, entregue_no_prazo
bool NOT NULL DEFAULT true, criado_em timestamp NOT NULL DEFAULT current_timestamp,
atualizado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CHECK (( valor_nota IS NULL OR ( valor_nota >= 0 AND valor_nota <= 10 ) )), CONSTRAINT
pk_nota_atividade PRIMARY KEY (fk_matricula,fk_atividade_professor) );
```

```
-- Table: tb_pagamento CREATE TABLE tb_pagamento ( pk_pagamento int NOT NULL
AUTO_INCREMENT, fk_contrato_financeiro int NOT NULL, fk_status_pagamento int NOT NULL,
valor_pago decimal(10,2) NULL, valor_original decimal(10,2) NOT NULL, data_vencimento
date NOT NULL, data_pagamento date NULL, mes_referencia tinyint UNSIGNED NOT NULL,
ano_referencia year NOT NULL, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
atualizado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE INDEX uq_pagamento_mes (fk_contrato_financeiro,mes_referencia,ano_referencia),
CHECK (( mes_referencia BETWEEN 1 AND 12 )), CHECK (( valor_pago IS NULL OR valor_pago
>= 0 )), CHECK (data_vencimento BETWEEN '2000-01-01' AND '2099-12-31'), CHECK
(ano_referencia BETWEEN 2000 AND 2099), CHECK (data_pagamento IS NULL OR
data_pagamento >= '2000-01-01'), CHECK (valor_original > 0), CHECK (data_pagamento IS
NULL OR data_pagamento >= data_vencimento - INTERVAL 30 DAY), CONSTRAINT pk_pagamento
PRIMARY KEY (pk_pagamento) );
```

```
-- Table: tb_pessoa CREATE TABLE tb_pessoa ( pk_pessoa int NOT NULL AUTO_INCREMENT,
cpf char(11) NOT NULL, rg varchar(11) NULL, nome varchar(30) NOT NULL, sobrenome
varchar(40) NOT NULL, sexo enum('masculino','feminino') NOT NULL DEFAULT 'feminino',
data_nascimento date NOT NULL, nacionalidade enum('brasileiro','estrangeiro') NOT NULL
DEFAULT 'brasileiro', raca enum('branca','negra','parda','amarela') NOT NULL DEFAULT
'parda', criado_em timestamp NOT NULL DEFAULT current_timestamp, atualizado_em
timestamp NOT NULL DEFAULT current_timestamp ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX
uq_pessoa_cpf (cpf), CHECK (CHAR_LENGTH(cpf) = 11 AND cpf REGEXP '^[0-9]+$'), CHECK
(rg IS NULL OR (CHAR_LENGTH(rg) BETWEEN 5 AND 15 AND rg REGEXP '^[0-9A-Za-z]+$')),
CONSTRAINT pk_pessoa PRIMARY KEY (pk_pessoa) );
```

```
-- Table: tb_presenca_aluno CREATE TABLE tb_presenca_aluno ( pk_presenca int NOT NULL
AUTO_INCREMENT, fk_matricula int NOT NULL, fk_calendario_letivo int NOT NULL,
fk_horario_aula int NOT NULL, presenca bool NOT NULL, justificativa varchar(255) NULL,
UNIQUE INDEX uq_presenca_dia (fk_matricula,fk_calendario_letivo,fk_horario_aula),
```

```
CHECK (justificativa IS NULL OR CHAR_LENGTH(justificativa) > 0), CONSTRAINT
pk_presenca_aluno PRIMARY KEY (pk_presenca) );
```

```
-- Table: tb_professor CREATE TABLE tb_professor ( pk_professor int NOT NULL
AUTO_INCREMENT, fk_funcionario int NOT NULL, fk_tipo_aula int NULL,
utiliza_rubrica_avaliativa bool NOT NULL DEFAULT false, atende_educacao_especial bool
NOT NULL DEFAULT false, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
atualizado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE INDEX uq_professor_funcionario (fk_funcionario), CONSTRAINT pk_professor
PRIMARY KEY (pk_professor) );
```

```
-- Table: tb_profissao CREATE TABLE tb_profissao ( pk_profissao int NOT NULL
AUTO_INCREMENT, profissao varchar(30) NOT NULL, descricao varchar(100) NOT NULL,
criado_em timestamp NOT NULL DEFAULT current_timestamp, UNIQUE INDEX uq_profissao
(profissao), CONSTRAINT tb_profissao_pk PRIMARY KEY (pk_profissao) );
```

```
-- Table: tb_profissao_responsavel CREATE TABLE tb_profissao_responsavel
( fk_responsavel int NOT NULL, fk_profissao int NOT NULL, ativo bool NOT NULL DEFAULT
true, criado_em timestamp NOT NULL DEFAULT current_timestamp, CONSTRAINT
tb_profissao_responsavel_pk PRIMARY KEY (fk_responsavel,fk_profissao) );
```

```
-- Table: tb_responsavel CREATE TABLE tb_responsavel ( pk_responsavel int NOT NULL
AUTO_INCREMENT, fk_pessoa int NOT NULL, criado_em timestamp NOT NULL DEFAULT
current_timestamp, atualizado_em timestamp NOT NULL DEFAULT current_timestamp ON
UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX uq_responsavel_pessoa (fk_pessoa), CONSTRAINT
pk_responsavel PRIMARY KEY (pk_responsavel) );
```

```
-- Table: tb_responsavel_aluno CREATE TABLE tb_responsavel_aluno ( fk_aluno int NOT
NULL, fk_responsavel int NOT NULL, responsavel_principal bool NOT NULL DEFAULT false,
responsavel_financeiro bool NOT NULL DEFAULT false, pode_retirar bool NOT NULL DEFAULT
false, CONSTRAINT pk_responsavel_aluno PRIMARY KEY (fk_aluno,fk_responsavel) );
```

```
-- Table: tb_situacao_boletim CREATE TABLE tb_situacao_boletim ( pk_situacao_boletim
int NOT NULL AUTO_INCREMENT, descricao varchar(30) NOT NULL, ativo bool NOT NULL
DEFAULT TRUE, UNIQUE INDEX uq_situacao_boletim_desc (descricao), CHECK
(CHAR_LENGTH(descricao) > 0), CONSTRAINT pk_situacao_boletim PRIMARY KEY
(pk_situacao_boletim) );
```

```
-- Table: tb_status_alerta CREATE TABLE tb_status_alerta ( pk_status_alerta int NOT
NULL AUTO_INCREMENT, descricao varchar(40) NOT NULL, ativo bool NOT NULL DEFAULT TRUE,
UNIQUE INDEX uq_status_alerta_desc (descricao), CHECK (CHAR_LENGTH(descricao) > 0),
CONSTRAINT pk_status_alerta PRIMARY KEY (pk_status_alerta) );
```

```
-- Table: tb_status_matricula CREATE TABLE tb_status_matricula ( pk_status_matricula
int NOT NULL AUTO_INCREMENT, descricao varchar(30) NOT NULL, UNIQUE INDEX
uq_status_matricula_desc (descricao), CONSTRAINT pk_status_matricula PRIMARY KEY
(pk_status_matricula) );
```

```
-- Table: tb_status_pagamento CREATE TABLE tb_status_pagamento ( pk_status_pagamento
int NOT NULL AUTO_INCREMENT, descricao varchar(30) NOT NULL, ativo bool NOT NULL
DEFAULT TRUE, UNIQUE INDEX uq_status_pagamento_desc (descricao), CHECK
(CHAR_LENGTH(descricao) > 0), CONSTRAINT pk_status_pagamento PRIMARY KEY
(pk_status_pagamento) );
```

```

-- Table: tb_telefone CREATE TABLE tb_telefone ( pk_telefone int NOT NULL
AUTO_INCREMENT,          fk_pessoa          int          NOT          NULL,          categoria
enum('celular','residencial','emergencia') NOT NULL, ddd char(2) NOT NULL, numero
varchar(9) NOT NULL, ativo bool NOT NULL DEFAULT true, principal bool NOT NULL DEFAULT
false, UNIQUE INDEX uq_ddd_telefone (ddd,numero), CHECK (CHAR_LENGTH(ddd) = 2 AND ddd
REGEXP '^[0-9]+$'), CHECK (CHAR_LENGTH(numero) BETWEEN 8 AND 9 AND numero REGEXP '^[0-
9]+$'), CONSTRAINT pk_telefone PRIMARY KEY (pk_telefone) );

-- Table: tb_tipo_atividade CREATE TABLE tb_tipo_atividade ( pk_tipo_atividade int NOT
NULL AUTO_INCREMENT,          descricao          varchar(40)          NOT          NULL,          UNIQUE          INDEX
uq_tipo_atividade_desc          (descricao),          CONSTRAINT          pk_tipo_atividade          PRIMARY          KEY
(pk_tipo_atividade) );

-- Table: tb_tipo_aula CREATE TABLE tb_tipo_aula ( pk_tipo_aula int NOT NULL
AUTO_INCREMENT, descricao varchar(40) NOT NULL, UNIQUE INDEX uq_tipo_aula_desc
(descricao), CONSTRAINT pk_tipo_aula PRIMARY KEY (pk_tipo_aula) );

-- Table: tb_tipo_contrato CREATE TABLE tb_tipo_contrato ( pk_tipo_contrato int NOT
NULL AUTO_INCREMENT,          descricao          varchar(30)          NOT          NULL,          UNIQUE          INDEX
uq_tipo_contrato_desc          (descricao),          CONSTRAINT          pk_tipo_contrato          PRIMARY          KEY
(pk_tipo_contrato) );

-- Table: tb_tipo_escola_anterior CREATE TABLE tb_tipo_escola_anterior
( pk_tipo_escola_anterior int NOT NULL AUTO_INCREMENT, descricao varchar(30) NOT NULL,
UNIQUE INDEX uq_tipo_escola_anterior_desc (descricao), CONSTRAINT
pk_tipo_escola_anterior PRIMARY KEY (pk_tipo_escola_anterior) );

-- Table: tb_tipo_ingresso CREATE TABLE tb_tipo_ingresso ( pk_tipo_ingresso int NOT
NULL AUTO_INCREMENT, descricao varchar(60) NOT NULL, ativo bool NOT NULL DEFAULT true,
UNIQUE INDEX uq_tipo_ingresso_desc (descricao), CONSTRAINT pk_tipo_ingresso PRIMARY
KEY (pk_tipo_ingresso) );

-- Table: tb_tipo_risco_evasao CREATE TABLE tb_tipo_risco_evasao
( pk_tipo_risco_evasao int NOT NULL AUTO_INCREMENT, descricao varchar(60) NOT NULL,
ativo bool NOT NULL DEFAULT TRUE, UNIQUE INDEX uq_tipo_risco_evasao_desc (descricao),
CHECK (CHAR_LENGTH(descricao) > 0), CONSTRAINT pk_tipo_risco_evasao PRIMARY KEY
(pk_tipo_risco_evasao) );

-- Table: tb_tipo_verba CREATE TABLE tb_tipo_verba ( pk_tipo_verba int NOT NULL
AUTO_INCREMENT,          descricao          varchar(60)          NOT          NULL,          natureza_verba
enum('credito','debito') NOT NULL, ativo bool NOT NULL DEFAULT TRUE, UNIQUE INDEX
uq_descricao_verba          (descricao),          CONSTRAINT          tb_tipo_verba_pk          PRIMARY          KEY
(pk_tipo_verba) );

-- Table: tb_turma CREATE TABLE tb_turma ( pk_turma int NOT NULL AUTO_INCREMENT,
fk_ano_letivo int NOT NULL, nome_turma varchar(10) NOT NULL, ano_escolar tinyint
UNSIGNED NOT NULL, periodo enum('manha','tarde') NOT NULL, capacidade_maxima tinyint
UNSIGNED NULL, criado_em timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP, atualizado_em
timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, UNIQUE INDEX
uq_turma_ano (fk_ano_letivo,nome_turma,periodo), CHECK (ano_escolar BETWEEN 1 AND 5),
CHECK (CHAR_LENGTH(nome_turma) > 0), CHECK (capacidade_maxima IS NULL OR
capacidade_maxima >= 1), CONSTRAINT pk_turma PRIMARY KEY (pk_turma) );

```

```

-- Table: tb_verba_folha CREATE TABLE tb_verba_folha ( fk_folha_pagamento int NOT NULL,
fk_tipo_verba int NOT NULL, valor decimal(10,2) NOT NULL, referencia varchar(80) NULL,
UNIQUE INDEX uq_verba_folha (fk_tipo_verba,fk_folha_pagamento), CHECK ( valor >= 0 ),
CONSTRAINT tb_verba_folha_pk PRIMARY KEY (fk_folha_pagamento,fk_tipo_verba) );

-- foreign keys -- Reference: fk_alerta_funcionario (table: tb_alerta_evasao) ALTER
TABLE tb_alerta_evasao ADD CONSTRAINT fk_alerta_funcionario FOREIGN KEY
fk_alerta_funcionario (fk_responsavel_alerta) REFERENCES tb_funcionario
(pk_funcionario) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_alerta_matricula (table: tb_alerta_evasao) ALTER TABLE
tb_alerta_evasao ADD CONSTRAINT fk_alerta_matricula FOREIGN KEY fk_alerta_matricula
(fk_matricula) REFERENCES tb_matricula (pk_matricula) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_alerta_nivel_risco (table: tb_alerta_evasao) ALTER TABLE
tb_alerta_evasao ADD CONSTRAINT fk_alerta_nivel_risco FOREIGN KEY
fk_alerta_nivel_risco (fk_nivel_risco) REFERENCES tb_nivel_risco_evasao
(pk_nivel_risco) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_alerta_status (table: tb_alerta_evasao) ALTER TABLE tb_alerta_evasao
ADD CONSTRAINT fk_alerta_status FOREIGN KEY fk_alerta_status (fk_status_alerta)
REFERENCES tb_status_alerta (pk_status_alerta) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_alerta_tipo_risco (table: tb_alerta_evasao) ALTER TABLE
tb_alerta_evasao ADD CONSTRAINT fk_alerta_tipo_risco FOREIGN KEY fk_alerta_tipo_risco
(fk_tipo_risco_evasao) REFERENCES tb_tipo_risco_evasao (pk_tipo_risco_evasao) ON
DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_aluno_pessoa (table: tb_aluno) ALTER TABLE tb_aluno ADD CONSTRAINT
fk_aluno_pessoa FOREIGN KEY fk_aluno_pessoa (fk_pessoa) REFERENCES tb_pessoa
(pk_pessoa) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_aluno_tipo_escola (table: tb_aluno) ALTER TABLE tb_aluno ADD
CONSTRAINT fk_aluno_tipo_escola FOREIGN KEY fk_aluno_tipo_escola
(fk_tipo_escola_anterior) REFERENCES tb_tipo_escola_anterior (pk_tipo_escola_anterior)
ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_atividade_grade (table: tb_atividade_professor) ALTER TABLE
tb_atividade_professor ADD CONSTRAINT fk_atividade_grade FOREIGN KEY
fk_atividade_grade (fk_grade) REFERENCES tb_grade_curricular (pk_grade) ON DELETE
RESTRICT ON UPDATE CASCADE;

-- Reference: fk_atividade_tipo (table: tb_atividade_professor) ALTER TABLE
tb_atividade_professor ADD CONSTRAINT fk_atividade_tipo FOREIGN KEY fk_atividade_tipo
(fk_tipo_atividade) REFERENCES tb_tipo_atividade (pk_tipo_atividade) ON DELETE
RESTRICT ON UPDATE CASCADE;

-- Reference: fk_boletim_matricula (table: tb_boletim) ALTER TABLE tb_boletim ADD
CONSTRAINT fk_boletim_matricula FOREIGN KEY fk_boletim_matricula (fk_matricula)
REFERENCES tb_matricula (pk_matricula) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_boletim_situacao (table: tb_boletim) ALTER TABLE tb_boletim ADD
CONSTRAINT fk_boletim_situacao FOREIGN KEY fk_boletim_situacao (fk_situacao_boletim)

```

```
REFERENCES tb_situacao_boletim (pk_situacao_boletim) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_calendario_ano (table: tb_calendario_letivo) ALTER TABLE tb_calendario_letivo ADD CONSTRAINT fk_calendario_ano FOREIGN KEY fk_calendario_ano (fk_ano_letivo) REFERENCES tb_ano_letivo (pk_ano_letivo) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_cont_fin_matricula (table: tb_contrato_financeiro) ALTER TABLE tb_contrato_financeiro ADD CONSTRAINT fk_cont_fin_matricula FOREIGN KEY fk_cont_fin_matricula (fk_matricula) REFERENCES tb_matricula (pk_matricula) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_contrato_func (table: tb_contrato) ALTER TABLE tb_contrato ADD CONSTRAINT fk_contrato_func FOREIGN KEY fk_contrato_func (fk_funcionario) REFERENCES tb_funcionario (pk_funcionario) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_contrato_tipo (table: tb_contrato) ALTER TABLE tb_contrato ADD CONSTRAINT fk_contrato_tipo FOREIGN KEY fk_contrato_tipo (fk_tipo_contrato) REFERENCES tb_tipo_contrato (pk_tipo_contrato) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_email_pessoa (table: tb_email) ALTER TABLE tb_email ADD CONSTRAINT fk_email_pessoa FOREIGN KEY fk_email_pessoa (fk_pessoa) REFERENCES tb_pessoa (pk_pessoa) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_endereco_pessoa (table: tb_endereco) ALTER TABLE tb_endereco ADD CONSTRAINT fk_endereco_pessoa FOREIGN KEY fk_endereco_pessoa (fk_pessoa) REFERENCES tb_pessoa (pk_pessoa) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_evento_calendario (table: tb_evento_escolar) ALTER TABLE tb_evento_escolar ADD CONSTRAINT fk_evento_calendario FOREIGN KEY fk_evento_calendario (fk_calendario) REFERENCES tb_calendario_letivo (pk_calendario) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_folha_contrato (table: tb_folha_pagamento) ALTER TABLE tb_folha_pagamento ADD CONSTRAINT fk_folha_contrato FOREIGN KEY fk_folha_contrato (fk_contrato) REFERENCES tb_contrato (pk_contrato) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_formacao_nivel (table: tb_formacao_professor) ALTER TABLE tb_formacao_professor ADD CONSTRAINT fk_formacao_nivel FOREIGN KEY fk_formacao_nivel (fk_nivel_formacao) REFERENCES tb_nivel_formacao (pk_nivel_formacao) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_formacao_professor (table: tb_formacao_professor) ALTER TABLE tb_formacao_professor ADD CONSTRAINT fk_formacao_professor FOREIGN KEY fk_formacao_professor (fk_professor) REFERENCES tb_professor (pk_professor) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_funcionario_pessoa (table: tb_funcionario) ALTER TABLE tb_funcionario ADD CONSTRAINT fk_funcionario_pessoa FOREIGN KEY fk_funcionario_pessoa (fk_pessoa) REFERENCES tb_pessoa (pk_pessoa) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```

-- Reference: fk_grade_materia (table: tb_grade_curricular) ALTER TABLE
tb_grade_curricular ADD CONSTRAINT fk_grade_materia FOREIGN KEY fk_grade_materia
(fk_materia) REFERENCES tb_materia (pk_materia) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_grade_professor (table: tb_grade_curricular) ALTER TABLE
tb_grade_curricular ADD CONSTRAINT fk_grade_professor FOREIGN KEY fk_grade_professor
(fk_professor) REFERENCES tb_professor (pk_professor) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_grade_turma (table: tb_grade_curricular) ALTER TABLE
tb_grade_curricular ADD CONSTRAINT fk_grade_turma FOREIGN KEY fk_grade_turma
(fk_turma) REFERENCES tb_turma (pk_turma) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_hist_cargo_cargo (table: tb_historico_cargo) ALTER TABLE
tb_historico_cargo ADD CONSTRAINT fk_hist_cargo_cargo FOREIGN KEY fk_hist_cargo_cargo
(fk_cargo) REFERENCES tb_cargo (pk_cargo) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_hist_cargo_contrato (table: tb_historico_cargo) ALTER TABLE
tb_historico_cargo ADD CONSTRAINT fk_hist_cargo_contrato FOREIGN KEY
fk_hist_cargo_contrato (fk_contrato) REFERENCES tb_contrato (pk_contrato) ON DELETE
RESTRICT ON UPDATE CASCADE;

-- Reference: fk_hist_salario_contrato (table: tb_historico_salario) ALTER TABLE
tb_historico_salario ADD CONSTRAINT fk_hist_salario_contrato FOREIGN KEY
fk_hist_salario_contrato (fk_contrato) REFERENCES tb_contrato (pk_contrato) ON DELETE
RESTRICT ON UPDATE CASCADE;

-- Reference: fk_horario_grade (table: tb_horario_aula) ALTER TABLE tb_horario_aula
ADD CONSTRAINT fk_horario_grade FOREIGN KEY fk_horario_grade (fk_grade) REFERENCES
tb_grade_curricular (pk_grade) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_materia_area (table: tb_materia) ALTER TABLE tb_materia ADD
CONSTRAINT fk_materia_area FOREIGN KEY fk_materia_area (fk_area_conhecimento)
REFERENCES tb_area_conhecimento (pk_area_conhecimento) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_matricula_aluno (table: tb_matricula) ALTER TABLE tb_matricula ADD
CONSTRAINT fk_matricula_aluno FOREIGN KEY fk_matricula_aluno (fk_aluno) REFERENCES
tb_aluno (pk_aluno) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_matricula_status (table: tb_matricula) ALTER TABLE tb_matricula ADD
CONSTRAINT fk_matricula_status FOREIGN KEY fk_matricula_status (fk_status_matricula)
REFERENCES tb_status_matricula (pk_status_matricula) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_matricula_tipo_ingresso (table: tb_matricula) ALTER TABLE
tb_matricula ADD CONSTRAINT fk_matricula_tipo_ingresso FOREIGN KEY
fk_matricula_tipo_ingresso (fk_tipo_ingresso) REFERENCES tb_tipo_ingresso
(pk_tipo_ingresso) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_matricula_turma (table: tb_matricula) ALTER TABLE tb_matricula ADD
CONSTRAINT fk_matricula_turma FOREIGN KEY fk_matricula_turma (fk_turma) REFERENCES
tb_turma (pk_turma) ON DELETE RESTRICT ON UPDATE CASCADE;

```

```

-- Reference: fk_nota_atividade_prof (table: tb_nota_atividade) ALTER TABLE
tb_nota_atividade ADD CONSTRAINT fk_nota_atividade_prof FOREIGN KEY
fk_nota_atividade_prof (fk_atividade_professor) REFERENCES tb_atividade_professor
(pk_atividade_professor) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_nota_matricula (table: tb_nota_atividade) ALTER TABLE
tb_nota_atividade ADD CONSTRAINT fk_nota_matricula FOREIGN KEY fk_nota_matricula
(fk_matricula) REFERENCES tb_matricula (pk_matricula) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_pagamento_contrato_fin (table: tb_pagamento) ALTER TABLE tb_pagamento
ADD CONSTRAINT fk_pagamento_contrato_fin FOREIGN KEY fk_pagamento_contrato_fin
(fk_contrato_financeiro) REFERENCES tb_contrato_financeiro (pk_contrato_financeiro) ON
DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_pagamento_status (table: tb_pagamento) ALTER TABLE tb_pagamento ADD
CONSTRAINT fk_pagamento_status FOREIGN KEY fk_pagamento_status (fk_status_pagamento)
REFERENCES tb_status_pagamento (pk_status_pagamento) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_presenca_calendario (table: tb_presenca_aluno) ALTER TABLE
tb_presenca_aluno ADD CONSTRAINT fk_presenca_calendario FOREIGN KEY
fk_presenca_calendario (fk_calendario_letivo) REFERENCES tb_calendario_letivo
(pk_calendario) ON DELETE RESTRICT ON UPDATE CASCADE;

-- Reference: fk_presenca_horario (table: tb_presenca_aluno) ALTER TABLE
tb_presenca_aluno ADD CONSTRAINT fk_presenca_horario FOREIGN KEY fk_presenca_horario
(fk_horario_aula) REFERENCES tb_horario_aula (pk_horario_aula) ON DELETE RESTRICT ON
UPDATE CASCADE;

-- Reference: fk_presenca_matricula (table: tb_presenca_aluno) ALTER TABLE
tb_presenca_aluno ADD CONSTRAINT fk_presenca_matricula FOREIGN KEY
fk_presenca_matricula (fk_matricula) REFERENCES tb_matricula (pk_matricula) ON DELETE
RESTRICT ON UPDATE CASCADE;

-- Reference: fk_professor_funcionario (table: tb_professor) ALTER TABLE tb_professor
ADD CONSTRAINT fk_professor_funcionario FOREIGN KEY fk_professor_funcionario
(fk_funcionario) REFERENCES tb_funcionario (pk_funcionario) ON DELETE RESTRICT ON
UPDATE CASCADE;

-- Reference: fk_professor_tipo_aula (table: tb_professor) ALTER TABLE tb_professor
ADD CONSTRAINT fk_professor_tipo_aula FOREIGN KEY fk_professor_tipo_aula
(fk_tipo_aula) REFERENCES tb_tipo_aula (pk_tipo_aula) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_profissao (table: tb_profissao_responsavel) ALTER TABLE
tb_profissao_responsavel ADD CONSTRAINT fk_profissao FOREIGN KEY fk_profissao
(fk_profissao) REFERENCES tb_profissao (pk_profissao) ON DELETE RESTRICT ON UPDATE
CASCADE;

-- Reference: fk_resp_aluno_aluno (table: tb_responsavel_aluno) ALTER TABLE
tb_responsavel_aluno ADD CONSTRAINT fk_resp_aluno_aluno FOREIGN KEY
fk_resp_aluno_aluno (fk_aluno) REFERENCES tb_aluno (pk_aluno) ON DELETE RESTRICT ON
UPDATE CASCADE;

```

```
-- Reference: fk_resp_aluno_responsavel (table: tb_responsavel_aluno) ALTER TABLE
tb_responsavel_aluno ADD CONSTRAINT fk_resp_aluno_responsavel FOREIGN KEY
fk_resp_aluno_responsavel (fk_responsavel) REFERENCES tb_responsavel (pk_responsavel)
ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_responsavel (table: tb_profissao_responsavel) ALTER TABLE
tb_profissao_responsavel ADD CONSTRAINT fk_responsavel FOREIGN KEY fk_responsavel
(fk_responsavel) REFERENCES tb_responsavel (pk_responsavel) ON DELETE RESTRICT ON
UPDATE CASCADE;
```

```
-- Reference: fk_responsavel_pessoa (table: tb_responsavel) ALTER TABLE tb_responsavel
ADD CONSTRAINT fk_responsavel_pessoa FOREIGN KEY fk_responsavel_pessoa (fk_pessoa)
REFERENCES tb_pessoa (pk_pessoa) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_telefone_pessoa (table: tb_telefone) ALTER TABLE tb_telefone ADD
CONSTRAINT fk_telefone_pessoa FOREIGN KEY fk_telefone_pessoa (fk_pessoa) REFERENCES
tb_pessoa (pk_pessoa) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: fk_turma_ano_letivo (table: tb_turma) ALTER TABLE tb_turma ADD
CONSTRAINT fk_turma_ano_letivo FOREIGN KEY fk_turma_ano_letivo (fk_ano_letivo)
REFERENCES tb_ano_letivo (pk_ano_letivo) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: tb_verba_folha_tb_folha_pagamento (table: tb_verba_folha) ALTER TABLE
tb_verba_folha ADD CONSTRAINT tb_verba_folha_tb_folha_pagamento FOREIGN KEY
tb_verba_folha_tb_folha_pagamento (fk_folha_pagamento) REFERENCES tb_folha_pagamento
(pk_folha_pagamento) ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- Reference: tb_verba_folha_tb_tipo_verba (table: tb_verba_folha) ALTER TABLE
tb_verba_folha ADD CONSTRAINT tb_verba_folha_tb_tipo_verba FOREIGN KEY
tb_verba_folha_tb_tipo_verba (fk_tipo_verba) REFERENCES tb_tipo_verba (pk_tipo_verba)
ON DELETE RESTRICT ON UPDATE CASCADE;
```

```
-- TRIGGERS - Aurora EduTech v06 DELIMITER //
```

```
CREATE TRIGGER trg_contrato_validar_insert BEFORE INSERT ON tb_contrato FOR EACH ROW
BEGIN DECLARE v_conflitos INT; DECLARE v_ativo BOOL;
```

```
--          1.          Funcionário          deve          estar          ativo
SELECT          ativo          INTO          v_ativo
FROM          tb_funcionario
WHERE          pk_funcionario          =          NEW.fk_funcionario;

IF          v_ativo          IS          NULL          OR          v_ativo          =          FALSE          THEN
    SIGNAL          SQLSTATE          '45000'
    SET          MESSAGE_TEXT          =          'Funcionario inativo nao pode ter contrato';
END          IF;
```

```
--          2.          Sem          sobreposição          de          períodos          para          o          mesmo          funcionário
SELECT          COUNT(*)          INTO          v_conflitos
FROM          tb_contrato
WHERE          fk_funcionario          =          NEW.fk_funcionario
AND          NEW.data_inicio          <=          COALESCE(data_fim,          '9999-12-31')
AND          COALESCE(NEW.data_fim,          '9999-12-31')          >=          data_inicio;
```

```

IF          v_conflitos          >          0          THEN
    SIGNAL          SQLSTATE          '45000'
        SET MESSAGE_TEXT = 'Funcionario ja possui contrato com periodo sobreposto';
END          IF;

--          3.          Consistência          status          x          data_fim
IF NEW.data_fim IS NOT NULL AND NEW.status_contrato = 'ativo' THEN
    SIGNAL          SQLSTATE          '45000'
        SET MESSAGE_TEXT = 'Contrato com data_fim preenchida nao pode ter status ativo';
END          IF;
IF NEW.data_fim IS NULL AND NEW.status_contrato != 'ativo' THEN
    SIGNAL          SQLSTATE          '45000'
        SET MESSAGE_TEXT = 'Contrato sem data_fim deve ter status ativo';
END          IF;

```

```
END//
```

```
CREATE TRIGGER trg_contrato_validar_update BEFORE UPDATE ON tb_contrato FOR EACH ROW
BEGIN DECLARE v_conflitos INT; DECLARE v_ativo BOOL;
```

```

--          1.          Funcionário ativo (validar apenas se o FK mudou)
IF NEW.fk_funcionario != OLD.fk_funcionario THEN
    SELECT          ativo          INTO          v_ativo
    FROM
    WHERE
        pk_funcionario = NEW.fk_funcionario;

    IF v_ativo IS NULL OR v_ativo = FALSE THEN
        SIGNAL          SQLSTATE          '45000'
            SET MESSAGE_TEXT = 'Funcionario inativo nao pode ter contrato';
    END          IF;
END          IF;

```

```

--          2.          Sobreposição (validar apenas se datas ou funcionário mudaram)
IF NEW.fk_funcionario != OLD.fk_funcionario
OR NEW.data_inicio != OLD.data_inicio
OR NOT (NEW.data_fim <=> OLD.data_fim) THEN

    SELECT          COUNT(*)          INTO          v_conflitos
    FROM
    WHERE
        fk_funcionario = NEW.fk_funcionario
        AND pk_contrato != NEW.pk_contrato
        AND NEW.data_inicio <= COALESCE(data_fim, '9999-12-31')
        AND COALESCE(NEW.data_fim, '9999-12-31') >= data_inicio;

    IF          v_conflitos          >          0          THEN
        SIGNAL          SQLSTATE          '45000'
            SET MESSAGE_TEXT = 'Funcionario ja possui contrato com periodo sobreposto';
    END          IF;

```

```

END                                                                                                     IF;

-- 3. Consistência status x data_fim (validar se algum dos dois mudou)
IF      NOT      (NEW.data_fim      <=>      OLD.data_fim)
OR      NEW.status_contrato      !=      OLD.status_contrato      THEN

    IF      NEW.data_fim      IS      NOT      NULL      AND      NEW.status_contrato      =      'ativo'      THEN
        SIGNAL      SQLSTATE      '45000'
        SET MESSAGE_TEXT = 'Contrato com data_fim preenchida nao pode ter status ativo';
    END
    IF      NEW.data_fim      IS      NULL      AND      NEW.status_contrato      !=      'ativo'      THEN
        SIGNAL      SQLSTATE      '45000'
        SET MESSAGE_TEXT = 'Contrato sem data_fim deve ter status ativo';
    END
END
IF;

END//

-- -----
GRUPO 2 - tb_historico_cargo -- Regras: -- • Cargo precisa estar ativo para ser
atribuído -- • Datas do cargo dentro do período do contrato -- • Sem sobreposição de
cargos          no          mesmo          contrato          --
-----

CREATE TRIGGER trg_hist_cargo_validar_insert BEFORE INSERT ON tb_historico_cargo FOR
EACH ROW BEGIN DECLARE v_conflitos INT; DECLARE v_cargo_ativo BOOL; DECLARE
v_contrato_inicio DATE; DECLARE v_contrato_fim DATE;

-- 1. Cargo deve estar ativo
SELECT      ativo          INTO      v_cargo_ativo
FROM      tb_cargo
WHERE      pk_cargo      =      NEW.fk_cargo;

IF      v_cargo_ativo      IS      NULL      OR      v_cargo_ativo      =      FALSE      THEN
    SIGNAL      SQLSTATE      '45000'
    SET MESSAGE_TEXT = 'Cargo desativado nao pode ser atribuido';
END
IF;

-- 2. Datas dentro do contrato
SELECT      data_inicio,      data_fim
INTO      v_contrato_inicio,      v_contrato_fim
FROM      tb_contrato
WHERE      pk_contrato      =      NEW.fk_contrato;

IF      NEW.data_inicio      <      v_contrato_inicio      THEN
    SIGNAL      SQLSTATE      '45000'
    SET MESSAGE_TEXT = 'data_inicio do cargo nao pode ser anterior ao inicio do contrato';
END
IF;
IF      v_contrato_fim      IS      NOT      NULL      AND      NEW.data_inicio      >      v_contrato_fim      THEN
    SIGNAL      SQLSTATE      '45000'

```

```

        SET MESSAGE_TEXT = 'data_inicio do cargo nao pode ser posterior ao fim do contrato';
END
IF NEW.data_fim IS NOT NULL AND v_contrato_fim IS NOT NULL
AND NEW.data_fim > v_contrato_fim THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'data_fim do cargo nao pode ultrapassar o fim do contrato';
END

```

```

--      3.      Sem      sobreposição      no      mesmo      contrato
SELECT      COUNT(*)      INTO      v_conflitos
FROM      tb_historico_cargo
WHERE      fk_contrato      =      NEW.fk_contrato
AND      NEW.data_inicio      <=      COALESCE(data_fim, '9999-12-31')
AND      COALESCE(NEW.data_fim, '9999-12-31')      >=      data_inicio;

IF      v_conflitos      >      0      THEN
    SIGNAL      SQLSTATE      '45000'
    SET      MESSAGE_TEXT      =      'Contrato ja possui cargo com periodo sobreposto';
END

```

END//

```

CREATE TRIGGER trg_hist_cargo_validar_update BEFORE UPDATE ON tb_historico_cargo FOR
EACH ROW BEGIN DECLARE v_conflitos INT; DECLARE v_cargo_ativo BOOL; DECLARE
v_contrato_inicio DATE; DECLARE v_contrato_fim DATE;

```

```

IF      NEW.fk_cargo      !=      OLD.fk_cargo      THEN
    SELECT      ativo      INTO      v_cargo_ativo
    FROM      tb_cargo
    WHERE      pk_cargo      =      NEW.fk_cargo;

    IF      v_cargo_ativo      IS      NULL      OR      v_cargo_ativo      =      FALSE      THEN
        SIGNAL      SQLSTATE      '45000'
        SET      MESSAGE_TEXT      =      'Cargo desativado nao pode ser atribuido';
    END
END

```

```

IF      NEW.fk_contrato      !=      OLD.fk_contrato
OR      NEW.data_inicio      !=      OLD.data_inicio
OR      NOT      (NEW.data_fim      <=>      OLD.data_fim)      THEN

    SELECT      data_inicio,      data_fim
    INTO      v_contrato_inicio,      v_contrato_fim
    FROM      tb_contrato
    WHERE      pk_contrato      =      NEW.fk_contrato;

    IF      NEW.data_inicio      <      v_contrato_inicio      THEN
        SIGNAL      SQLSTATE      '45000'
        SET MESSAGE_TEXT = 'data_inicio do cargo nao pode ser anterior ao inicio do

```

```

contrato';
END
IF v_contrato_fim IS NOT NULL AND NEW.data_inicio > v_contrato_fim THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'data_inicio do cargo nao pode ser posterior ao fim do
contrato';
END
IF NEW.data_fim IS NOT NULL AND v_contrato_fim IS NOT NULL
AND NEW.data_fim > v_contrato_fim THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'data_fim do cargo nao pode ultrapassar o fim do contrato';
END
IF;

SELECT COUNT(*) INTO v_conflitos
FROM tb_historico_cargo
WHERE fk_contrato = NEW.fk_contrato
AND pk_historico_cargo != NEW.pk_historico_cargo
AND NEW.data_inicio <= COALESCE(data_fim, '9999-12-31')
AND COALESCE(NEW.data_fim, '9999-12-31') >= data_inicio;

IF v_conflitos > 0 THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Contrato ja possui cargo com periodo sobreposto';
END
IF;
END
IF;

```

END//

```

-- -----
GRUPO 3 - tb_historico_salario -- Regras: -- • Datas do salário dentro do periodo do
contrato -- • Sem sobreposição de salários no mesmo contrato --

```

```

CREATE TRIGGER trg_hist_salario_validar_insert BEFORE INSERT ON tb_historico_salario
FOR EACH ROW BEGIN DECLARE v_conflitos INT; DECLARE v_contrato_inicio DATE; DECLARE
v_contrato_fim DATE;

```

```

SELECT data_inicio, data_fim
INTO v_contrato_inicio, v_contrato_fim
FROM tb_contrato
WHERE pk_contrato = NEW.fk_contrato;

```

```

-- 1. Datas dentro do contrato
IF NEW.data_inicio < v_contrato_inicio THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'data_inicio do salario nao pode ser anterior ao inicio do
contrato';
END
IF;
IF v_contrato_fim IS NOT NULL AND NEW.data_inicio > v_contrato_fim THEN
    SIGNAL SQLSTATE '45000'

```

```

        SET MESSAGE_TEXT = 'data_inicio do salario nao pode ser posterior ao fim do contrato';
END
IF NEW.data_fim IS NOT NULL AND v_contrato_fim IS NOT NULL
AND NEW.data_fim > v_contrato_fim THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'data_fim do salario nao pode ultrapassar o fim do contrato';
END

-- 2. Sem sobreposição no mesmo contrato
SELECT COUNT(*) INTO v_conflitos
FROM tb_historico_salario
WHERE fk_contrato = NEW.fk_contrato
AND NEW.data_inicio <= COALESCE(data_fim, '9999-12-31')
AND COALESCE(NEW.data_fim, '9999-12-31') >= data_inicio;

IF v_conflitos > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Contrato ja possui salario com periodo sobreposto';
END

END//

CREATE TRIGGER trg_hist_salario_validar_update BEFORE UPDATE ON tb_historico_salario
FOR EACH ROW BEGIN DECLARE v_conflitos INT; DECLARE v_contrato_inicio DATE; DECLARE
v_contrato_fim DATE;

IF NEW.fk_contrato != OLD.fk_contrato
OR NEW.data_inicio != OLD.data_inicio
OR NOT (NEW.data_fim <=> OLD.data_fim) THEN

    SELECT data_inicio, data_fim
    INTO v_contrato_inicio, v_contrato_fim
    FROM tb_contrato
    WHERE pk_contrato = NEW.fk_contrato;

    IF NEW.data_inicio < v_contrato_inicio THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'data_inicio do salario nao pode ser anterior ao inicio do
contrato';
    END
    IF v_contrato_fim IS NOT NULL AND NEW.data_inicio > v_contrato_fim THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'data_inicio do salario nao pode ser posterior ao fim do
contrato';
    END
    IF NEW.data_fim IS NOT NULL AND v_contrato_fim IS NOT NULL
    AND NEW.data_fim > v_contrato_fim THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'data_fim do salario nao pode ultrapassar o fim do contrato';
    END
END

```

```

SELECT          COUNT(*)          INTO          v_conflitos
FROM
WHERE    fk_contrato              =          tb_historico_salario
AND      pk_historico_salario      !=          NEW.fk_contrato
AND      NEW.data_inicio           <=          COALESCE(data_fim,          '9999-12-31')
AND      COALESCE(NEW.data_fim,    '9999-12-31') >=          data_inicio;

IF          v_conflitos          >          0          THEN
    SIGNAL          SQLSTATE          '45000'
    SET MESSAGE_TEXT = 'Contrato ja possui salario com periodo sobreposto';
END          IF;
END          IF;

END//

-- -----
GRUPO 4 - tb_matricula (RN015 - capacidade máxima da turma) -- Regras: -- • Turmas com
capacidade_maxima definida não podem ultrapassar o limite -- de matrículas com status
'ativa' -- • capacidade_maxima NULL = sem limite --
-----

CREATE TRIGGER trg_matricula_capacidade_insert BEFORE INSERT ON tb_matricula FOR EACH
ROW BEGIN DECLARE v_capacidade TINYINT UNSIGNED; DECLARE v_ocupacao INT; DECLARE
v_pk_ativa INT;

SELECT          capacidade_maxima          INTO          v_capacidade
FROM
WHERE    pk_turma              =          NEW.fk_turma;

IF          v_capacidade          IS          NOT          NULL          THEN
    SELECT          pk_status_matricula          INTO          v_pk_ativa
    FROM          tb_status_matricula
    WHERE          descricao          =          'ativa'
    LIMIT          1;

    SELECT          COUNT(*)          INTO          v_ocupacao
    FROM          tb_matricula
    WHERE    fk_turma              =          NEW.fk_turma
    AND      fk_status_matricula    =          v_pk_ativa;

    IF NEW.fk_status_matricula = v_pk_ativa AND v_ocupacao >= v_capacidade THEN
        SIGNAL          SQLSTATE          '45000'
        SET MESSAGE_TEXT = 'Turma atingiu capacidade maxima de alunos ativos';
    END          IF;
END          IF;

END//

```

```
CREATE TRIGGER trg_matricula_capacidade_update BEFORE UPDATE ON tb_matricula FOR EACH
ROW BEGIN DECLARE v_capacidade TINYINT UNSIGNED; DECLARE v_ocupacao INT; DECLARE
v_pk_ativa INT;
```

```
IF NEW.fk_turma != OLD.fk_turma
OR NEW.fk_status_matricula != OLD.fk_status_matricula THEN
```

```
SELECT capacidade_maxima INTO v_capacidade
FROM tb_turma
WHERE pk_turma = NEW.fk_turma;
```

```
IF v_capacidade IS NOT NULL THEN
SELECT pk_status_matricula INTO v_pk_ativa
FROM tb_status_matricula
WHERE descricao = 'ativa'
LIMIT 1;
```

```
SELECT COUNT(*) INTO v_ocupacao
FROM tb_matricula
WHERE fk_turma = NEW.fk_turma
AND fk_status_matricula = v_pk_ativa
AND pk_matricula != NEW.pk_matricula;
```

```
IF NEW.fk_status_matricula = v_pk_ativa AND v_ocupacao >= v_capacidade THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Turma atingiu capacidade maxima de alunos ativos';
```

```
END IF;
```

```
END IF;
```

```
END IF;
```

```
END//
```

```
-- -----
GRUPO 5 - tb_ano_letivo -- Regra: apenas um ano letivo pode estar ativo (ativo = TRUE)
por vez -- -----
```

```
CREATE TRIGGER trg_ano_letivo_unico_ativo_insert BEFORE INSERT ON tb_ano_letivo FOR
EACH ROW BEGIN DECLARE v_count INT;
```

```
IF NEW.ativo = TRUE THEN
SELECT COUNT(*) INTO v_count
FROM tb_ano_letivo
WHERE ativo = TRUE;
```

```
IF v_count > 0 THEN
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Ja existe um ano letivo ativo. Desative-o antes de ativar
outro.';
```

```
END IF;
```

```
END                                                                                                     IF;
```

```
END//
```

```
CREATE TRIGGER trg_ano_letivo_unico_ativo_update BEFORE UPDATE ON tb_ano_letivo FOR  
EACH ROW BEGIN DECLARE v_count INT;
```

```
IF      NEW.ativo      =      TRUE      AND      OLD.ativo      =      FALSE      THEN  
  SELECT      COUNT(*)      INTO      v_count  
  FROM  
  WHERE      ativo  
  AND      pk_ano_letivo      !=      NEW.pk_ano_letivo;
```

```
  IF      v_count      >      0      THEN  
    SIGNAL      SQLSTATE      '45000'  
    SET MESSAGE_TEXT = 'Ja existe um ano letivo ativo. Desative-o antes de ativar  
outro.';  
  END  
END                                                                                                     IF;
```

```
END//
```

```
--      -----      --  
GRUPO 6 - tb_email -- Regra: cada pessoa pode ter no máximo um e-mail marcado como  
principal      --  
-----
```

```
CREATE TRIGGER trg_email_principal_insert BEFORE INSERT ON tb_email FOR EACH ROW BEGIN  
DECLARE v_count INT;
```

```
IF      NEW.principal      =      TRUE      THEN  
  SELECT      COUNT(*)      INTO      v_count  
  FROM      tb_email  
  WHERE      fk_pessoa      =      NEW.fk_pessoa  
  AND      principal      =      TRUE;
```

```
  IF      v_count      >      0      THEN  
    SIGNAL      SQLSTATE      '45000'  
    SET MESSAGE_TEXT = 'Esta pessoa ja possui um e-mail principal cadastrado';  
  END  
END                                                                                                     IF;
```

```
END//
```

```
CREATE TRIGGER trg_email_principal_update BEFORE UPDATE ON tb_email FOR EACH ROW BEGIN  
DECLARE v_count INT;
```

```

IF      NEW.principal      =      TRUE      AND      OLD.principal      =      FALSE      THEN
    SELECT                  COUNT(*)          INTO          v_count
    FROM
    WHERE                  fk_pessoa          =          NEW.fk_pessoa
    AND                  principal          =          TRUE
    AND                  pk_email          !=          NEW.pk_email;

    IF          v_count          >          0          THEN
        SIGNAL          SQLSTATE          '45000'
        SET MESSAGE_TEXT = 'Esta pessoa ja possui um e-mail principal cadastrado';
    END
END
IF;
END
IF;

```

END//

```

-- ----- --
GRUPO 7 - tb_telefone -- Regra: cada pessoa pode ter no máximo um telefone marcado
como                  principal
-- -----

```

```

CREATE TRIGGER trg_telefone_principal_insert BEFORE INSERT ON tb_telefone FOR EACH ROW
BEGIN DECLARE v_count INT;

```

```

IF      NEW.principal      =      TRUE      THEN
    SELECT                  COUNT(*)          INTO          v_count
    FROM
    WHERE                  fk_pessoa          =          NEW.fk_pessoa
    AND                  principal          =          TRUE;

    IF          v_count          >          0          THEN
        SIGNAL          SQLSTATE          '45000'
        SET MESSAGE_TEXT = 'Esta pessoa ja possui um telefone principal cadastrado';
    END
END
IF;
END
IF;

```

END//

```

CREATE TRIGGER trg_telefone_principal_update BEFORE UPDATE ON tb_telefone FOR EACH ROW
BEGIN DECLARE v_count INT;

```

```

IF      NEW.principal      =      TRUE      AND      OLD.principal      =      FALSE      THEN
    SELECT                  COUNT(*)          INTO          v_count
    FROM
    WHERE                  fk_pessoa          =          NEW.fk_pessoa
    AND                  principal          =          TRUE
    AND                  pk_telefone          !=          NEW.pk_telefone;

    IF          v_count          >          0          THEN
        SIGNAL          SQLSTATE          '45000'

```

```

        SET MESSAGE_TEXT = 'Esta pessoa ja possui um telefone principal cadastrado';
    END IF;
END IF;

```

```
END//
```

```

-- -----
GRUPO 8 - tb_responsavel_aluno -- Regra: cada aluno pode ter no máximo um responsável
marcado                        como principal
-- -----

```

```
CREATE TRIGGER trg_resp_aluno_principal_insert BEFORE INSERT ON tb_responsavel_aluno
FOR EACH ROW BEGIN DECLARE v_count INT;
```

```

IF      NEW.responsavel_principal = TRUE THEN
    SELECT      COUNT(*) INTO v_count
    FROM        tb_responsavel_aluno
    WHERE       fk_aluno = NEW.fk_aluno
    AND         responsavel_principal = TRUE;

    IF      v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Este aluno ja possui um responsavel principal cadastrado';
    END IF;
END IF;

```

```
END//
```

```
CREATE TRIGGER trg_resp_aluno_principal_update BEFORE UPDATE ON tb_responsavel_aluno
FOR EACH ROW BEGIN DECLARE v_count INT;
```

```

IF      NEW.responsavel_principal = TRUE AND OLD.responsavel_principal = FALSE THEN
    SELECT      COUNT(*) INTO v_count
    FROM        tb_responsavel_aluno
    WHERE       fk_aluno = NEW.fk_aluno
    AND         responsavel_principal = TRUE
    AND         fk_responsavel != NEW.fk_responsavel;

    IF      v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Este aluno ja possui um responsavel principal cadastrado';
    END IF;
END IF;

```

```
END//
```

```

-- -----
GRUPO 9 - tb_presenca_aluno -- Regra: o horário de aula registrado deve pertencer à

```

```

mesma      turma      --      da      matricula      do      aluno      --

```

```

CREATE TRIGGER trg_presenca_valida_turma_insert BEFORE INSERT ON tb_presenca_aluno FOR
EACH ROW BEGIN DECLARE v_turma_matricula INT; DECLARE v_turma_horario INT;

```

```

SELECT          fk_turma          INTO          v_turma_matricula
FROM
WHERE          pk_matricula          =          NEW.fk_matricula;

```

```

SELECT          gc.fk_turma          INTO          v_turma_horario
FROM          tb_horario_aula          ha
JOIN          tb_grade_curricular          gc          ON          gc.pk_grade          =          ha.fk_grade
WHERE          ha.pk_horario_aula          =          NEW.fk_horario_aula;

```

```

IF          v_turma_matricula          !=          v_turma_horario          THEN
    SIGNAL          SQLSTATE          '45000'
    SET MESSAGE_TEXT = 'O horario de aula nao pertence a turma da matricula informada';
END          IF;

```

```

END//

```

```

CREATE TRIGGER trg_presenca_valida_turma_update BEFORE UPDATE ON tb_presenca_aluno FOR
EACH ROW BEGIN DECLARE v_turma_matricula INT; DECLARE v_turma_horario INT;

```

```

IF          NEW.fk_matricula          !=          OLD.fk_matricula
OR          NEW.fk_horario_aula          !=          OLD.fk_horario_aula          THEN

```

```

SELECT          fk_turma          INTO          v_turma_matricula
FROM
WHERE          pk_matricula          =          NEW.fk_matricula;

```

```

SELECT          gc.fk_turma          INTO          v_turma_horario
FROM          tb_horario_aula          ha
JOIN          tb_grade_curricular          gc          ON          gc.pk_grade          =          ha.fk_grade
WHERE          ha.pk_horario_aula          =          NEW.fk_horario_aula;

```

```

IF          v_turma_matricula          !=          v_turma_horario          THEN
    SIGNAL          SQLSTATE          '45000'
    SET MESSAGE_TEXT = 'O horario de aula nao pertence a turma da matricula
informada';
END          IF;
END          IF;

```

```

END//

```

```

--      --
GRUPO 10 - tb_nota_atividade -- Regra: a atividade lançada deve pertencer à mesma

```

turma	da	matricula	do	aluno	--
-------	----	-----------	----	-------	----

```
CREATE TRIGGER trg_nota_valida_turma_insert BEFORE INSERT ON tb_nota_atividade FOR
EACH ROW BEGIN DECLARE v_turma_matricula INT; DECLARE v_turma_atividade INT;
```

```
SELECT          fk_turma          INTO          v_turma_matricula
FROM
WHERE          pk_matricula          =          NEW.fk_matricula;
```

```
SELECT          gc.fk_turma          INTO          v_turma_atividade
FROM          tb_atividade_professor          ap
JOIN          tb_grade_curricular          gc          ON          gc.pk_grade          =          ap.fk_grade
WHERE          ap.pk_atividade_professor          =          NEW.fk_atividade_professor;
```

```
IF          v_turma_matricula          !=          v_turma_atividade          THEN
    SIGNAL          SQLSTATE          '45000'
    SET MESSAGE_TEXT = 'A atividade nao pertence a turma da matricula informada';
END          IF;
```

```
END//
```

```
CREATE TRIGGER trg_nota_valida_turma_update BEFORE UPDATE ON tb_nota_atividade FOR
EACH ROW BEGIN DECLARE v_turma_matricula INT; DECLARE v_turma_atividade INT;
```

```
IF          NEW.fk_matricula          !=          OLD.fk_matricula
OR          NEW.fk_atividade_professor          !=          OLD.fk_atividade_professor          THEN
```

```
SELECT          fk_turma          INTO          v_turma_matricula
FROM
WHERE          pk_matricula          =          NEW.fk_matricula;
```

```
SELECT          gc.fk_turma          INTO          v_turma_atividade
FROM          tb_atividade_professor          ap
JOIN          tb_grade_curricular          gc          ON          gc.pk_grade          =          ap.fk_grade
WHERE          ap.pk_atividade_professor          =          NEW.fk_atividade_professor;
```

```
IF          v_turma_matricula          !=          v_turma_atividade          THEN
    SIGNAL          SQLSTATE          '45000'
    SET MESSAGE_TEXT = 'A atividade nao pertence a turma da matricula informada';
END          IF;
END          IF;
```

```
END//
```

```
DELIMITER ;
```

```
--
RESUMO - 20 triggers no total (10 grupos x INSERT + UPDATE) -- -- GRUPO 1 -
tb_contrato -> 2 triggers -- GRUPO 2 - tb_historico_cargo -> 2 triggers -- GRUPO 3 -
tb_historico_salario -> 2 triggers -> NOVO -- GRUPO 4 - tb_matricula -> 2 triggers --
GRUPO 5 - tb_ano_letivo -> 2 triggers -> NOVO -- GRUPO 6 - tb_email -> 2 triggers -> NOVO
-- GRUPO 7 - tb_telefone -> 2 triggers -> NOVO -- GRUPO 8 - tb_responsavel_aluno -> 2
triggers -> NOVO -- GRUPO 9 - tb_presenca_aluno -> 2 triggers -> NOVO -- GRUPO 10 -
tb_nota_atividade -> 2 triggers -> NOVO -- -- DEPENDÊNCIA DE SEED: --
tb_status_matricula deve conter um registro com descricao = 'ativa' -- (exatamente,
minúsculo) para que o GRUPO 4 funcione corretamente. --
```

```
SELECT p.nome, p.sobrenome, a.ra, t.nome_turma, t.ano_escolar, sm.descricao AS
status_matricula FROM tb_matricula m JOIN tb_aluno a ON a.pk_aluno = m.fk_aluno JOIN
tb_pessoa p ON p.pk_pessoa = a.fk_pessoa JOIN tb_turma t ON t.pk_turma = m.fk_turma
JOIN tb_status_matricula sm ON sm.pk_status_matricula = m.fk_status_matricula WHERE
sm.descricao = 'ativa' ORDER BY t.ano_escolar, t.nome_turma, p.sobrenome;
```

```
SELECT t.nome_turma, t.ano_escolar, t.periodo, COUNT(m.pk_matricula) AS total_alunos,
t.capacidade_maxima, ROUND(COUNT(m.pk_matricula) * 100.0 / NULLIF(t.capacidade_maxima,
0), 1) AS pct_ocupacao FROM tb_turma t JOIN tb_ano_letivo al ON al.pk_ano_letivo =
t.fk_ano_letivo JOIN tb_matricula m ON m.fk_turma = t.pk_turma JOIN
tb_status_matricula sm ON sm.pk_status_matricula = m.fk_status_matricula WHERE
al.ativo = TRUE AND sm.descricao = 'ativa' GROUP BY t.pk_turma, t.nome_turma,
t.ano_escolar, t.periodo, t.capacidade_maxima ORDER BY t.ano_escolar, t.nome_turma;
```

```
SELECT p.nome, p.sobrenome, a.ra, t.nome_turma, b.bimestre, ROUND( SUM(na.valor_nota *
ap.peso) / NULLIF(SUM(ap.peso), 0) , 2) AS media_ponderada FROM tb_boletim b JOIN
tb_matricula m ON m.pk_matricula = b.fk_matricula JOIN tb_aluno a ON a.pk_aluno =
m.fk_aluno JOIN tb_pessoa p ON p.pk_pessoa = a.fk_pessoa JOIN tb_turma t ON t.pk_turma
= m.fk_turma JOIN tb_nota_atividade na ON na.fk_matricula = m.pk_matricula JOIN
tb_atividade_professor ap ON ap.pk_atividade_professor = na.fk_atividade_professor AND
ap.bimestre = b.bimestre WHERE b.bimestre = ( SELECT MAX(b2.bimestre) FROM tb_boletim
b2 WHERE b2.fk_matricula = b.fk_matricula ) AND na.valor_nota IS NOT NULL GROUP BY
b.pk_boletim, p.nome, p.sobrenome, a.ra, t.nome_turma, b.bimestre HAVING
media_ponderada < 5 ORDER BY media_ponderada;
```

```
SELECT p_aluno.nome AS aluno_nome, p_aluno.sobrenome AS aluno_sobrenome, a.ra,
t.nome_turma, pg.data_vencimento, DATEDIFF(CURDATE(), pg.data_vencimento) AS
dias_atraso, pg.valor_original, p_resp.nome AS responsavel_nome, p_resp.sobrenome AS
responsavel_sobrenome, e.email AS email_responsavel FROM tb_pagamento pg JOIN
tb_status_pagamento sp ON sp.pk_status_pagamento = pg.fk_status_pagamento JOIN
tb_contrato_financeiro cf ON cf.pk_contrato_financeiro = pg.fk_contrato_financeiro
JOIN tb_matricula m ON m.pk_matricula = cf.fk_matricula JOIN tb_aluno a ON a.pk_aluno
= m.fk_aluno JOIN tb_pessoa p_aluno ON p_aluno.pk_pessoa = a.fk_pessoa JOIN tb_turma t
ON t.pk_turma = m.fk_turma JOIN tb_responsavel_aluno ra ON ra.fk_aluno = a.pk_aluno
AND ra.responsavel_financeiro = TRUE JOIN tb_responsavel r ON r.pk_responsavel =
ra.fk_responsavel JOIN tb_pessoa p_resp ON p_resp.pk_pessoa = r.fk_pessoa LEFT JOIN
tb_email e ON e.pk_email = ( SELECT pk_email FROM tb_email WHERE fk_pessoa =
r.fk_pessoa AND principal = TRUE LIMIT 1 ) WHERE sp.descricao != 'pago' AND
sp.descricao != 'cancelado' AND pg.data_vencimento < CURDATE() - INTERVAL 5 DAY ORDER
BY dias_atraso DESC;
```

```

WITH frequencia AS ( SELECT m.pk_matricula, p.nome, p.sobrenome, a.ra, t.pk_turma,
t.nome_turma, COUNT(pa.pk_presenca) AS total_aulas, SUM(CASE WHEN pa.presenca = TRUE
THEN 1 ELSE 0 END) AS presencas, ROUND( SUM(CASE WHEN pa.presenca = TRUE THEN 1 ELSE 0
END) * 100.0 / NULLIF(COUNT(pa.pk_presenca), 0) , 2) AS pct_frequencia FROM
tb_matricula m JOIN tb_aluno a ON a.pk_aluno = m.fk_aluno JOIN tb_pessoa p ON
p.pk_pessoa = a.fk_pessoa JOIN tb_turma t ON t.pk_turma = m.fk_turma JOIN
tb_presenca_aluno pa ON pa.fk_matricula = m.pk_matricula JOIN tb_ano_letivo al ON
al.pk_ano_letivo = t.fk_ano_letivo WHERE al.ativo = TRUE GROUP BY m.pk_matricula,
p.nome, p.sobrenome, a.ra, t.pk_turma, t.nome_turma ) SELECT nome_turma, nome,
sobrenome, ra, total_aulas, presencas, pct_frequencia, RANK() OVER (PARTITION BY
pk_turma ORDER BY pct_frequencia DESC) AS rank_turma FROM frequencia ORDER BY
nome_turma, rank_turma;

```

```

DROP PROCEDURE IF EXISTS sp_registrar_pagamento;

```

```

DELIMITER // CREATE PROCEDURE sp_registrar_pagamento( IN p_contrato_fin INT, IN p_mes
TINYINT, IN p_ano YEAR, IN p_valor_pago DECIMAL(10,2) ) BEGIN DECLARE v_pk_pago INT;
DECLARE v_pk_pagamento INT; DECLARE v_rows INT;

```

```

START TRANSACTION;

```

```

SELECT          pk_status_pagamento          INTO          v_pk_pago
FROM
WHERE          descricao          =          tb_status_pagamento
LIMIT          1;

```

```

SELECT          pk_pagamento          INTO          v_pk_pagamento
FROM          tb_pagamento
WHERE          fk_contrato_financeiro          =          p_contrato_fin
AND          mes_referencia          =          p_mes
AND          ano_referencia          =          p_ano
LIMIT          1;

```

```

UPDATE          tb_pagamento
SET
    fk_status_pagamento          =          v_pk_pago,
    valor_pago          =          p_valor_pago,
    data_pagamento          =          CURDATE()
WHERE          pk_pagamento          =          v_pk_pagamento;

```

```

SET          v_rows          =          ROW_COUNT();

```

```

IF          v_rows          =          1          THEN
    COMMIT;
    SELECT  'Pagamento registrado com sucesso' AS resultado, v_pk_pagamento AS
pk_pagamento;
ELSE
    ROLLBACK;
    SELECT  'Nenhum pagamento localizado - operacao revertida' AS resultado, NULL AS
pk_pagamento;

```

END

IF;

END// DELIMITER ;

```
-- Substitua os valores abaixo pelos IDs reais antes de executar: -- CALL
sp_registrar_pagamento(p_contrato_fin,    p_mes,    p_ano,    p_valor_pago);    CALL
sp_registrar_pagamento(1, 5, 2025, 950.00);
```

```
CREATE INDEX idx_pagamento_vencimento_status ON tb_pagamento (data_vencimento,
fk_status_pagamento);
```

```
CREATE INDEX idx_pagamento_ano_mes ON tb_pagamento (ano_referencia, mes_referencia);
```

```
CREATE INDEX idx_folha_ano_mes_status ON tb_folha_pagamento (ano, mes, status_folha);
```

```
CREATE INDEX idx_presenca_matricula_cal ON tb_presenca_aluno (fk_matricula,
fk_calendario_letivo);
```

```
CREATE INDEX idx_nota_atividade ON tb_nota_atividade (fk_atividade_professor,
valor_nota);
```

```
CREATE INDEX idx_boletim_matricula_bimestre ON tb_boletim (fk_matricula, bimestre);
```

```
CREATE INDEX idx_matricula_turma_status ON tb_matricula (fk_turma,
fk_status_matricula);
```

```
CREATE INDEX idx_alerta_status_data ON tb_alerta_evasao (fk_status_alerta,
data_alerta);
```

```
EXPLAIN SELECT pk_pagamento, data_vencimento, valor_original FROM tb_pagamento IGNORE
INDEX (idx_pagamento_vencimento_status) WHERE fk_status_pagamento = 2 -- status
"pendente" AND data_vencimento < CURDATE() - INTERVAL 5 DAY;
```

```
EXPLAIN SELECT pk_pagamento, data_vencimento, valor_original FROM tb_pagamento WHERE
fk_status_pagamento = 2 AND data_vencimento < CURDATE() - INTERVAL 5 DAY;
```

```
EXPLAIN SELECT fk_matricula, COUNT(*) AS total, SUM(presenca) AS presentes FROM
tb_presenca_aluno IGNORE INDEX (idx_presenca_matricula_cal) WHERE fk_matricula = 42
GROUP BY fk_matricula;
```

```
EXPLAIN SELECT fk_matricula, COUNT(*) AS total, SUM(presenca) AS presentes FROM
tb_presenca_aluno WHERE fk_matricula = 42 GROUP BY fk_matricula;
```

```
/* ← remova este bloco de comentário para executar
```

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
TRUNCATE TABLE tb_verba_folha; TRUNCATE TABLE tb_folha_pagamento; TRUNCATE TABLE
tb_pagamento; TRUNCATE TABLE tb_contrato_financeiro; TRUNCATE TABLE tb_nota_atividade;
TRUNCATE TABLE tb_boletim; TRUNCATE TABLE tb_presenca_aluno; TRUNCATE TABLE
tb_alerta_evasao; TRUNCATE TABLE tb_atividade_professor; TRUNCATE TABLE
tb_horario_aula; TRUNCATE TABLE tb_grade_curricular; TRUNCATE TABLE tb_matricula;
TRUNCATE TABLE tb_responsavel_aluno; TRUNCATE TABLE tb_profissao_responsavel; TRUNCATE
```

```
TABLE tb_responsavel; TRUNCATE TABLE tb_formacao_professor; TRUNCATE TABLE
tb_professor; TRUNCATE TABLE tb_historico_salario; TRUNCATE TABLE tb_historico_cargo;
TRUNCATE TABLE tb_contrato; TRUNCATE TABLE tb_funcionario; TRUNCATE TABLE tb_aluno;
TRUNCATE TABLE tb_evento_escolar; TRUNCATE TABLE tb_calendario_letivo; TRUNCATE TABLE
tb_turma; TRUNCATE TABLE tb_email; TRUNCATE TABLE tb_telefone; TRUNCATE TABLE
tb_endereco; TRUNCATE TABLE tb_pessoa;
```

```
TRUNCATE TABLE tb_area_conhecimento; TRUNCATE TABLE tb_materia; TRUNCATE TABLE
tb_cargo; TRUNCATE TABLE tb_tipo_contrato; TRUNCATE TABLE tb_tipo_atividade; TRUNCATE
TABLE tb_tipo_aula; TRUNCATE TABLE tb_tipo_ingresso; TRUNCATE TABLE
tb_tipo_escola_anterior; TRUNCATE TABLE tb_tipo_risco_evasao; TRUNCATE TABLE
tb_tipo_verba; TRUNCATE TABLE tb_nivel_formacao; TRUNCATE TABLE tb_nivel_risco_evasao;
TRUNCATE TABLE tb_situacao_boletim; TRUNCATE TABLE tb_status_alerta; TRUNCATE TABLE
tb_status_matricula; TRUNCATE TABLE tb_status_pagamento; TRUNCATE TABLE tb_profissao;
TRUNCATE TABLE tb_ano_letivo;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

```
*/
```

```
/* ← remova este bloco de comentário para executar
```

```
DROP DATABASE IF EXISTS sistema_aurora_edutech;
```

```
*/
```

3. SCRIPT DE CARGA DE DADOS:

```
-- 

---


-- SCRIPT DML – Aurora EduTech v06
-- Carga de Dados Operacionais – IDEMPOTENTE (INSERT IGNORE)
--
-- Estratégia de idempotência:
--
-- • INSERT IGNORE → ignora silenciosamente duplicatas (UNIQUE ou PK)
--
-- • Dados com AUTO_INCREMENT usam INSERT IGNORE sem especificar PK,
--   mas com valores únicos nos campos com UNIQUE INDEX para garantir
--   que reexecuções não dupliquem registros.
--
-- • Onde a PK é composta (sem AUTO_INCREMENT), INSERT IGNORE protege
```

-- diretamente pela chave primária.

-- 1. tb_tipo_escola_anterior

-- _____

INSERT IGNORE INTO tb_tipo_escola_anterior (pk_tipo_escola_anterior, descricao) VALUES

(1, 'Pública Municipal'),

(2, 'Pública Estadual'),

(3, 'Particular'),

(4, 'Federal'),

(5, 'Não informado');

-- _____

-- 2. tb_tipo_ingresso

-- _____

INSERT IGNORE INTO tb_tipo_ingresso (pk_tipo_ingresso, descricao, ativo) VALUES

(1, 'Matrícula Nova', TRUE),

(2, 'Transferência Externa', TRUE),

(3, 'Rematrícula', TRUE),

(4, 'Transferência Interna', TRUE),

(5, 'Cortesia Institucional', FALSE);

-- _____

-- 3. tb_status_matricula ← 'ativa' OBRIGATÓRIO para trigger Grupo 4

-- _____

INSERT IGNORE INTO tb_status_matricula (pk_status_matricula, descricao) VALUES

(1, 'ativa'),

(2, 'trancada'),

(3, 'cancelada'),

(4, 'concluída'),

```
(5, 'pendente');
```

```
-- _____
```

```
-- 4. tb_situacao_boletim
```

```
-- _____
```

```
INSERT IGNORE INTO tb_situacao_boletim (pk_situacao_boletim, descricao, ativo) VALUES
```

```
(1, 'Aprovado',      TRUE),
```

```
(2, 'Reprovado',     TRUE),
```

```
(3, 'Em Recuperação', TRUE),
```

```
(4, 'Em Andamento', TRUE),
```

```
(5, 'Aguardando Notas', TRUE);
```

```
-- _____
```

```
-- 5. tb_nivel_risco_evasao
```

```
-- _____
```

```
INSERT IGNORE INTO tb_nivel_risco_evasao (pk_nivel_risco, descricao, ordem) VALUES
```

```
(1, 'Baixo',      1),
```

```
(2, 'Médio',      2),
```

```
(3, 'Alto',       3),
```

```
(4, 'Crítico',    4);
```

```
-- _____
```

```
-- 6. tb_tipo_risco_evasao
```

```
-- _____
```

```
INSERT IGNORE INTO tb_tipo_risco_evasao (pk_tipo_risco_evasao, descricao, ativo) VALUES
```

```
(1, 'Inadimplência Financeira', TRUE),
```

```
(2, 'Frequência Abaixo do Mínimo', TRUE),
```

```
(3, 'Baixo Rendimento Acadêmico', TRUE),
```

```
(4, 'Problemas Disciplinares',      TRUE),  
(5, 'Ausência de Responsável',      TRUE),  
(6, 'Solicitação de Transferência', TRUE);
```

```
-- _____
```

```
-- 7. tb_status_alerta
```

```
-- _____
```

```
INSERT IGNORE INTO tb_status_alerta (pk_status_alerta, descricao, ativo) VALUES
```

```
(1, 'Aberto',      TRUE),  
(2, 'Em Análise',  TRUE),  
(3, 'Resolvido',   TRUE),  
(4, 'Arquivado',   TRUE),  
(5, 'Cancelado',   FALSE);
```

```
-- _____
```

```
-- 8. tb_area_conhecimento
```

```
-- _____
```

```
INSERT IGNORE INTO tb_area_conhecimento (pk_area_conhecimento, descricao) VALUES
```

```
(1, 'Linguagens'),  
(2, 'Matemática'),  
(3, 'Ciências da Natureza'),  
(4, 'Ciências Humanas'),  
(5, 'Artes'),  
(6, 'Educação Física');
```

```
-- _____
```

```
-- 9. tb_materia
```

```
-- _____
```

```
INSERT IGNORE INTO tb_materia (pk_materia, fk_area_conhecimento, nome_materia, ativo)
VALUES
```

```
(1, 1, 'Português', TRUE),
(2, 1, 'Inglês', TRUE),
(3, 2, 'Matemática', TRUE),
(4, 3, 'Ciências', TRUE),
(5, 4, 'História', TRUE),
(6, 4, 'Geografia', TRUE),
(7, 5, 'Artes', TRUE),
(8, 6, 'Educação Física', TRUE),
(9, 3, 'Física', FALSE),
(10, 3, 'Química', FALSE);
```

```
-- _____
```

```
-- 10. tb_tipo_atividade
```

```
-- _____
```

```
INSERT IGNORE INTO tb_tipo_atividade (pk_tipo_atividade, descricao) VALUES
```

```
(1, 'Prova'),
(2, 'Trabalho'),
(3, 'Exercício'),
(4, 'Seminário'),
(5, 'Projeto');
```

```
-- _____
```

```
-- 11. tb_tipo_aula
```

```
-- _____
```

```
INSERT IGNORE INTO tb_tipo_aula (pk_tipo_aula, descricao) VALUES
```

```
(1, 'Presencial'),
(2, 'Híbrida'),
```

```
(3, 'Reforço'),  
(4, 'Educação Especial');
```

```
-- _____
```

```
-- 12. tb_tipo_contrato
```

```
-- _____
```

```
INSERT IGNORE INTO tb_tipo_contrato (pk_tipo_contrato, descricao) VALUES
```

```
(1, 'CLT'),  
(2, 'PJ'),  
(3, 'Estágio'),  
(4, 'Temporário');
```

```
-- _____
```

```
-- 13. tb_cargo
```

```
-- _____
```

```
INSERT IGNORE INTO tb_cargo (pk_cargo, nome_cargo, descricao, ativo) VALUES
```

```
(1, 'Professor',          'Docente responsável por ministrar aulas',      TRUE),  
(2, 'Coordenador',        'Coordenação pedagógica',                        TRUE),  
(3, 'Diretor',            'Gestão geral da unidade escolar',                TRUE),  
(4, 'Secretário',         'Atendimento e documentação escolar',            TRUE),  
(5, 'Auxiliar Administrativo', 'Suporte às atividades administrativas',          TRUE),  
(6, 'Inspetor',           'Supervisão de alunos no ambiente escolar',      TRUE),  
(7, 'Psicólogo Escolar',  'Acompanhamento psicológico de alunos',          TRUE),  
(8, 'Porteiro',           'Controle de acesso à unidade',                   FALSE);
```

```
-- _____
```

```
-- 14. tb_nivel_formacao
```

```
-- _____
```

```
INSERT IGNORE INTO tb_nivel_formacao (pk_nivel_formacao, descricao) VALUES
```

```
(1, 'Graduação'),  
(2, 'Especialização'),  
(3, 'Mestrado'),  
(4, 'Doutorado'),  
(5, 'Técnico');
```

```
-- 15. tb_profissao
```

```
INSERT IGNORE INTO tb_profissao (pk_profissao, profissao, descricao) VALUES
```

```
(1, 'Médico',      'Profissional da área da saúde'),  
(2, 'Engenheiro',  'Profissional de engenharia'),  
(3, 'Advogado',    'Profissional do direito'),  
(4, 'Professor',   'Profissional da educação'),  
(5, 'Comerciante', 'Profissional do comércio'),  
(6, 'Autônomo',    'Profissional liberal sem vínculo'),  
(7, 'Funcionário Público', 'Servidor do setor público');
```

```
-- 16. tb_tipo_verba
```

```
INSERT IGNORE INTO tb_tipo_verba (pk_tipo_verba, descricao, natureza_verba, ativo)  
VALUES
```

```
(1, 'Salário Base',      'credito', TRUE),  
(2, 'Hora Extra',        'credito', TRUE),  
(3, 'Adicional Noturno', 'credito', TRUE),  
(4, 'Vale Transporte',    'credito', TRUE),  
(5, 'Vale Alimentação',  'credito', TRUE),  
(6, 'INSS',              'debito',  TRUE),
```

```
(7, 'IRRF', 'debito', TRUE),
(8, 'Desconto Falta', 'debito', TRUE),
(9, '13° Salário', 'credito', TRUE),
(10, 'Férias', 'credito', TRUE),
(11, 'Rescisão', 'credito', TRUE),
(12, 'Adiantamento Quinzenal', 'credito', TRUE);
```

```
-- 17. tb_ano_letivo ← apenas 2025 ativo (trigger Grupo 5)
```

```
INSERT IGNORE INTO tb_ano_letivo (pk_ano_letivo, ano, data_inicio, data_fim, ativo)
VALUES
```

```
(1, 2023, '2023-02-06', '2023-12-15', FALSE),
(2, 2024, '2024-02-05', '2024-12-13', FALSE),
(3, 2025, '2025-02-03', '2025-12-12', TRUE);
```

```
-- 18. tb_turma (5 turmas em 2025 – anos_escolares 1 a 5)
```

```
INSERT IGNORE INTO tb_turma (pk_turma, fk_ano_letivo, nome_turma, ano_escolar, periodo,
capacidade_maxima) VALUES
```

```
(1, 3, '1A', 1, 'manha', 30),
(2, 3, '2A', 2, 'manha', 30),
(3, 3, '3A', 3, 'tarde', 28),
(4, 3, '4A', 4, 'tarde', 28),
(5, 3, '5A', 5, 'manha', 25),
(6, 2, '1A', 1, 'manha', 30),
(7, 2, '2A', 2, 'tarde', 30);
```

-- 19. tb_calendario_letivo (dias úteis e feriados - amostra)

-- _____

INSERT IGNORE INTO tb_calendario_letivo (pk_calendario, fk_ano_letivo, data_evento, tipo_dia, letivo) VALUES

(1, 3, '2025-02-03', 'util', TRUE),
(2, 3, '2025-02-04', 'util', TRUE),
(3, 3, '2025-02-05', 'util', TRUE),
(4, 3, '2025-03-03', 'util', TRUE),
(5, 3, '2025-03-04', 'util', TRUE),
(6, 3, '2025-04-18', 'feriado', FALSE),
(7, 3, '2025-04-21', 'feriado', FALSE),
(8, 3, '2025-05-01', 'feriado', FALSE),
(9, 3, '2025-06-02', 'util', TRUE),
(10, 3, '2025-06-03', 'util', TRUE),
(11, 3, '2025-07-07', 'recesso', FALSE),
(12, 3, '2025-07-08', 'recesso', FALSE),
(13, 3, '2025-08-04', 'util', TRUE),
(14, 3, '2025-08-05', 'util', TRUE),
(15, 3, '2025-09-07', 'feriado', FALSE),
(16, 3, '2025-10-06', 'util', TRUE),
(17, 3, '2025-10-07', 'util', TRUE),
(18, 3, '2025-11-02', 'feriado', FALSE),
(19, 3, '2025-11-15', 'feriado', FALSE),
(20, 3, '2025-12-12', 'util', TRUE);

-- _____

-- 20. tb_evento_escolar

-- _____

INSERT IGNORE INTO tb_evento_escolar (pk_evento, fk_calendario, titulo_evento, descricao, hora_inicio, hora_fim) VALUES

```

(1, 1, 'Primeiro Dia de Aula 2025', 'Abertura do ano letivo 2025',
'07:00:00', '12:00:00'),

(2, 6, 'Sexta-feira Santa', 'Feriado nacional - Paixão de Cristo',
NULL, NULL),

(3, 7, 'Tiradentes', 'Feriado nacional',
NULL, NULL),

(4, 8, 'Dia do Trabalho', 'Feriado nacional',
NULL, NULL),

(5, 11, 'Recesso Escolar Julho', 'Início do recesso de meio de ano',
NULL, NULL),

(6, 15, 'Independência do Brasil', 'Feriado nacional',
NULL, NULL),

(7, 20, 'Encerramento do Ano Letivo', 'Último dia letivo de 2025',
'07:00:00', '12:00:00');

```

```
-- 21. tb_pessoa (10 alunos + 5 responsáveis + 5 funcionários = 20 pessoas)
```

```
INSERT IGNORE INTO tb_pessoa (pk_pessoa, cpf, rg, nome, sobrenome, sexo,
data_nascimento, nacionalidade, raca) VALUES
```

```
-- Alunos
```

```

(1, '11122233344', '123456789', 'Lucas', 'Oliveira', 'masculino', '2016-03-10',
'brasileiro', 'parda'),

(2, '22233344455', '234567890', 'Ana', 'Souza', 'feminino', '2015-07-22',
'brasileiro', 'branca'),

(3, '33344455566', '345678901', 'Pedro', 'Santos', 'masculino', '2014-11-05',
'brasileiro', 'negra'),

(4, '44455566677', '456789012', 'Maria', 'Lima', 'feminino', '2013-02-18',
'brasileiro', 'parda'),

(5, '55566677788', '567890123', 'João', 'Costa', 'masculino', '2012-09-30',
'brasileiro', 'branca'),

(6, '66677788899', '678901234', 'Isabela', 'Fernandes', 'feminino', '2016-05-14',
'brasileiro', 'amarela'),

(7, '77788899900', '789012345', 'Matheus', 'Rodrigues', 'masculino', '2015-01-27',
'brasileiro', 'parda'),

(8, '88899900011', '890123456', 'Sophia', 'Alves', 'feminino', '2014-08-09',
'brasileiro', 'branca'),

```

(9, '99900011122', '901234567', 'Gabriel', 'Nascimento', 'masculino', '2013-12-03', 'brasileiro', 'negra'),

(10, '00011122233', '012345678', 'Valentina', 'Carvalho', 'feminino', '2012-06-16', 'brasileiro', 'parda'),

-- Responsáveis

(11, '11122233300', '111222333', 'Carlos', 'Oliveira', 'masculino', '1985-04-20', 'brasileiro', 'parda'),

(12, '22233344400', '222333444', 'Fernanda', 'Souza', 'feminino', '1983-09-15', 'brasileiro', 'branca'),

(13, '33344455500', '333444555', 'Roberto', 'Santos', 'masculino', '1980-12-01', 'brasileiro', 'negra'),

(14, '44455566600', '444555666', 'Patricia', 'Lima', 'feminino', '1979-06-28', 'brasileiro', 'parda'),

(15, '55566677700', '555666777', 'Marcelo', 'Costa', 'masculino', '1975-11-11', 'brasileiro', 'branca'),

-- Funcionários

(16, '66677788800', '666777888', 'Renata', 'Mendes', 'feminino', '1978-03-07', 'brasileiro', 'parda'),

(17, '77788899800', '777888999', 'Eduardo', 'Pereira', 'masculino', '1975-08-22', 'brasileiro', 'branca'),

(18, '88899900800', '888999000', 'Claudia', 'Martins', 'feminino', '1982-01-14', 'brasileiro', 'negra'),

(19, '99900011800', '999000111', 'André', 'Gomes', 'masculino', '1988-05-30', 'brasileiro', 'parda'),

(20, '00011122800', '000111222', 'Juliana', 'Ribeiro', 'feminino', '1990-10-19', 'brasileiro', 'branca');

-- _____

-- 22. tb_email

-- _____

INSERT IGNORE INTO tb_email (pk_email, fk_pessoa, email, principal) VALUES

-- Responsáveis

(1, 11, 'carlos.oliveira@email.com', TRUE),

(2, 12, 'fernanda.souza@email.com', TRUE),

(3, 13, 'roberto.santos@email.com', TRUE),

(4, 14, 'patricia.lima@email.com', TRUE),

```
(5, 15, 'marcelo.costa@email.com', TRUE),
```

```
-- Funcionários
```

```
(6, 16, 'renata.mendes@escola.edu.br', TRUE),
```

```
(7, 17, 'eduardo.pereira@escola.edu.br', TRUE),
```

```
(8, 18, 'claudia.martins@escola.edu.br', TRUE),
```

```
(9, 19, 'andre.gomes@escola.edu.br', TRUE),
```

```
(10, 20, 'juliana.ribeiro@escola.edu.br', TRUE),
```

```
-- Emails secundários
```

```
(11, 16, 'renata.mendes@gmail.com', FALSE),
```

```
(12, 17, 'edu.pereira@gmail.com', FALSE);
```

```
-- _____
```

```
-- 23. tb_telefone
```

```
-- _____
```

```
INSERT IGNORE INTO tb_telefone (pk_telefone, fk_pessoa, categoria, ddd, numero, ativo, principal) VALUES
```

```
(1, 11, 'celular', '11', '991234567', TRUE, TRUE),
```

```
(2, 12, 'celular', '11', '992345678', TRUE, TRUE),
```

```
(3, 13, 'celular', '21', '993456789', TRUE, TRUE),
```

```
(4, 14, 'celular', '21', '994567890', TRUE, TRUE),
```

```
(5, 15, 'celular', '31', '995678901', TRUE, TRUE),
```

```
(6, 16, 'celular', '11', '996789012', TRUE, TRUE),
```

```
(7, 17, 'celular', '11', '997890123', TRUE, TRUE),
```

```
(8, 18, 'celular', '11', '998901234', TRUE, TRUE),
```

```
(9, 19, 'celular', '11', '999012345', TRUE, TRUE),
```

```
(10, 20, 'celular', '11', '990123456', TRUE, TRUE),
```

```
(11, 11, 'residencial', '11', '33334444', TRUE, FALSE),
```

```
(12, 12, 'residencial', '11', '33335555', TRUE, FALSE),
```

```
(13, 16, 'emergencia', '11', '33336666', TRUE, FALSE);
```

```

-- _____

-- 24. tb_endereco

-- _____

INSERT IGNORE INTO tb_endereco (pk_endereco, fk_pessoa, cep, numero, complemento,
principal) VALUES

(1, 11, '01310100', '100', NULL, TRUE),
(2, 12, '04038001', '250', 'Apto 32', TRUE),
(3, 13, '20040020', '88', NULL, TRUE),
(4, 14, '30130010', '310', 'Casa', TRUE),
(5, 15, '40020010', '45', NULL, TRUE),
(6, 16, '01001001', '500', 'Sala 2', TRUE),
(7, 17, '01310200', '120', NULL, TRUE),
(8, 18, '04038002', '77', 'Apto 10', TRUE),
(9, 19, '04038003', '33', NULL, TRUE),
(10, 20, '01310300', '200', 'Bloco B', TRUE);

-- _____

-- 25. tb_aluno (10 alunos, pessoas 1-10)

-- _____

INSERT IGNORE INTO tb_aluno (pk_aluno, fk_pessoa, fk_tipo_escola_anterior,
data_ingresso, faz_atividade_extracurricular, ra) VALUES

(1, 1, 1, '2022-02-07', FALSE, '202200001'),
(2, 2, 2, '2021-02-08', TRUE, '202100002'),
(3, 3, 1, '2020-02-10', FALSE, '202000003'),
(4, 4, 3, '2019-02-11', TRUE, '201900004'),
(5, 5, 2, '2018-02-05', FALSE, '201800005'),
(6, 6, 1, '2022-02-07', TRUE, '202200006'),
(7, 7, 3, '2021-02-08', FALSE, '202100007'),
(8, 8, 2, '2020-02-10', TRUE, '202000008'),
(9, 9, 1, '2019-02-11', FALSE, '201900009'),

```

```
(10, 10, 3, '2018-02-05', TRUE, '201800010');
```

```
-- _____
```

```
-- 26. tb_responsavel (5 responsáveis, pessoas 11-15)
```

```
-- _____
```

```
INSERT IGNORE INTO tb_responsavel (pk_responsavel, fk_pessoa) VALUES
```

```
(1, 11),
```

```
(2, 12),
```

```
(3, 13),
```

```
(4, 14),
```

```
(5, 15);
```

```
-- _____
```

```
-- 27. tb_responsavel_aluno
```

```
-- Regra trigger Grupo 8: apenas 1 responsável_principal por aluno
```

```
-- _____
```

```
INSERT IGNORE INTO tb_responsavel_aluno (fk_aluno, fk_responsavel,  
responsavel_principal, responsavel_financeiro, pode_retirar) VALUES
```

```
(1, 1, TRUE, TRUE, TRUE),
```

```
(2, 2, TRUE, TRUE, TRUE),
```

```
(3, 3, TRUE, TRUE, TRUE),
```

```
(4, 4, TRUE, TRUE, TRUE),
```

```
(5, 5, TRUE, TRUE, TRUE),
```

```
(6, 1, FALSE, FALSE, TRUE), -- irmão / segundo responsável
```

```
(7, 2, FALSE, FALSE, TRUE),
```

```
(8, 3, FALSE, FALSE, TRUE),
```

```
(9, 4, FALSE, FALSE, TRUE),
```

```
(10, 5, FALSE, TRUE, TRUE),
```

```
(6, 2, TRUE, TRUE, TRUE),
```

```
(7, 3, TRUE, TRUE, TRUE),  
(8, 4, TRUE, TRUE, TRUE),  
(9, 5, TRUE, TRUE, TRUE),  
(10, 1, TRUE, TRUE, TRUE);
```

```
-- _____
```

```
-- 28. tb_profissao_responsavel
```

```
-- _____
```

```
INSERT IGNORE INTO tb_profissao_responsavel (fk_responsavel, fk_profissao, ativo)  
VALUES
```

```
(1, 2, TRUE),  
(2, 4, TRUE),  
(3, 1, TRUE),  
(4, 3, TRUE),  
(5, 5, TRUE);
```

```
-- _____
```

```
-- 29. tb_funcionario (5 funcionários, pessoas 16-20)
```

```
-- _____
```

```
INSERT IGNORE INTO tb_funcionario (pk_funcionario, fk_pessoa, ativo) VALUES
```

```
(1, 16, TRUE),  
(2, 17, TRUE),  
(3, 18, TRUE),  
(4, 19, TRUE),  
(5, 20, TRUE);
```

```
-- _____
```

```
-- 30. tb_professor (3 professores dentre os 5 funcionários)
```

```
-- _____
```

```
INSERT IGNORE INTO tb_professor (pk_professor, fk_funcionario, fk_tipo_aula,
utiliza_rubrica_avaliativa, atende_educacao_especial) VALUES
```

```
(1, 1, 1, TRUE, FALSE),
```

```
(2, 2, 1, FALSE, FALSE),
```

```
(3, 3, 1, TRUE, TRUE);
```

```
-- _____
```

```
-- 31. tb_formacao_professor
```

```
-- _____
```

```
INSERT IGNORE INTO tb_formacao_professor (pk_formacao, fk_professor, fk_nivel_formacao,
curso, instituicao, ano_conclusao) VALUES
```

```
(1, 1, 1, 'Letras', 'USP', 2003),
```

```
(2, 1, 2, 'Linguística Aplicada', 'UNICAMP', 2006),
```

```
(3, 2, 1, 'Matemática', 'UNESP', 2000),
```

```
(4, 2, 3, 'Educação Matemática', 'UNIFESP', 2008),
```

```
(5, 3, 1, 'Pedagogia', 'PUC-SP', 2005),
```

```
(6, 3, 2, 'Educação Especial', 'SENAC', 2010);
```

```
-- _____
```

```
-- 32. tb_contrato (1 por funcionário - datas abertas = status ativo)
```

```
-- _____
```

```
INSERT IGNORE INTO tb_contrato (pk_contrato, fk_funcionario, fk_tipo_contrato,
status_contrato, carga_horaria_semanal, data_inicio, data_fim) VALUES
```

```
(1, 1, 1, 'ativo', 40, '2020-03-01', NULL),
```

```
(2, 2, 1, 'ativo', 40, '2018-02-15', NULL),
```

```
(3, 3, 1, 'ativo', 30, '2021-07-01', NULL),
```

```
(4, 4, 1, 'ativo', 40, '2019-01-10', NULL),
```

```
(5, 5, 2, 'ativo', 20, '2022-08-01', NULL);
```

```
-- _____
```

-- 33. tb_historico_cargo

-- _____

INSERT IGNORE INTO tb_historico_cargo (pk_historico_cargo, fk_contrato, fk_cargo,
data_inicio, data_fim, motivo_mudanca) VALUES

(1, 1, 1, '2020-03-01', NULL, NULL),
(2, 2, 1, '2018-02-15', NULL, NULL),
(3, 3, 1, '2021-07-01', NULL, NULL),
(4, 4, 2, '2019-01-10', NULL, NULL),
(5, 5, 4, '2022-08-01', NULL, NULL);

-- _____

-- 34. tb_historico_salario

-- _____

INSERT IGNORE INTO tb_historico_salario (pk_historico_salario, fk_contrato, salario,
data_inicio, data_fim, motivo_reajuste) VALUES

(1, 1, 4500.00, '2020-03-01', '2022-02-28', 'Admissão'),
(2, 1, 5000.00, '2022-03-01', NULL, 'Reajuste anual'),
(3, 2, 4800.00, '2018-02-15', '2021-02-14', 'Admissão'),
(4, 2, 5300.00, '2021-02-15', NULL, 'Promoção'),
(5, 3, 3800.00, '2021-07-01', NULL, 'Admissão'),
(6, 4, 6500.00, '2019-01-10', NULL, 'Admissão'),
(7, 5, 2800.00, '2022-08-01', NULL, 'Admissão');

-- _____

-- 35. tb_grade_curricular

-- _____

INSERT IGNORE INTO tb_grade_curricular (pk_grade, fk_turma, fk_materia, fk_professor,
data_inicio, data_fim) VALUES

(1, 1, 1, 1, '2025-02-03', NULL), -- Turma 1A, Português, Prof.1
(2, 1, 3, 2, '2025-02-03', NULL), -- Turma 1A, Matemática, Prof.2

```

(3, 1, 4, 3, '2025-02-03', NULL), -- Turma 1A, Ciências, Prof.3
(4, 2, 1, 1, '2025-02-03', NULL), -- Turma 2A, Português, Prof.1
(5, 2, 3, 2, '2025-02-03', NULL), -- Turma 2A, Matemática, Prof.2
(6, 3, 1, 1, '2025-02-03', NULL), -- Turma 3A, Português, Prof.1
(7, 3, 3, 2, '2025-02-03', NULL), -- Turma 3A, Matemática, Prof.2
(8, 4, 1, 1, '2025-02-03', NULL), -- Turma 4A, Português, Prof.1
(9, 4, 3, 2, '2025-02-03', NULL), -- Turma 4A, Matemática, Prof.2
(10, 5, 1, 1, '2025-02-03', NULL), -- Turma 5A, Português, Prof.1
(11, 5, 3, 2, '2025-02-03', NULL); -- Turma 5A, Matemática, Prof.2

```

```
-- _____
```

```
-- 36. tb_horario_aula
```

```
-- _____
```

```
INSERT IGNORE INTO tb_horario_aula (pk_horario_aula, fk_grade, dia_semana, hora_inicio,
hora_fim, sala) VALUES
```

```
-- Grade 1 (Turma 1A - Português)
```

```
(1, 1, 'segunda', '07:00:00', '07:50:00', '101'),
```

```
(2, 1, 'quarta', '07:00:00', '07:50:00', '101'),
```

```
-- Grade 2 (Turma 1A - Matemática)
```

```
(3, 2, 'terca', '07:00:00', '07:50:00', '101'),
```

```
(4, 2, 'quinta', '07:00:00', '07:50:00', '101'),
```

```
-- Grade 3 (Turma 1A - Ciências)
```

```
(5, 3, 'sexta', '07:00:00', '07:50:00', '101'),
```

```
-- Grade 4 (Turma 2A - Português)
```

```
(6, 4, 'segunda', '07:00:00', '07:50:00', '102'),
```

```
(7, 4, 'quarta', '07:00:00', '07:50:00', '102'),
```

```
-- Grade 5 (Turma 2A - Matemática)
```

```
(8, 5, 'terca', '07:00:00', '07:50:00', '102'),
```

```
(9, 5, 'quinta', '07:00:00', '07:50:00', '102'),
```

```
-- Grade 6 (Turma 3A - Português)
```

```

(10, 6, 'segunda', '13:00:00', '13:50:00', '201'),
(11, 6, 'quarta', '13:00:00', '13:50:00', '201'),
-- Grade 7 (Turma 3A - Matemática)
(12, 7, 'terca', '13:00:00', '13:50:00', '201'),
(13, 7, 'quinta', '13:00:00', '13:50:00', '201'),
-- Grade 8 (Turma 4A - Português)
(14, 8, 'segunda', '13:00:00', '13:50:00', '202'),
(15, 8, 'quarta', '13:00:00', '13:50:00', '202'),
-- Grade 9 (Turma 4A - Matemática)
(16, 9, 'terca', '13:00:00', '13:50:00', '202'),
(17, 9, 'quinta', '13:00:00', '13:50:00', '202'),
-- Grade 10 (Turma 5A - Português)
(18, 10, 'segunda', '07:00:00', '07:50:00', '103'),
(19, 10, 'quarta', '07:00:00', '07:50:00', '103'),
-- Grade 11 (Turma 5A - Matemática)
(20, 11, 'terca', '07:00:00', '07:50:00', '103'),
(21, 11, 'quinta', '07:00:00', '07:50:00', '103');

```

```
-- 37. tb_matricula
```

```
-- Distribuição:
```

```

-- Turma 1 (1A, cap=30): alunos 1, 6
-- Turma 2 (2A, cap=30): alunos 2, 7
-- Turma 3 (3A, cap=28): alunos 3, 8
-- Turma 4 (4A, cap=28): alunos 4, 9
-- Turma 5 (5A, cap=25): alunos 5, 10

```

```

INSERT IGNORE INTO tb_matricula (pk_matricula, fk_turma, fk_tipo_ingresso,
fk_status_matricula, data_matricula, fk_aluno) VALUES

```

```
(1, 1, 3, 1, '2025-01-20', 1),
```

```
(2, 2, 3, 1, '2025-01-20', 2),
(3, 3, 3, 1, '2025-01-20', 3),
(4, 4, 3, 1, '2025-01-20', 4),
(5, 5, 3, 1, '2025-01-20', 5),
(6, 1, 3, 1, '2025-01-20', 6),
(7, 2, 3, 1, '2025-01-20', 7),
(8, 3, 3, 1, '2025-01-20', 8),
(9, 4, 3, 1, '2025-01-20', 9),
(10, 5, 3, 1, '2025-01-20', 10);
```

```
-- _____
```

```
-- 38. tb_contrato_financeiro
```

```
-- _____
```

```
INSERT IGNORE INTO tb_contrato_financeiro (pk_contrato_financeiro, fk_matricula,
valor_matricula, valor_mensalidade, dia_vencimento, data_inicio, data_fim) VALUES
```

```
(1, 1, 500.00, 850.00, 10, '2025-01-20', NULL),
(2, 2, 500.00, 850.00, 10, '2025-01-20', NULL),
(3, 3, 500.00, 850.00, 10, '2025-01-20', NULL),
(4, 4, 500.00, 850.00, 10, '2025-01-20', NULL),
(5, 5, 500.00, 850.00, 10, '2025-01-20', NULL),
(6, 6, 500.00, 850.00, 15, '2025-01-20', NULL),
(7, 7, 500.00, 850.00, 15, '2025-01-20', NULL),
(8, 8, 500.00, 850.00, 15, '2025-01-20', NULL),
(9, 9, 500.00, 850.00, 15, '2025-01-20', NULL),
(10, 10, 500.00, 850.00, 15, '2025-01-20', NULL);
```

```
-- _____
```

```
-- 39. tb_status_pagamento
```

```
-- _____
```

```

INSERT IGNORE INTO tb_status_pagamento (pk_status_pagamento, descricao, ativo) VALUES

(1, 'Pendente',    TRUE),

(2, 'Pago',        TRUE),

(3, 'Atrasado',    TRUE),

(4, 'Isento',      TRUE),

(5, 'Cancelado',    FALSE);

```

```

-- _____

-- 40. tb_pagamento  (fev a mai/2025 × 10 alunos = 40 registros)

```

```

INSERT      IGNORE      INTO      tb_pagamento      (pk_pagamento,      fk_contrato_financeiro,
fk_status_pagamento,  valor_pago,  valor_original,  data_vencimento,  data_pagamento,
mes_referencia, ano_referencia) VALUES

```

```

-- Aluno 1

```

```

(1,  1, 2, 850.00, 850.00, '2025-02-10', '2025-02-10', 2, 2025),
(2,  1, 2, 850.00, 850.00, '2025-03-10', '2025-03-08', 3, 2025),
(3,  1, 2, 850.00, 850.00, '2025-04-10', '2025-04-10', 4, 2025),
(4,  1, 1, NULL,   850.00, '2025-05-10', NULL,          5, 2025),

```

```

-- Aluno 2

```

```

(5,  2, 2, 850.00, 850.00, '2025-02-10', '2025-02-09', 2, 2025),
(6,  2, 2, 850.00, 850.00, '2025-03-10', '2025-03-10', 3, 2025),
(7,  2, 2, 850.00, 850.00, '2025-04-10', '2025-04-11', 4, 2025),
(8,  2, 1, NULL,   850.00, '2025-05-10', NULL,          5, 2025),

```

```

-- Aluno 3

```

```

(9,  3, 2, 850.00, 850.00, '2025-02-10', '2025-02-10', 2, 2025),
(10, 3, 3, NULL,   850.00, '2025-03-10', NULL,          3, 2025),
(11, 3, 3, NULL,   850.00, '2025-04-10', NULL,          4, 2025),
(12, 3, 1, NULL,   850.00, '2025-05-10', NULL,          5, 2025),

```

```

-- Aluno 4

```

```

(13, 4, 2, 850.00, 850.00, '2025-02-10', '2025-02-10', 2, 2025),

```

```

(14, 4, 2, 850.00, 850.00, '2025-03-10', '2025-03-10', 3, 2025),
(15, 4, 2, 850.00, 850.00, '2025-04-10', '2025-04-10', 4, 2025),
(16, 4, 1, NULL, 850.00, '2025-05-10', NULL, 5, 2025),
-- Aluno 5
(17, 5, 2, 850.00, 850.00, '2025-02-10', '2025-02-10', 2, 2025),
(18, 5, 2, 850.00, 850.00, '2025-03-10', '2025-03-10', 3, 2025),
(19, 5, 1, NULL, 850.00, '2025-04-10', NULL, 4, 2025),
(20, 5, 1, NULL, 850.00, '2025-05-10', NULL, 5, 2025),
-- Aluno 6
(21, 6, 2, 850.00, 850.00, '2025-02-15', '2025-02-15', 2, 2025),
(22, 6, 2, 850.00, 850.00, '2025-03-15', '2025-03-14', 3, 2025),
(23, 6, 2, 850.00, 850.00, '2025-04-15', '2025-04-15', 4, 2025),
(24, 6, 1, NULL, 850.00, '2025-05-15', NULL, 5, 2025),
-- Aluno 7
(25, 7, 2, 850.00, 850.00, '2025-02-15', '2025-02-15', 2, 2025),
(26, 7, 2, 850.00, 850.00, '2025-03-15', '2025-03-15', 3, 2025),
(27, 7, 2, 850.00, 850.00, '2025-04-15', '2025-04-15', 4, 2025),
(28, 7, 1, NULL, 850.00, '2025-05-15', NULL, 5, 2025),
-- Aluno 8
(29, 8, 2, 850.00, 850.00, '2025-02-15', '2025-02-15', 2, 2025),
(30, 8, 2, 850.00, 850.00, '2025-03-15', '2025-03-15', 3, 2025),
(31, 8, 3, NULL, 850.00, '2025-04-15', NULL, 4, 2025),
(32, 8, 1, NULL, 850.00, '2025-05-15', NULL, 5, 2025),
-- Aluno 9
(33, 9, 2, 850.00, 850.00, '2025-02-15', '2025-02-15', 2, 2025),
(34, 9, 2, 850.00, 850.00, '2025-03-15', '2025-03-15', 3, 2025),
(35, 9, 2, 850.00, 850.00, '2025-04-15', '2025-04-15', 4, 2025),
(36, 9, 1, NULL, 850.00, '2025-05-15', NULL, 5, 2025),
-- Aluno 10

```

```
(37, 10, 2, 850.00, 850.00, '2025-02-15', '2025-02-15', 2, 2025),
(38, 10, 2, 850.00, 850.00, '2025-03-15', '2025-03-15', 3, 2025),
(39, 10, 2, 850.00, 850.00, '2025-04-15', '2025-04-15', 4, 2025),
(40, 10, 1, NULL, 850.00, '2025-05-15', NULL, 5, 2025);
```

```
-- _____
```

```
-- 41. tb_boletim (4 bimestres x 10 matrículas = 40 registros)
```

```
-- _____
```

```
INSERT IGNORE INTO tb_boletim (pk_boletim, fk_matricula, fk_situacao_boletim,
bimestre) VALUES
```

```
-- Matrícula 1 (aluno 1, turma 1)
```

```
(1, 1, 1, 1), (2, 1, 1, 2), (3, 1, 4, 3), (4, 1, 4, 4),
```

```
-- Matrícula 2 (aluno 2, turma 2)
```

```
(5, 2, 1, 1), (6, 2, 1, 2), (7, 2, 4, 3), (8, 2, 4, 4),
```

```
-- Matrícula 3 (aluno 3, turma 3)
```

```
(9, 3, 3, 1), (10, 3, 2, 2), (11, 3, 4, 3), (12, 3, 4, 4),
```

```
-- Matrícula 4 (aluno 4, turma 4)
```

```
(13, 4, 1, 1), (14, 4, 1, 2), (15, 4, 4, 3), (16, 4, 4, 4),
```

```
-- Matrícula 5 (aluno 5, turma 5)
```

```
(17, 5, 1, 1), (18, 5, 1, 2), (19, 5, 4, 3), (20, 5, 4, 4),
```

```
-- Matrícula 6 (aluno 6, turma 1)
```

```
(21, 6, 1, 1), (22, 6, 1, 2), (23, 6, 4, 3), (24, 6, 4, 4),
```

```
-- Matrícula 7 (aluno 7, turma 2)
```

```
(25, 7, 1, 1), (26, 7, 1, 2), (27, 7, 4, 3), (28, 7, 4, 4),
```

```
-- Matrícula 8 (aluno 8, turma 3)
```

```
(29, 8, 3, 1), (30, 8, 4, 2), (31, 8, 4, 3), (32, 8, 4, 4),
```

```
-- Matrícula 9 (aluno 9, turma 4)
```

```
(33, 9, 1, 1), (34, 9, 1, 2), (35, 9, 4, 3), (36, 9, 4, 4),
```

```
-- Matrícula 10 (aluno 10, turma 5)
```

```
(37, 10,1, 1), (38, 10,1, 2), (39, 10,4, 3), (40, 10,4, 4);
```

```

-- _____

-- 42. tb_atividade_professor (2 atividades por grade x bimestre 1 e 2)

-- _____

INSERT IGNORE INTO tb_atividade_professor (pk_atividade_professor, fk_grade,
fk_tipo_atividade, nome_atividade, bimestre, data_prevista, peso, valor_maximo) VALUES

-- Grade 1 (Turma 1A, Português)

(1, 1, 1, 'Prova 1 - Interpretação de Texto', 1, '2025-03-20', 2.00, 10.00),
(2, 1, 2, 'Trabalho 1 - Redação', 1, '2025-03-27', 1.50, 10.00),
(3, 1, 1, 'Prova 2 - Gramática', 2, '2025-06-05', 2.00, 10.00),
(4, 1, 2, 'Trabalho 2 - Conto', 2, '2025-06-12', 1.50, 10.00),
-- Grade 2 (Turma 1A, Matemática)
(5, 2, 1, 'Prova 1 - Operações Básicas', 1, '2025-03-21', 2.00, 10.00),
(6, 2, 3, 'Exercício 1 - Frações', 1, '2025-03-14', 1.00, 10.00),
(7, 2, 1, 'Prova 2 - Geometria', 2, '2025-06-06', 2.00, 10.00),
(8, 2, 3, 'Exercício 2 - Porcentagem', 2, '2025-06-13', 1.00, 10.00),
-- Grade 4 (Turma 2A, Português)
(9, 4, 1, 'Prova 1 - Leitura e Compreensão', 1, '2025-03-20', 2.00, 10.00),
(10, 4, 2, 'Trabalho 1 - Poesia', 1, '2025-03-28', 1.50, 10.00),
-- Grade 5 (Turma 2A, Matemática)
(11, 5, 1, 'Prova 1 - Álgebra', 1, '2025-03-21', 2.00, 10.00),
(12, 5, 3, 'Exercício 1 - Equações', 1, '2025-03-15', 1.00, 10.00),
-- Grade 6 (Turma 3A, Português)
(13, 6, 1, 'Prova 1 - Análise Sintática', 1, '2025-03-20', 2.00, 10.00),
(14, 6, 2, 'Trabalho 1 - Dissertação', 1, '2025-03-28', 1.50, 10.00),
-- Grade 7 (Turma 3A, Matemática)
(15, 7, 1, 'Prova 1 - Números Decimais', 1, '2025-03-21', 2.00, 10.00),
(16, 7, 3, 'Exercício 1 - Divisores', 1, '2025-03-15', 1.00, 10.00);

```

```

-- _____

-- 43. tb_nota_atividade

-- Regra trigger Grupo 10: atividade deve pertencer à turma da matrícula

-- Matrículas 1 e 6 → Turma 1 → grades 1, 2, 3 → atividades 1-8

-- Matrículas 2 e 7 → Turma 2 → grades 4, 5 → atividades 9-12

-- Matrículas 3 e 8 → Turma 3 → grades 6, 7 → atividades 13-16

-- _____

INSERT IGNORE INTO tb_nota_atividade (fk_matricula, fk_atividade_professor, valor_nota,
entregue_no_prazo) VALUES

-- Matrícula 1 (Turma 1A)

(1, 1, 8.50, TRUE),

(1, 2, 7.00, TRUE),

(1, 3, 9.00, TRUE),

(1, 4, 8.00, TRUE),

(1, 5, 7.50, TRUE),

(1, 6, 6.50, TRUE),

(1, 7, 8.00, TRUE),

(1, 8, 7.00, TRUE),

-- Matrícula 6 (Turma 1A)

(6, 1, 9.00, TRUE),

(6, 2, 8.50, TRUE),

(6, 3, 9.50, TRUE),

(6, 4, 8.00, TRUE),

(6, 5, 9.00, TRUE),

(6, 6, 8.00, TRUE),

(6, 7, 9.00, TRUE),

(6, 8, 8.50, TRUE),

-- Matrícula 2 (Turma 2A)

(2, 9, 8.00, TRUE),

```

```

(2, 10, 7.50, TRUE),

(2, 11, 8.50, TRUE),

(2, 12, 7.00, TRUE),

-- Matrícula 7 (Turma 2A)

(7, 9, 9.00, TRUE),

(7, 10, 8.00, TRUE),

(7, 11, 9.50, TRUE),

(7, 12, 8.50, TRUE),

-- Matrícula 3 (Turma 3A) – aluno com risco de evasão, algumas notas baixas

(3, 13, 4.50, TRUE),

(3, 14, NULL, FALSE), -- não entregou

(3, 15, 3.00, TRUE),

(3, 16, NULL, FALSE), -- não entregou

-- Matrícula 8 (Turma 3A)

(8, 13, 8.00, TRUE),

(8, 14, 7.50, TRUE),

(8, 15, 8.50, TRUE),

(8, 16, 7.00, TRUE);

-- _____

-- 44. tb_presenca_aluno

-- Regra trigger Grupo 9: horário deve pertencer à turma da matrícula

-- Horários 1-5 → grades 1, 2, 3 → Turma 1 → matrículas 1 e 6

-- Horários 6-9 → grades 4, 5 → Turma 2 → matrículas 2 e 7

-- Horários 10-13 → grades 6, 7 → Turma 3 → matrículas 3 e 8

-- _____

INSERT IGNORE INTO tb_presenca_aluno (pk_presenca, fk_matricula, fk_calendario_letivo,
fk_horario_aula, presenca, justificativa) VALUES

-- Calendário 1 (2025-02-03, segunda)

```

```

(1, 1, 1, 1, TRUE, NULL), -- matr.1, hor.1 (grade 1, turma 1, segunda)
(2, 6, 1, 1, TRUE, NULL), -- matr.6, hor.1
(3, 2, 1, 6, TRUE, NULL), -- matr.2, hor.6 (grade 4, turma 2, segunda)
(4, 7, 1, 6, TRUE, NULL), -- matr.7, hor.6
(5, 3, 1, 10,TRUE, NULL), -- matr.3, hor.10 (grade 6, turma 3, segunda)
(6, 8, 1, 10,TRUE, NULL), -- matr.8, hor.10
-- Calendário 2 (2025-02-04, terça)
(7, 1, 2, 3, TRUE, NULL), -- matr.1, hor.3 (grade 2, turma 1, terça)
(8, 6, 2, 3, TRUE, NULL),
(9, 2, 2, 8, TRUE, NULL), -- matr.2, hor.8 (grade 5, turma 2, terça)
(10, 7, 2, 8, TRUE, NULL),
(11, 3, 2, 12,TRUE, NULL), -- matr.3, hor.12 (grade 7, turma 3, terça)
(12, 8, 2, 12,TRUE, NULL),
-- Calendário 3 (2025-02-05, quarta)
(13, 1, 3, 2, TRUE, NULL), -- matr.1, hor.2 (grade 1, turma 1, quarta)
(14, 6, 3, 2, FALSE, 'Consulta médica'), -- faltou justificado
(15, 2, 3, 7, TRUE, NULL), -- matr.2, hor.7 (grade 4, turma 2, quarta)
(16, 7, 3, 7, TRUE, NULL),
(17, 3, 3, 11,FALSE, NULL), -- matr.3 falta sem justificativa
(18, 8, 3, 11,TRUE, NULL),
-- Calendário 4 (2025-03-03, segunda)
(19, 1, 4, 1, TRUE, NULL),
(20, 6, 4, 1, TRUE, NULL),
(21, 2, 4, 6, TRUE, NULL),
(22, 7, 4, 6, FALSE, NULL), -- matr.7 falta
(23, 3, 4, 10,FALSE, NULL), -- matr.3 falta
(24, 8, 4, 10,TRUE, NULL),
-- Calendário 5 (2025-03-04, terça)
(25, 1, 5, 3, TRUE, NULL),

```

```

(26, 6, 5, 3, TRUE, NULL),

(27, 2, 5, 8, TRUE, NULL),

(28, 7, 5, 8, TRUE, NULL),

(29, 3, 5, 12, FALSE, NULL), -- matr.3 terceira falta consecutiva

(30, 8, 5, 12, TRUE, NULL);

```

```

-- _____

-- 45. tb_alerta_evasao

-- Aluno 3 apresenta frequência baixa e inadimplência - gerado alertas

-- fk_responsavel_alerta = funcionário 4 (coordenador)

-- _____

```

```

INSERT IGNORE INTO tb_alerta_evasao (pk_alerta, fk_matricula, fk_responsavel_alerta,
fk_tipo_risco_evasao, fk_nivel_risco, fk_status_alerta, data_alerta, descricao,
data_resolucao, observacao_resolucao) VALUES

```

```

(1, 3, 4, 2, 3, 1, '2025-03-10 08:00:00', 'Aluno com 3 faltas consecutivas sem
justificativa no bimestre.', NULL, NULL),

```

```

(2, 3, 4, 1, 2, 2, '2025-04-01 09:00:00', 'Inadimplência: mensalidades de março e
abril em aberto.', NULL, NULL),

```

```

(3, 3, 4, 3, 3, 1, '2025-04-15 10:00:00', 'Notas abaixo da média em Português e
Matemática no 1º bimestre.', NULL, NULL),

```

```

(4, 8, 4, 3, 1, 3, '2025-02-28 08:00:00', 'Rendimento abaixo do esperado em Ciências
no início do bimestre.',

```

```

'2025-03-15 00:00:00', 'Aluno realizou atividade de recuperação e atingiu média
satisfatória.');
```

```

-- _____

-- 46. tb_folha_pagamento (fev e mar 2025 para os 5 contratos)

-- _____

```

```

INSERT IGNORE INTO tb_folha_pagamento (pk_folha_pagamento, fk_contrato, tipo_folha,
valor, ano, mes, data_emissao, status_folha) VALUES

```

```

-- Fevereiro 2025

```

```

(1, 1, 'folha_mensal', 5000.00, 2025, 2, '2025-02-28', 'pago'),

```

```

(2, 2, 'folha_mensal', 5300.00, 2025, 2, '2025-02-28', 'pago'),

```

```

(3, 3, 'folha_mensal', 3800.00, 2025, 2, '2025-02-28', 'pago'),
(4, 4, 'folha_mensal', 6500.00, 2025, 2, '2025-02-28', 'pago'),
(5, 5, 'folha_mensal', 2800.00, 2025, 2, '2025-02-28', 'pago'),
-- Março 2025
(6, 1, 'folha_mensal', 5000.00, 2025, 3, '2025-03-31', 'pago'),
(7, 2, 'folha_mensal', 5300.00, 2025, 3, '2025-03-31', 'pago'),
(8, 3, 'folha_mensal', 3800.00, 2025, 3, '2025-03-31', 'pago'),
(9, 4, 'folha_mensal', 6500.00, 2025, 3, '2025-03-31', 'pago'),
(10, 5, 'folha_mensal', 2800.00, 2025, 3, '2025-03-31', 'pago'),
-- Abril 2025
(11, 1, 'folha_mensal', 5000.00, 2025, 4, '2025-04-30', 'pendente'),
(12, 2, 'folha_mensal', 5300.00, 2025, 4, '2025-04-30', 'pendente'),
(13, 3, 'folha_mensal', 3800.00, 2025, 4, '2025-04-30', 'pendente'),
(14, 4, 'folha_mensal', 6500.00, 2025, 4, '2025-04-30', 'pendente'),
(15, 5, 'folha_mensal', 2800.00, 2025, 4, '2025-04-30', 'pendente');

-- _____

-- 47. tb_verba_folha (verbas de crédito e débito por folha)

-- _____

INSERT IGNORE INTO tb_verba_folha (fk_folha_pagamento, fk_tipo_verba, valor,
referencia) VALUES

-- Folha 1 (Contrato 1, fev/2025) - salário + VT + VA - INSS - IRRF
(1, 1, 5000.00, 'Salário base fev/2025'),
(1, 4, 220.00, 'VT fev/2025'),
(1, 5, 550.00, 'VA fev/2025'),
(1, 6, 550.00, 'INSS 11% fev/2025'),
(1, 7, 225.00, 'IRRF fev/2025'),
-- Folha 2 (Contrato 2, fev/2025)
(2, 1, 5300.00, 'Salário base fev/2025'),
(2, 4, 220.00, 'VT fev/2025'),

```

```

(2, 5, 550.00, 'VA fev/2025'),
(2, 6, 583.00, 'INSS 11% fev/2025'),
(2, 7, 280.00, 'IRRF fev/2025'),
-- Folha 6 (Contrato 1, mar/2025)
(6, 1, 5000.00, 'Salário base mar/2025'),
(6, 4, 220.00, 'VT mar/2025'),
(6, 5, 550.00, 'VA mar/2025'),
(6, 6, 550.00, 'INSS 11% mar/2025'),
(6, 7, 225.00, 'IRRF mar/2025'),
-- Folha 7 (Contrato 2, mar/2025)
(7, 1, 5300.00, 'Salário base mar/2025'),
(7, 4, 220.00, 'VT mar/2025'),
(7, 5, 550.00, 'VA mar/2025'),
(7, 6, 583.00, 'INSS 11% mar/2025'),
(7, 7, 280.00, 'IRRF mar/2025');

```

```

-- =====

```

```

SET FOREIGN_KEY_CHECKS = 1;

```

```

-- =====

```

```

-- VERIFICAÇÃO DE IDEMPOTÊNCIA – SELECT COUNT(*) em todas as tabelas
-- Execute após a 1ª carga, depois execute o script novamente e compare.
-- Os valores abaixo devem ser IDÊNTICOS nas duas execuções.

```

```

-- =====

```

```

SELECT  'tb_tipo_escola_anterior'          AS  tabela,  COUNT(*)  AS  total  FROM
tb_tipo_escola_anterior

UNION ALL SELECT  'tb_tipo_ingresso',          COUNT(*) FROM tb_tipo_ingresso

UNION ALL SELECT  'tb_status_matricula',          COUNT(*) FROM tb_status_matricula

```

UNION ALL SELECT 'tb_situacao_boletim',	COUNT(*) FROM tb_situacao_boletim
UNION ALL SELECT 'tb_nivel_risco_evasao',	COUNT(*) FROM tb_nivel_risco_evasao
UNION ALL SELECT 'tb_tipo_risco_evasao',	COUNT(*) FROM tb_tipo_risco_evasao
UNION ALL SELECT 'tb_status_alerta',	COUNT(*) FROM tb_status_alerta
UNION ALL SELECT 'tb_area_conhecimento',	COUNT(*) FROM tb_area_conhecimento
UNION ALL SELECT 'tb_materia',	COUNT(*) FROM tb_materia
UNION ALL SELECT 'tb_tipo_atividade',	COUNT(*) FROM tb_tipo_atividade
UNION ALL SELECT 'tb_tipo_aula',	COUNT(*) FROM tb_tipo_aula
UNION ALL SELECT 'tb_tipo_contrato',	COUNT(*) FROM tb_tipo_contrato
UNION ALL SELECT 'tb_cargo',	COUNT(*) FROM tb_cargo
UNION ALL SELECT 'tb_nivel_formacao',	COUNT(*) FROM tb_nivel_formacao
UNION ALL SELECT 'tb_profissao',	COUNT(*) FROM tb_profissao
UNION ALL SELECT 'tb_tipo_verba',	COUNT(*) FROM tb_tipo_verba
UNION ALL SELECT 'tb_ano_letivo',	COUNT(*) FROM tb_ano_letivo
UNION ALL SELECT 'tb_turma',	COUNT(*) FROM tb_turma
UNION ALL SELECT 'tb_calendario_letivo',	COUNT(*) FROM tb_calendario_letivo
UNION ALL SELECT 'tb_evento_escolar',	COUNT(*) FROM tb_evento_escolar
UNION ALL SELECT 'tb_pessoa',	COUNT(*) FROM tb_pessoa
UNION ALL SELECT 'tb_email',	COUNT(*) FROM tb_email
UNION ALL SELECT 'tb_telefone',	COUNT(*) FROM tb_telefone
UNION ALL SELECT 'tb_endereco',	COUNT(*) FROM tb_endereco
UNION ALL SELECT 'tb_aluno',	COUNT(*) FROM tb_aluno
UNION ALL SELECT 'tb_responsavel',	COUNT(*) FROM tb_responsavel
UNION ALL SELECT 'tb_responsavel_aluno',	COUNT(*) FROM tb_responsavel_aluno
UNION ALL SELECT 'tb_profissao_responsavel',	COUNT(*) FROM tb_profissao_responsavel
UNION ALL SELECT 'tb_funcionario',	COUNT(*) FROM tb_funcionario
UNION ALL SELECT 'tb_professor',	COUNT(*) FROM tb_professor
UNION ALL SELECT 'tb_formacao_professor',	COUNT(*) FROM tb_formacao_professor
UNION ALL SELECT 'tb_contrato',	COUNT(*) FROM tb_contrato

```

UNION ALL SELECT 'tb_historico_cargo',          COUNT(*) FROM tb_historico_cargo
UNION ALL SELECT 'tb_historico_salario',        COUNT(*) FROM tb_historico_salario
UNION ALL SELECT 'tb_grade_curricular',        COUNT(*) FROM tb_grade_curricular
UNION ALL SELECT 'tb_horario_aula',            COUNT(*) FROM tb_horario_aula
UNION ALL SELECT 'tb_matricula',               COUNT(*) FROM tb_matricula
UNION ALL SELECT 'tb_contrato_financeiro',      COUNT(*) FROM tb_contrato_financeiro
UNION ALL SELECT 'tb_status_pagamento',       COUNT(*) FROM tb_status_pagamento
UNION ALL SELECT 'tb_pagamento',              COUNT(*) FROM tb_pagamento
UNION ALL SELECT 'tb_boletim',                 COUNT(*) FROM tb_boletim
UNION ALL SELECT 'tb_atividade_professor',     COUNT(*) FROM tb_atividade_professor
UNION ALL SELECT 'tb_nota_atividade',          COUNT(*) FROM tb_nota_atividade
UNION ALL SELECT 'tb_presenca_aluno',          COUNT(*) FROM tb_presenca_aluno
UNION ALL SELECT 'tb_alerta_evasao',           COUNT(*) FROM tb_alerta_evasao
UNION ALL SELECT 'tb_folha_pagamento',        COUNT(*) FROM tb_folha_pagamento
UNION ALL SELECT 'tb_verba_folha',             COUNT(*) FROM tb_verba_folha;

-- FIM DO SCRIPT DE CARGA

```

4. Regras de Negócio e Analítica:

4.1 Regras de Negócio – Módulo Acadêmico:

Número	Área	Descrição
RN001	Acadêmico	Todo aluno deve possuir cadastro completo com dados pessoais obrigatórios (nome completo, CPF, data de nascimento, sexo e nacionalidade) registrados em tb_pessoa, mantidos permanentemente atualizados.
RN002	Acadêmico	Para efetuar matrícula no Ensino Fundamental I, o aluno deve

		ter no mínimo seis anos completos até a data de início do ano letivo registrado em tb_ano_letivo.
RN003	Acadêmico	Todo aluno matriculado deve estar obrigatoriamente vinculado a um turno de estudo (manhã ou tarde), definido no campo 'periodo' da tabela tb_turma.
RN004	Acadêmico	Cada disciplina de uma turma deve ter exatamente um professor responsável vinculado. Não é permitido manter um registro em tb_grade_curricular sem professor atribuído (fk_professor NOT NULL).
RN005	Acadêmico	Um professor pode ser responsável por mais de uma disciplina, inclusive em turmas diferentes, desde que cada vínculo seja registrado individualmente em tb_grade_curricular.
RN006	Acadêmico	Alunos com média final igual ou superior a 6,0 são considerados aprovados ao término do ano letivo. A situação deve ser registrada em tb_boletim com o status correspondente.
RN007	Acadêmico	Alunos com média final inferior a 5,0 são reprovados automaticamente, sem direito a avaliação de recuperação. A situação deve ser registrada em tb_boletim..
RN008	Acadêmico	Médias finais iguais a 5,75 devem ser arredondadas para 6,0 antes do registro no boletim, sendo então consideradas aprovadas conforme RN-A06. Este é o único caso de arredondamento previsto.
RN009	Acadêmico	Alunos com média final entre 5,0 (inclusive) e 6,0 (exclusive) exceto os contemplados por RN-08 têm direito a uma avaliação de recuperação. O resultado da recuperação substitui a nota do bimestre correspondente.
RN010	Acadêmico	Após a avaliação de recuperação, alunos com nota inferior a 5,0 em mais de uma disciplina são reprovados. O resultado final deve ser atualizado em tb_boletim via tb_situacao_boletim.
RN011	Acadêmico	Alunos com frequência anual inferior a 75% dos dias letivos são reprovados por falta, independentemente das notas obtidas. A frequência é calculada a partir dos registros em tb_presenca_aluno cruzados com os dias letivos em

		tb_calendario_letivo (letivo = TRUE)
RN012	Acadêmico	A média final do aluno é composta por: avaliações formais (60%), participação em sala (20%) e atividades complementares (20%). Os pesos são aplicados pela camada de aplicação com base nos valores em tb_nota_atividade e nos pesos definidos em tb_atividade_professor.peso.
RN013	Acadêmico	Faltas com justificativa registrada no campo 'justificativa' de tb_presenca_aluno não são descontadas do percentual de frequência e garantem ao aluno o direito a nova oportunidade de avaliação para a atividade perdida.
RN014	Acadêmico	O lançamento de notas pelos professores deve respeitar os prazos definidos pela coordenação. O sistema deve sinalizar lançamentos realizados após a data prevista da atividade.
RN015	Acadêmico	Uma turma não pode ter mais alunos matriculados com status 'ativa' do que o valor definido em tb_turma.capacidade_maxima. Novas matrículas devem ser bloqueadas ao atingir o limite.
RN016	Acadêmico	Não pode existir mais de uma turma com o mesmo nome no mesmo ano letivo e mesmo turno. Esta restrição é garantida pela constraint UNIQUE (fk_ano_letivo, nome_turma, periodo) em tb_turma.
RN017	Acadêmico	Transferências de turma devem preservar o histórico acadêmico do aluno. A matrícula anterior deve ter seu status atualizado para 'transferida' e uma nova matrícula deve ser criada na turma de destino, mantendo intactos os registros em tb_nota_atividade e tb_presenca_aluno..
RN018	Acadêmico	Registros acadêmicos não podem ser excluídos fisicamente do banco de dados. A inativação deve ser feita por exclusão lógica, por meio de campos de status (ex: fk_status_matricula em tb_matricula, ativo em tb_materia) ou por encerramento de vigência via data_fim..
RN019	Acadêmico	Não pode existir mais de um aluno cadastrado com o mesmo CPF no sistema. Esta unicidade é garantida pela constraint UNIQUE no campo 'cpf' da tabela tb_pessoa.
RN020	Acadêmico	O campo 'faz_atividade_extracurricular' em tb_aluno deve ser atualizado sempre que o aluno ingressar ou sair de uma

		atividade extracurricular, servindo como flag de controle para regras financeiras e de carga horária.
--	--	---

4.2 Regras de Negócio – Módulo RH:

Número	Área	Descrição
RN001	RH	Cada funcionário deve possuir cadastro completo com informações pessoais obrigatórias (nome, CPF, data de nascimento, sexo, nacionalidade) em tb_pessoa, mantidas permanentemente atualizadas.
RN002	RH	Todo funcionário ativo (tb_funcionario.ativo = TRUE) deve possuir credenciais de acesso ao sistema vinculadas ao seu cadastro, garantindo rastreabilidade das ações realizadas.
RN003	RH	O histórico profissional do funcionário deve ser mantido permanentemente, incluindo todos os cargos exercidos (tb_historico_cargo) e todos os reajustes salariais recebidos (tb_historico_salario), para fins de auditoria e análise de RH.
RN004	RH	Não é permitido o cadastro de mais de um funcionário com o mesmo CPF. Esta unicidade é garantida pela constraint UNIQUE no campo 'cpf' de tb_pessoa..
RN005	RH	Em caso de falta justificada, as horas não trabalhadas não geram desconto na remuneração do funcionário nem são lançadas como débito no banco de horas. O impacto deve ser refletido nas verbas registradas em tb_verba_folha.
RN006	RH	A solicitação de férias só pode ocorrer após um ano completo de vínculo empregatício, contado a partir da data de início registrada em tb_contrato.data_inicio.
RN007	RH	Professores podem ser alocados em turmas dos períodos matutino e vespertino simultaneamente, desde que cada vínculo seja registrado individualmente em tb_grade_curricular.
RN008	RH	A folha de pagamento deve considerar o salário base vigente (tb_historico_salario), os créditos e descontos aplicáveis (tb_verba_folha, com natureza_verba = 'credito' ou 'debito') e a carga horária contratada (tb_contrato.carga_horaria_semanal).
RN009	RH	Os dados bancários do funcionário são obrigatórios para o processamento do pagamento de salário e devem estar sempre

		atualizados.
RN010	RH	Todo pagamento de salário processado deve gerar informe de rendimentos ao funcionário, com detalhamento de créditos e descontos aplicados na folha do período.
RN011	RH + Acadêmico	Um professor só pode ser vinculado a uma disciplina em tb_grade_curricular se possuir ao menos uma formação registrada em tb_formacao_professor compatível com a área do conhecimento da disciplina (tb_materia.fk_area_conhecimento).
RN012	RH + Acadêmico	Um professor só pode ser vinculado à supervisão de uma atividade extracurricular se possuir formação e/ou habilitação comprovada na área correspondente, registrada em tb_formacao_professor.
RN013	RH + Financeiro	Ao fechar a folha de pagamento de um período (status em tb_folha_pagamento atualizado para o status equivalente a 'pago'), o valor total líquido deve ser registrado como conta a pagar no módulo financeiro, integrando os dois módulos.

4.3 Regras de Negócio – Módulo Financeiro:

Número	Área	Descrição
RN001	Financeiro	Toda cobrança em tb_pagamento deve estar vinculada a um

		contrato financeiro ativo em tb_contrato_financeiro, que por sua vez deve estar associado a uma matrícula com status 'ativa' em tb_matricula..
RN002	Financeiro	Um contrato financeiro só pode ser encerrado (data_fim preenchida em tb_contrato_financeiro) se não houver registros em tb_pagamento vinculados com status pendente ou atrasado.
RN003	Financeiro	Não pode existir mais de uma cobrança do mesmo mês e ano de referência para o mesmo contrato financeiro. Esta unicidade é garantida pela constraint UNIQUE (fk_contrato_financeiro, mes_referencia, ano_referencia) em tb_pagamento.
RN004	Financeiro	O valor final da cobrança deve considerar o valor base do contrato (tb_contrato_financeiro.valor_mensalidade), acrescido de juros e multas por atraso quando aplicável, e subtraído de descontos autorizados. O resultado deve ser registrado em tb_pagamento.valor_cobrado.
RN005	Financeiro	A reemissão de boleto bancário não pode gerar um novo registro em tb_pagamento. Apenas o documento de cobrança externo é atualizado; o registro existente no banco permanece inalterado.
RN006	Financeiro	Cobranças referentes a itens extras (passeios, materiais, atividades) devem ser registradas como parcelas distintas em tb_pagamento, com mes_referencia e ano_referencia correspondentes ao período de competência.
RN007	Financeiro	Descontos familiares devem seguir a política institucional: para 2 filhos matriculados simultaneamente, aplica-se 10% de desconto na mensalidade do segundo; para 3 ou mais filhos, 15% a partir do terceiro. Os valores com desconto devem ser refletidos em tb_pagamento.valor_cobrado.
RN008	Financeiro	A confirmação de um pagamento deve alterar automaticamente o status em tb_pagamento (via fk_status_pagamento) de pendente ou atrasado para pago, e registrar a data efetiva do pagamento em tb_pagamento.data_pagamento.
RN009	Financeiro	Estornos só podem ser realizados em pagamentos cujo status

		em tb_pagamento seja equivalente a 'pago'. Ao estornar, o status deve ser atualizado para 'estornado' e o valor em tb_pagamento.valor_pago deve ser zerado.
RN010	Financeiro	Juros e multas por atraso devem ser calculados proporcionalmente ao tempo em aberto: multa de 2% sobre o valor original na data do vencimento, acrescida de juros de 0,033% ao dia. O total resultante deve ser registrado em tb_pagamento.valor_cobrado.
RN011	Financeiro	Alunos com qualquer registro em tb_pagamento com status equivalente a 'atrasado' ou 'pendente' de período anterior não podem renovar a matrícula. A renovação deve ser bloqueada automaticamente pelo sistema..
RN012	Financeiro + Acadêmico	A geração de cobrança referente a atividades extracurriculares só pode ocorrer se o campo 'faz_atividade_extracurricular' estiver marcado como TRUE em tb_aluno para o aluno em questão.
RN013	Financeiro + Acadêmico	A cobrança de taxa de rematrícula só pode ser gerada para alunos cuja situação acadêmica em tb_boletim (via tb_situacao_boletim) esteja definida como 'Aprovado' ou 'Reprovado' ao final do ano letivo. Alunos com situação 'Em curso' não podem ser cobrados.
RN014	Financeiro + Acadêmico	A renovação de matrícula de um aluno com pendências financeiras em aberto deve ser automaticamente bloqueada, impedindo a criação de novo registro em tb_matricula.
RN015	Financeiro + Acadêmico	A participação de um aluno em passeios e eventos escolares está condicionada ao pagamento da cobrança correspondente. O sistema deve verificar se o registro em tb_pagamento vinculado ao evento possui status equivalente a 'pago' antes de liberar a participação.

4.4 Regras de Negócio – Alerta de Evasão Escolar:

Número	Área	Descrição
RN001	Acadêmico	Todo alerta de evasão registrado em tb_alerta_evasao deve estar obrigatoriamente vinculado a uma matrícula ativa (fk_matricula), a um funcionário responsável pelo acompanhamento (fk_responsavel_alerta), a um tipo de risco (fk_tipo_risco_evasao) e a um nível de criticidade (fk_nivel_risco)..
RN002	Acadêmico	Os níveis de risco cadastrados em tb_nivel_risco_evasao devem seguir uma ordem de criticidade crescente, definida pelo campo 'ordem'. Isso permite ao sistema e aos gestores identificar e priorizar os casos mais críticos.
RN003	Acadêmico	Ao registrar um alerta, o status inicial em tb_status_alerta deve ser equivalente a 'aberto'. O status deve ser atualizado conforme o acompanhamento do caso (em acompanhamento, resolvido ou evadiu).
RN004	Acadêmico	Ao encerrar um alerta (status 'resolvido' ou 'evadiu'), os campos data_resolucao e observacao_resolucao em tb_alerta_evasao devem ser obrigatoriamente preenchidos, registrando o desfecho do caso.
RN005	Acadêmico + Financeiro	O sistema deve ser capaz de gerar alertas de evasão de forma automática quando identificar combinações de fatores de risco, como: frequência abaixo de 75% em tb_presenca_aluno, queda consecutiva de notas em tb_nota_atividade, ou inadimplência registrada em tb_pagamento. O tipo de risco correspondente deve ser selecionado de tb_tipo_risco_evasao.

4.5 Observação sobre uso de dados para BI/IA:

O modelo relacional do SisGESC foi projetado para suportar tanto operações transacionais (OLTP) quanto análises estratégicas por meio de Business Intelligence (BI) e Inteligência Artificial (IA), com base nos dados gerados pelos módulos Acadêmico, Financeiro e de Recursos Humanos.

4.5.1 — Desempenho Acadêmico:

O cruzamento entre `tb_nota_atividade`, `tb_presenca_aluno` e `tb_boletim` permite construir análises históricas do rendimento dos alunos por bimestre, disciplina e turma. Esses dados podem alimentar painéis de BI para acompanhamento pedagógico e treinar modelos preditivos capazes de identificar, com antecedência, alunos com risco de reprovação ou evasão. Quando um caso de risco é detectado, o sistema pode registrar automaticamente um alerta em `tb_alerta_evasao`, classificado por tipo (`tb_tipo_risco_evasao`) e criticidade (`tb_nivel_risco_evasao`).

4.5.2 — Inadimplência Financeira:

A partir de `tb_pagamento` e `tb_contrato_financeiro`, é possível construir indicadores como receita prevista versus realizada, tempo médio de atraso e percentual de contratos em aberto por período. Esses indicadores apoiam a gestão financeira na identificação de padrões de inadimplência e podem ser utilizados para treinar modelos de previsão de atraso, subsidiando ações preventivas de cobrança.

4.5.3 — Gestão de RH:

As tabelas `tb_historico_cargo`, `tb_historico_salario`, `tb_folha_pagamento` e `tb_verba_folha` permitem analisar rotatividade de cargos, evolução salarial e custo total de folha por período. O cruzamento entre `tb_formacao_professor` e `tb_nota_atividade`, via `tb_grade_curricular`, viabiliza análises correlacionais sobre o impacto da qualificação docente no desempenho das turmas.

4.5.4 — Visão Estratégica Integrada:

A integração entre os módulos permite diagnósticos multifatoriais de evasão escolar, combinando queda de notas (`tb_nota_atividade`), baixa frequência (`tb_presenca_aluno`) e inadimplência (`tb_pagamento`) em um único alerta consolidado em `tb_alerta_evasao`. O banco de dados está estruturado para integração futura com ferramentas de BI, como Power BI ou Metabase, e com pipelines de aprendizado de máquina que utilizem o histórico de matrículas encerradas registrado via `tb_status_matricula` como variável-alvo para modelos supervisionados de previsão de evasão.

5. Modelo Analítico OLAP – Star Schema

O modelo OLAP do SisGESC adota a arquitetura Star Schema: tabelas fato registram eventos e métricas no centro, e tabelas dimensão fornecem o contexto analítico ao redor. As Dimensões Conformadas são compartilhadas entre os módulos Acadêmico, Financeiro, RH e Gestão, garantindo análises cruzadas consistentes.

5.1 Diagrama Star Schema

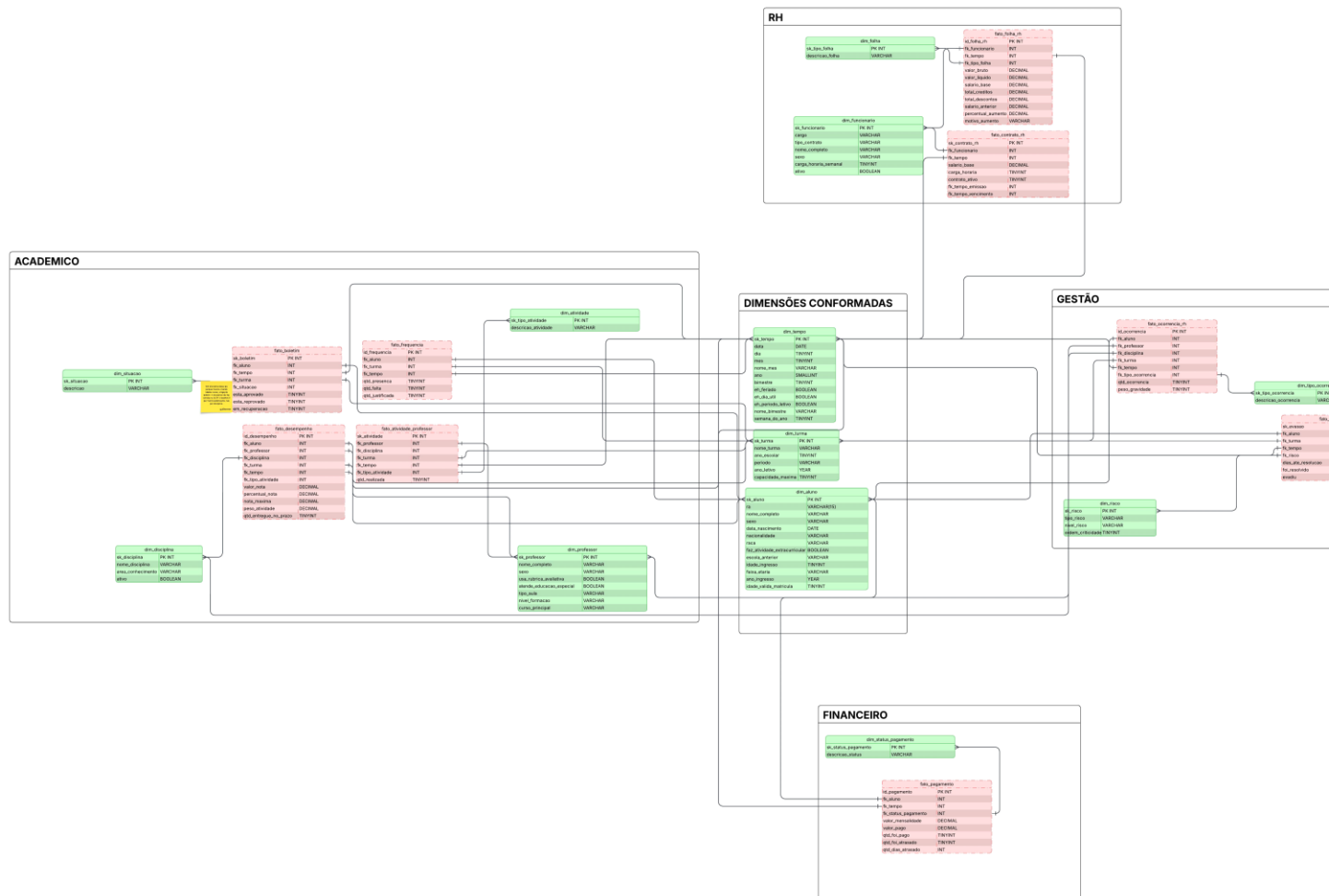


Diagrama Star Schema completo com as Dimensões Conformadas no centro e as Tabelas Fato organizadas por módulo.

5.1 SCRIPT ETL

```
-- DATA WAREHOUSE - AURORA
-- ETL COMPLETO (Iterações 1 + 2 unificadas)
--
-- Cobertura:
--   Dimensões : dim_tempo, dim_aluno, dim_funcionario, dim_professor,
--               dim_disciplina, dim_turma, dim_status_pagamento,
--               dim_atividade, dim_folha,
--               dim_risco, dim_situacao
--
--   Fatos      : fato_folha_rh, fato_desempenho,
--               fato_frequencia, fato_boletim,
--               fato_atividade_professor, fato_pagamento,
```

```

--      fato_evasao, fato_contrato_rh
-- =====

-- =====
-- 1. CRIAÇÃO DO BANCO
-- =====

CREATE DATABASE IF NOT EXISTS dw_aurora;
USE dw_aurora;

SET FOREIGN_KEY_CHECKS = 0;

-- Fatos (dependem das dims — removidos primeiro)
DROP TABLE IF EXISTS fato_contrato_rh;
DROP TABLE IF EXISTS fato_evasao;
DROP TABLE IF EXISTS fato_pagamento;
DROP TABLE IF EXISTS fato_atividade_professor;
DROP TABLE IF EXISTS fato_boletim;
DROP TABLE IF EXISTS fato_frequencia;
DROP TABLE IF EXISTS fato_folha_rh;
DROP TABLE IF EXISTS fato_desempenho;

-- Dimensões da iteração 2
DROP TABLE IF EXISTS dim_risco;
DROP TABLE IF EXISTS dim_situacao;

-- Dimensões da iteração 1
DROP TABLE IF EXISTS dim_tempo;
DROP TABLE IF EXISTS dim_aluno;
DROP TABLE IF EXISTS dim_funcionario;
DROP TABLE IF EXISTS dim_professor;
DROP TABLE IF EXISTS dim_disciplina;
DROP TABLE IF EXISTS dim_turma;
DROP TABLE IF EXISTS dim_status_pagamento;
DROP TABLE IF EXISTS dim_atividade;
DROP TABLE IF EXISTS dim_folha;

SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- 2. DIMENSÕES
-- =====

-- =====
-- dim_tempo (calendário completo 2020-2030)
-- =====

CREATE TABLE dim_tempo (
    sk_tempo      INT      PRIMARY KEY,
    data_completa  DATE,
    dia           TINYINT,
    mes           TINYINT,

```

```

nome_mes      VARCHAR(15),
semana_do_ano TINYINT,
ano           SMALLINT,
bimestre      TINYINT NULL,
nome_bimestre VARCHAR(20),
eh_feriado    BOOLEAN DEFAULT FALSE,
eh_dia_util   BOOLEAN DEFAULT FALSE,
eh_periodo_letivo BOOLEAN DEFAULT FALSE
);

```

```

-----
-- dim_aluno (com atributos demográficos e derivados)
-----

```

```

CREATE TABLE dim_aluno (
  sk_aluno      INT      PRIMARY KEY AUTO_INCREMENT,
  pk_aluno_origem INT,
  ra            VARCHAR(15),
  nome_completo VARCHAR(100),
  sexo          VARCHAR(15),
  data_nascimento DATE,
  nacionalidade VARCHAR(15),
  raca          VARCHAR(15),
  faz_atividade_extracurricular BOOLEAN,
  escola_anterior VARCHAR(30),
  idade_ingresso TINYINT,
  faixa_etaria   VARCHAR(20),
  ano_ingresso  YEAR,
  idade_valida_matricula TINYINT -- 1 = idade compatível com ensino fundamental
);

```

```

-----
-- dim_funcionario
-----

```

```

CREATE TABLE dim_funcionario (
  sk_funcionario INT      PRIMARY KEY AUTO_INCREMENT,
  pk_funcionario_origem INT,
  nome_completo  VARCHAR(100),
  sexo          VARCHAR(15),
  cargo         VARCHAR(60),
  tipo_contrato  VARCHAR(30),
  carga_horaria_semanal TINYINT,
  ativo         BOOLEAN
);

```

```

-----
-- dim_professor
-----

```

```

CREATE TABLE dim_professor (
  sk_professor INT      PRIMARY KEY AUTO_INCREMENT,
  pk_professor_origem INT,
  nome_completo  VARCHAR(100),

```

```

    sexo          VARCHAR(15),
    usa_rubrica_avaliativa  BOOLEAN,
    atende_educacao_especial  BOOLEAN,
    tipo_aula      VARCHAR(40),
    nivel_formacao VARCHAR(40),
    curso_principal VARCHAR(100)
);

```

```

-----
-- dim_disciplina
-----

```

```

CREATE TABLE dim_disciplina (
    sk_disciplina  INT      PRIMARY KEY AUTO_INCREMENT,
    pk_materia_origem  INT,
    nome_disciplina VARCHAR(60),
    area_conhecimento VARCHAR(40),
    ativo          BOOLEAN
);

```

```

-----
-- dim_turma
-----

```

```

CREATE TABLE dim_turma (
    sk_turma      INT      PRIMARY KEY AUTO_INCREMENT,
    pk_turma_origem  INT,
    nome_turma     VARCHAR(50),
    ano_escolar    TINYINT,
    periodo        VARCHAR(20),
    ano_letivo      YEAR,
    capacidade_maxima  TINYINT
);

```

```

-----
-- dim_status_pagamento
-----

```

```

CREATE TABLE dim_status_pagamento (
    sk_status_pagamento INT      PRIMARY KEY AUTO_INCREMENT,
    pk_status_origem  INT,
    descricao_status   VARCHAR(30)
);

```

```

-----
-- dim_atividade (tipo de atividade — prova, trabalho, seminário, etc.)
-----

```

```

CREATE TABLE dim_atividade (
    sk_tipo_atividade  INT      PRIMARY KEY AUTO_INCREMENT,
    pk_tipo_atividade_origem  INT,
    descricao_atividade VARCHAR(60)
);

```

```

-----

```

-- dim_folha (tipo de folha — mensal, 13º, rescisão, etc.)
-- Não tem PK na origem (é ENUM), então usamos lista fixa

```
-----  
CREATE TABLE dim_folha (  
    sk_tipo_folha    INT      PRIMARY KEY AUTO_INCREMENT,  
    descricao_folha  VARCHAR(30)  
);
```

-- dim_risco
-- Origem: tb_tipo_risco_evasao + tb_nivel_risco (OLTP)
-- Combina tipo e nível em uma dimensão degenerada de risco,
-- facilitando análises de evasão sem múltiplos JOINS.

```
-----  
CREATE TABLE dim_risco (  
    sk_risco          INT      PRIMARY KEY AUTO_INCREMENT,  
    pk_tipo_risco_orig INT,  
    pk_nivel_risco_orig INT,  
    tipo_risco        VARCHAR(60),  
    nivel_risco        VARCHAR(40),  
    ordem_criticidade TINYINT -- 1 = menor risco, valores maiores = mais crítico  
);
```

-- dim_situacao
-- Origem: tb_situacao_boletim (OLTP)
-- Representa a situação do aluno ao final de cada bimestre
-- (aprovado, reprovado, em recuperação, etc.)

```
-----  
CREATE TABLE dim_situacao (  
    sk_situacao      INT      PRIMARY KEY AUTO_INCREMENT,  
    pk_situacao_orig INT,  
    descricao        VARCHAR(60)  
);
```

-- =====
-- 3. TABELAS FATO
-- =====

```
CREATE TABLE fato_folha_rh (  
    id_folha_rh      INT      PRIMARY KEY AUTO_INCREMENT,  
    fk_funcionario    INT,  
    fk_tempo          INT,  
    fk_tipo_folha     INT,  
    salario_base      DECIMAL(10,2),  
    valor_bruto        DECIMAL(10,2),  
    valor_liquido      DECIMAL(10,2),  
    total_creditos     DECIMAL(10,2),  
    total_descontos    DECIMAL(10,2),  
    salario_anterior  DECIMAL(10,2),  
    percentual_aumento DECIMAL(6,2),
```

```

motivo_aumento    VARCHAR(100),
CONSTRAINT fk_folha_funcionario
    FOREIGN KEY (fk_funcionario) REFERENCES dim_funcionario(sk_funcionario),
CONSTRAINT fk_folha_tempo
    FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
CONSTRAINT fk_folha_tipo
    FOREIGN KEY (fk_tipo_folha) REFERENCES dim_folha(sk_tipo_folha)
);

```

```

CREATE TABLE fato_desempenho (
    id_desempenho    INT          PRIMARY KEY AUTO_INCREMENT,
    fk_aluno         INT,
    fk_professor     INT,
    fk_disciplina    INT,
    fk_turma         INT,
    fk_tempo         INT,
    fk_tipo_atividade INT,
    valor_nota       DECIMAL(4,2),
    nota_maxima      DECIMAL(4,2),
    percentual_nota  DECIMAL(5,2),
    peso_atividade   DECIMAL(4,2),
    qtd_entregue_no_prazo TINYINT,
    CONSTRAINT fk_desempenho_aluno
        FOREIGN KEY (fk_aluno) REFERENCES dim_aluno(sk_aluno),
    CONSTRAINT fk_desempenho_professor
        FOREIGN KEY (fk_professor) REFERENCES dim_professor(sk_professor),
    CONSTRAINT fk_desempenho_disciplina
        FOREIGN KEY (fk_disciplina) REFERENCES dim_disciplina(sk_disciplina),
    CONSTRAINT fk_desempenho_turma
        FOREIGN KEY (fk_turma) REFERENCES dim_turma(sk_turma),
    CONSTRAINT fk_desempenho_tempo
        FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
    CONSTRAINT fk_desempenho_atividade
        FOREIGN KEY (fk_tipo_atividade) REFERENCES dim_atividade(sk_tipo_atividade)
);

```

```

-- -----
-- fato_frequencia
-- Granularidade: 1 linha por aluno x turma x dia
-- Origem: tb_frequencia (OLTP)
-- Métricas: qtd_presenca, qtd_falta, qtd_justificada
-- -----

```

```

CREATE TABLE fato_frequencia (
    id_frequencia    INT          PRIMARY KEY AUTO_INCREMENT,
    fk_aluno         INT          NOT NULL,
    fk_turma         INT          NOT NULL,
    fk_tempo         INT          NOT NULL,
    qtd_presenca     TINYINT      NOT NULL DEFAULT 0,
    qtd_falta        TINYINT      NOT NULL DEFAULT 0,
    qtd_justificada   TINYINT      NOT NULL DEFAULT 0,
    CONSTRAINT fk_freq_aluno

```

```

        FOREIGN KEY (fk_aluno) REFERENCES dim_aluno(sk_aluno),
        CONSTRAINT fk_freq_turma
        FOREIGN KEY (fk_turma) REFERENCES dim_turma(sk_turma),
        CONSTRAINT fk_freq_tempo
        FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo)
);

```

```

-----
-- fato_boletim
-- Granularidade: 1 linha por aluno × turma × bimestre
-- Origem: tb_boletim (OLTP)
-- Métricas: flags de aprovação, reprovação e recuperação
-----

```

```

CREATE TABLE fato_boletim (
    sk_boletim      INT      PRIMARY KEY AUTO_INCREMENT,
    fk_aluno        INT      NOT NULL,
    fk_turma        INT      NOT NULL,
    fk_tempo        INT      NOT NULL,
    fk_situacao     INT      NOT NULL,
    esta_aprovado   TINYINT  NOT NULL DEFAULT 0, -- 1 = sim
    esta_reprovado  TINYINT  NOT NULL DEFAULT 0,
    em_recuperacao  TINYINT  NOT NULL DEFAULT 0,
    CONSTRAINT fk_boletim_aluno
        FOREIGN KEY (fk_aluno) REFERENCES dim_aluno(sk_aluno),
    CONSTRAINT fk_boletim_turma
        FOREIGN KEY (fk_turma) REFERENCES dim_turma(sk_turma),
    CONSTRAINT fk_boletim_tempo
        FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
    CONSTRAINT fk_boletim_situacao
        FOREIGN KEY (fk_situacao) REFERENCES dim_situacao(sk_situacao)
);

```

```

-----
-- fato_atividade_professor
-- Granularidade: 1 linha por professor × disciplina × turma × tipo de
--                atividade × data prevista
-- Origem: tb_atividade_professor (OLTP)
-- Métrica: qtd_realizada (agregação de atividades planejadas)
-----

```

```

CREATE TABLE fato_atividade_professor (
    sk_atividade     INT      PRIMARY KEY AUTO_INCREMENT,
    fk_professor     INT      NOT NULL,
    fk_disciplina    INT      NOT NULL,
    fk_turma         INT      NOT NULL,
    fk_tempo         INT      NOT NULL,
    fk_tipo_atividade INT      NOT NULL,
    qtd_realizada    TINYINT  NOT NULL DEFAULT 0,
    CONSTRAINT fk_atv_prof_professor
        FOREIGN KEY (fk_professor) REFERENCES dim_professor(sk_professor),
    CONSTRAINT fk_atv_prof_disciplina
        FOREIGN KEY (fk_disciplina) REFERENCES dim_disciplina(sk_disciplina),

```

```

CONSTRAINT fk_atv_prof_turma
    FOREIGN KEY (fk_turma) REFERENCES dim_turma(sk_turma),
CONSTRAINT fk_atv_prof_tempo
    FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
CONSTRAINT fk_atv_prof_tipo
    FOREIGN KEY (fk_tipo_atividade) REFERENCES dim_atividade(sk_tipo_atividade)
);

-----
-- fato_pagamento
-- Granularidade: 1 linha por aluno x mês de vencimento
-- Origem: tb_cobranca (OLTP)
-- Métricas: valor_mensalidade, valor_pago, flags de pagamento e atraso
-----
CREATE TABLE fato_pagamento (
    id_pagamento INT PRIMARY KEY AUTO_INCREMENT,
    fk_aluno INT NOT NULL,
    fk_tempo INT NOT NULL, -- mês/ano do vencimento
    fk_status_pagamento INT NOT NULL,
    valor_mensalidade DECIMAL(10,2),
    valor_pago DECIMAL(10,2),
    qtd_foi_pago TINYINT NOT NULL DEFAULT 0, -- 1 = pago
    qtd_foi_atrasado TINYINT NOT NULL DEFAULT 0, -- 1 = houve atraso
    qtd_dias_atrasado INT NOT NULL DEFAULT 0,
    CONSTRAINT fk_pgto_aluno
        FOREIGN KEY (fk_aluno) REFERENCES dim_aluno(sk_aluno),
    CONSTRAINT fk_pgto_tempo
        FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
    CONSTRAINT fk_pgto_status
        FOREIGN KEY (fk_status_pagamento) REFERENCES
dim_status_pagamento(sk_status_pagamento)
);

-----
-- fato_evasao
-- Granularidade: 1 linha por alerta de evasão
-- Origem: tb_alerta_evasao (OLTP)
-- Métricas: dias_ate_resolucao, flags foi_resolvido e evadiu
-----
CREATE TABLE fato_evasao (
    sk_evasao INT PRIMARY KEY AUTO_INCREMENT,
    fk_aluno INT NOT NULL,
    fk_turma INT NOT NULL,
    fk_tempo INT NOT NULL, -- data do alerta
    fk_risco INT NOT NULL,
    dias_ate_resolucao DECIMAL(8,2), -- NULL se ainda não resolvido
    foi_resolvido TINYINT NOT NULL DEFAULT 0, -- 1 = resolvido
    evadiu TINYINT NOT NULL DEFAULT 0, -- 1 = aluno saiu
    CONSTRAINT fk_evasao_aluno
        FOREIGN KEY (fk_aluno) REFERENCES dim_aluno(sk_aluno),
    CONSTRAINT fk_evasao_turma

```

```

        FOREIGN KEY (fk_turma) REFERENCES dim_turma(sk_turma),
        CONSTRAINT fk_evasao_tempo
        FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
        CONSTRAINT fk_evasao_risco
        FOREIGN KEY (fk_risco) REFERENCES dim_risco(sk_risco)
    );

-- =====
-- fato_contrato_rh
-- Granularidade: 1 linha por contrato (foto atual, não histórico)
-- Origem: tb_contrato + tb_historico_salario (OLTP)
-- Métricas: salario_base, carga_horaria, contrato_ativo
-- fk_tempo_emissao = sk_tempo da data_inicio do contrato
-- fk_tempo_vencimento = sk_tempo da data_fim (NULL = indeterminado)
-- =====
CREATE TABLE fato_contrato_rh (
    sk_contrato_rh INT PRIMARY KEY AUTO_INCREMENT,
    fk_funcionario INT NOT NULL,
    fk_tempo INT NOT NULL, -- mês de referência
    fk_tempo_emissao INT, -- sk_tempo da data_inicio do contrato
    fk_tempo_vencimento INT, -- sk_tempo da data_fim (NULL = indeterminado)
    salario_base DECIMAL(10,2),
    carga_horaria TINYINT,
    contrato_ativo TINYINT NOT NULL DEFAULT 1, -- 1 = ativo
    CONSTRAINT fk_contrato_funcionario
        FOREIGN KEY (fk_funcionario) REFERENCES dim_funcionario(sk_funcionario),
    CONSTRAINT fk_contrato_tempo
        FOREIGN KEY (fk_tempo) REFERENCES dim_tempo(sk_tempo),
    CONSTRAINT fk_contrato_tempo_emissao
        FOREIGN KEY (fk_tempo_emissao) REFERENCES dim_tempo(sk_tempo),
    CONSTRAINT fk_contrato_tempo_venc
        FOREIGN KEY (fk_tempo_vencimento) REFERENCES dim_tempo(sk_tempo)
);

-- =====
-- 4. ÍNDICES
-- =====

-- fato_folha_rh
CREATE INDEX idx_folha_funcionario ON fato_folha_rh(fk_funcionario);
CREATE INDEX idx_folha_tempo ON fato_folha_rh(fk_tempo);
CREATE INDEX idx_folha_tipo ON fato_folha_rh(fk_tipo_folha);

-- fato_desempenho
CREATE INDEX idx_desempenho_aluno ON fato_desempenho(fk_aluno);
CREATE INDEX idx_desempenho_professor ON fato_desempenho(fk_professor);
CREATE INDEX idx_desempenho_disciplina ON fato_desempenho(fk_disciplina);
CREATE INDEX idx_desempenho_turma ON fato_desempenho(fk_turma);
CREATE INDEX idx_desempenho_tempo ON fato_desempenho(fk_tempo);
CREATE INDEX idx_desempenho_atividade ON fato_desempenho(fk_tipo_atividade);

```

```

-- fato_frequencia
CREATE INDEX idx_freq_aluno ON fato_frequencia(fk_aluno);
CREATE INDEX idx_freq_turma ON fato_frequencia(fk_turma);
CREATE INDEX idx_freq_tempo ON fato_frequencia(fk_tempo);

-- fato_boletim
CREATE INDEX idx_boletim_aluno ON fato_boletim(fk_aluno);
CREATE INDEX idx_boletim_turma ON fato_boletim(fk_turma);
CREATE INDEX idx_boletim_tempo ON fato_boletim(fk_tempo);
CREATE INDEX idx_boletim_situacao ON fato_boletim(fk_situacao);

-- fato_atividade_professor
CREATE INDEX idx_atvprof_professor ON fato_atividade_professor(fk_professor);
CREATE INDEX idx_atvprof_disciplina ON fato_atividade_professor(fk_disciplina);
CREATE INDEX idx_atvprof_turma ON fato_atividade_professor(fk_turma);
CREATE INDEX idx_atvprof_tempo ON fato_atividade_professor(fk_tempo);
CREATE INDEX idx_atvprof_tipo ON fato_atividade_professor(fk_tipo_atividade);

-- fato_pagamento
CREATE INDEX idx_pgto_aluno ON fato_pagamento(fk_aluno);
CREATE INDEX idx_pgto_tempo ON fato_pagamento(fk_tempo);
CREATE INDEX idx_pgto_status ON fato_pagamento(fk_status_pagamento);

-- fato_evasao
CREATE INDEX idx_evasao_aluno ON fato_evasao(fk_aluno);
CREATE INDEX idx_evasao_turma ON fato_evasao(fk_turma);
CREATE INDEX idx_evasao_tempo ON fato_evasao(fk_tempo);
CREATE INDEX idx_evasao_risco ON fato_evasao(fk_risco);

-- fato_contrato_rh
CREATE INDEX idx_contrato_funcionario ON fato_contrato_rh(fk_funcionario);
CREATE INDEX idx_contrato_tempo ON fato_contrato_rh(fk_tempo);

-- =====
-- 5. CARGA DA DIM_TEMPO
-- =====
-- Gera intervalo completo 2020-01-01 a 2030-12-31
-- =====

SET lc_time_names = 'pt_BR';

INSERT INTO dim_tempo (
    sk_tempo, data_completa, dia, mes, nome_mes, semana_do_ano, ano,
    bimestre, nome_bimestre, eh_feriado, eh_dia_util, eh_periodo_letivo
)
SELECT
    CAST(DATE_FORMAT(d, '%Y%m%d') AS UNSIGNED),
    d,
    DAY(d),
    MONTH(d),
    MONTHNAME(d),

```

```

WEEK(d, 3),
YEAR(d),
CASE
    WHEN MONTH(d) BETWEEN 2 AND 4 THEN 1
    WHEN MONTH(d) BETWEEN 5 AND 7 THEN 2
    WHEN MONTH(d) BETWEEN 8 AND 9 THEN 3
    WHEN MONTH(d) BETWEEN 10 AND 12 THEN 4
    ELSE NULL
END,
CASE
    WHEN MONTH(d) BETWEEN 2 AND 4 THEN '1º Bimestre'
    WHEN MONTH(d) BETWEEN 5 AND 7 THEN '2º Bimestre'
    WHEN MONTH(d) BETWEEN 8 AND 9 THEN '3º Bimestre'
    WHEN MONTH(d) BETWEEN 10 AND 12 THEN '4º Bimestre'
    ELSE 'Recesso'
END,
FALSE,
CASE WHEN DAYOFWEEK(d) IN (1, 7) THEN FALSE ELSE TRUE END,
CASE
    WHEN EXISTS (
        SELECT 1 FROM sistema_aurora_edutech.tb_ano_letivo al
        WHERE d BETWEEN al.data_inicio AND al.data_fim
    ) THEN TRUE
    ELSE FALSE
END
FROM (
    SELECT DATE_ADD('2020-01-01', INTERVAL seq.n DAY) AS d
    FROM (
        SELECT a.n + b.n * 10 + c.n * 100 + d.n * 1000 AS n
        FROM (SELECT 0 AS n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
            SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
            SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) a
        CROSS JOIN (SELECT 0 AS n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
            SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
            SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9)
        b
        CROSS JOIN (SELECT 0 AS n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
            SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
            SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9)
        c
        CROSS JOIN (SELECT 0 AS n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
            SELECT 3 UNION ALL SELECT 4) d
    ) seq
    WHERE DATE_ADD('2020-01-01', INTERVAL seq.n DAY) <= '2030-12-31'
) datas
ORDER BY d;

```

```

-- =====
-- 6. CARGA DAS DIMENSÕES
-- =====

```

```

-----
-- dim_aluno
-- Campos derivados:
-- - faixa_etaria: agrupa idade_ingresso em faixas
-- - ano_ingresso: extrai do data_ingresso
-- - idade_valida_matricula: 1 se idade compatível com ensino fundamental
--   (5-12 anos — ajustável conforme regra do negócio)
-----

INSERT INTO dim_aluno (
    pk_aluno_origem, ra, nome_completo, sexo, data_nascimento,
    nacionalidade, raca, faz_atividade_extracurricular, escola_anterior,
    idade_ingresso, faixa_etaria, ano_ingresso, idade_valida_matricula
)
SELECT
    a.pk_aluno,
    a.ra,
    CONCAT(p.nome, ' ', p.sobrenome),
    p.sexo,
    p.data_nascimento,
    p.nacionalidade,
    p.raca,
    a.faz_atividade_extracurricular,
    tea.descricao,
    TIMESTAMPDIFF(YEAR, p.data_nascimento, a.data_ingresso),
    CASE
        WHEN TIMESTAMPDIFF(YEAR, p.data_nascimento, a.data_ingresso) <= 6 THEN '0-6
anos'
        WHEN TIMESTAMPDIFF(YEAR, p.data_nascimento, a.data_ingresso) <= 10 THEN '7-10
anos'
        WHEN TIMESTAMPDIFF(YEAR, p.data_nascimento, a.data_ingresso) <= 14 THEN '11-14
anos'
        WHEN TIMESTAMPDIFF(YEAR, p.data_nascimento, a.data_ingresso) <= 17 THEN '15-17
anos'
        ELSE '18+ anos'
    END,
    YEAR(a.data_ingresso),
    CASE
        WHEN TIMESTAMPDIFF(YEAR, p.data_nascimento, a.data_ingresso) BETWEEN 5 AND
12 THEN 1
        ELSE 0
    END
FROM sistema_aurora_edutech.tb_aluno a
JOIN sistema_aurora_edutech.tb_pessoa p
    ON a.fk_pessoa = p.pk_pessoa
LEFT JOIN sistema_aurora_edutech.tb_tipo_escola_anterior tea
    ON a.fk_tipo_escola_anterior = tea.pk_tipo_escola_anterior;

-----
-- dim_funcionario
-- Cargo, tipo_contrato e carga_horaria vêm do contrato mais recente
-----

```

```

INSERT INTO dim_funcionario (
    pk_funcionario_origem, nome_completo, sexo, cargo,
    tipo_contrato, carga_horaria_semanal, ativo
)
SELECT
    f.pk_funcionario,
    CONCAT(p.nome, ' ', p.sobrenome),
    p.sexo,
    ca.nome_cargo,
    tc.descricao,
    ca.carga_horaria_semanal,
    f.ativo
FROM sistema_aurora_edutech.tb_funcionario f
JOIN sistema_aurora_edutech.tb_pessoa p
    ON f.fk_pessoa = p.pk_pessoa
LEFT JOIN (
    SELECT
        con.fk_funcionario,
        con.fk_tipo_contrato,
        con.carga_horaria_semanal,
        cg.nome_cargo,
        ROW_NUMBER() OVER (
            PARTITION BY con.fk_funcionario
            ORDER BY hc.data_inicio DESC
        ) AS rn
    FROM sistema_aurora_edutech.tb_historico_cargo hc
    JOIN sistema_aurora_edutech.tb_contrato con
        ON hc.fk_contrato = con.pk_contrato
    JOIN sistema_aurora_edutech.tb_cargo cg
        ON hc.fk_cargo = cg.pk_cargo
) ca ON ca.fk_funcionario = f.pk_funcionario AND ca.rn = 1
LEFT JOIN sistema_aurora_edutech.tb_tipo_contrato tc
    ON ca.fk_tipo_contrato = tc.pk_tipo_contrato;

```

```

-- -----
-- dim_professor
-- Formação (nível + curso) vem da formação mais recente
-- -----

```

```

INSERT INTO dim_professor (
    pk_professor_origem, nome_completo, sexo, usa_rubrica_avaliativa,
    atende_educacao_especial, tipo_aula, nivel_formacao, curso_principal
)
SELECT
    pr.pk_professor,
    CONCAT(p.nome, ' ', p.sobrenome),
    p.sexo,
    pr.utiliza_rubrica_avaliativa,
    pr.atende_educacao_especial,
    ta.descricao,
    nf.descricao,
    fp.curso

```

```

FROM sistema_aurora_edutech.tb_professor pr
JOIN sistema_aurora_edutech.tb_funcionario f
    ON pr.fk_funcionario = f.pk_funcionario
JOIN sistema_aurora_edutech.tb_pessoa p
    ON f.fk_pessoa = p.pk_pessoa
LEFT JOIN sistema_aurora_edutech.tb_tipo_aula ta
    ON pr.fk_tipo_aula = ta.pk_tipo_aula
LEFT JOIN (
    SELECT
        fp.fk_professor,
        fp.curso,
        fp.fk_nivel_formacao,
        ROW_NUMBER() OVER (
            PARTITION BY fp.fk_professor
            ORDER BY fp.ano_conclusao DESC
        ) AS rn
    FROM sistema_aurora_edutech.tb_formacao_professor fp
) fp ON fp.fk_professor = pr.pk_professor AND fp.rn = 1
LEFT JOIN sistema_aurora_edutech.tb_nivel_formacao nf
    ON fp.fk_nivel_formacao = nf.pk_nivel_formacao;

```

```

-----
-- dim_disciplina
-----

```

```

INSERT INTO dim_disciplina (
    pk_materia_origem, nome_disciplina, area_conhecimento, ativo
)
SELECT
    m.pk_materia,
    m.nome_materia,
    ac.descricao,
    m.ativo
FROM sistema_aurora_edutech.tb_materia m
JOIN sistema_aurora_edutech.tb_area_conhecimento ac
    ON m.fk_area_conhecimento = ac.pk_area_conhecimento;

```

```

-----
-- dim_turma
-----

```

```

INSERT INTO dim_turma (
    pk_turma_origem, nome_turma, ano_escolar, periodo, ano_letivo, capacidade_maxima
)
SELECT
    t.pk_turma,
    t.nome_turma,
    t.ano_escolar,
    t.periodo,
    al.ano,
    t.capacidade_maxima
FROM sistema_aurora_edutech.tb_turma t
JOIN sistema_aurora_edutech.tb_ano_letivo al

```

```
ON t.fk_ano_letivo = al.pk_ano_letivo;
```

```
-- -----  
-- dim_status_pagamento  
-- -----
```

```
INSERT INTO dim_status_pagamento (pk_status_origem, descricao_status)  
SELECT pk_status_pagamento, descricao  
FROM sistema_aurora_edutech.tb_status_pagamento;
```

```
-- -----  
-- dim_atividade  
-- -----
```

```
INSERT INTO dim_atividade (pk_tipo_atividade_origem, descricao_atividade)  
SELECT pk_tipo_atividade, descricao  
FROM sistema_aurora_edutech.tb_tipo_atividade;
```

```
-- -----  
-- dim_folha (lista fixa, baseada no ENUM tb_folha_pagamento.tipo_folha)  
-- -----
```

```
INSERT INTO dim_folha (descricao_folha) VALUES  
('folha_mensal'),  
('adiantamento'),  
('decimo_terceiro'),  
('rescisao'),  
('ferias');
```

```
-- -----  
-- dim_risco  
-- Cruza tb_tipo_risco_evasao × tb_nivel_risco.  
-- ordem_criticidade é derivada do nivel_risco:  
-- baixo = 1, medio = 2, alto = 3, critico = 4  
-- -----
```

```
INSERT INTO dim_risco (  
    pk_tipo_risco_orig, pk_nivel_risco_orig,  
    tipo_risco, nivel_risco, ordem_criticidade  
)  
SELECT  
    tr.pk_tipo_risco_evasao,  
    nr.pk_nivel_risco,  
    tr.descricao,  
    nr.descricao,  
    CASE LOWER(nr.descricao)  
        WHEN 'baixo' THEN 1  
        WHEN 'medio' THEN 2  
        WHEN 'médio' THEN 2 -- aceita variação com acento  
        WHEN 'alto' THEN 3  
        WHEN 'critico' THEN 4  
        ELSE 2 -- fallback: médio  
    END  
FROM sistema_aurora_edutech.tb_tipo_risco_evasao tr  
CROSS JOIN sistema_aurora_edutech.tb_nivel_risco nr;
```

```

-----
-- dim_situacao
-- Origem: tb_situacao_boletim
-----

INSERT INTO dim_situacao (pk_situacao_orig, descricao)
SELECT pk_situacao_boletim, descricao
FROM sistema_aurora_edutech.tb_situacao_boletim;

-- =====
-- 7. CARGA FATO_FOLHA_RH
-- =====
-- salario_base: vem de tb_historico_salario (salário vigente na data)
-- salario_anterior: LAG sobre o histórico de salário do contrato
-- percentual_aumento: ((salario_base - salario_anterior) / salario_anterior) * 100
-- =====

INSERT INTO fato_folha_rh (
    fk_funcionario, fk_tempo, fk_tipo_folha, salario_base,
    valor_bruto, valor_liquido, total_creditos, total_descontos,
    salario_anterior, percentual_aumento, motivo_aumento
)
SELECT
    df.sk_funcionario,
    CAST(DATE_FORMAT(fp.data_emissao, '%Y%m%d') AS UNSIGNED),
    dfo.sk_tipo_folha,
    hs.salario_base,

    SUM(CASE WHEN tv.natureza_verba = 'credito' THEN vf.valor ELSE 0 END)      AS
valor_bruto,
    SUM(CASE WHEN tv.natureza_verba = 'credito' THEN vf.valor ELSE -vf.valor END) AS
valor_liquido,
    SUM(CASE WHEN tv.natureza_verba = 'credito' THEN vf.valor ELSE 0 END)      AS
total_creditos,
    SUM(CASE WHEN tv.natureza_verba = 'debito' THEN vf.valor ELSE 0 END)      AS
total_descontos,

    hs.salario_anterior,
    CASE
        WHEN hs.salario_anterior IS NULL OR hs.salario_anterior = 0 THEN NULL
        ELSE ROUND(((hs.salario_base - hs.salario_anterior) / hs.salario_anterior) * 100, 2)
    END AS percentual_aumento,
    hs.motivo_reajuste

FROM sistema_aurora_edutech.tb_folha_pagamento fp

JOIN sistema_aurora_edutech.tb_contrato con
    ON fp.fk_contrato = con.pk_contrato

JOIN dim_funcionario df
    ON con.fk_funcionario = df.pk_funcionario_origem

```

```

JOIN dim_folha dfo
  ON dfo.descricao_folha = fp.tipo_folha

JOIN sistema_aurora_edutech.tb_verba_folha vf
  ON vf.fk_folha_pagamento = fp.pk_folha_pagamento

JOIN sistema_aurora_edutech.tb_tipo_verba tv
  ON vf.fk_tipo_verba = tv.pk_tipo_verba

-- salário vigente na data de emissão + salário anterior (via LAG)
LEFT JOIN (
  SELECT
    fk_contrato,
    salario AS salario_base,
    data_inicio,
    data_fim,
    motivo_reajuste,
    LAG(salario) OVER (PARTITION BY fk_contrato ORDER BY data_inicio) AS
    salario_anterior
  FROM sistema_aurora_edutech.tb_historico_salario
) hs
  ON hs.fk_contrato = con.pk_contrato
 AND fp.data_emissao >= hs.data_inicio
 AND (hs.data_fim IS NULL OR fp.data_emissao <= hs.data_fim)

GROUP BY
  fp.pk_folha_pagamento, df.sk_funcionario, fp.data_emissao,
  dfo.sk_tipo_folha, hs.salario_base, hs.salario_anterior, hs.motivo_reajuste;

-- =====
-- 8. CARGA FATO_DESEMPENHO
-- =====
-- Inclui:
-- - fk_professor, fk_turma (via grade curricular e matrícula)
-- - fk_tipo_atividade
-- - percentual_nota, nota_maxima, peso_atividade
-- - qtd_entregue_no_prazo (1/0)
-- =====

INSERT INTO fato_desempenho (
  fk_aluno, fk_professor, fk_disciplina, fk_turma, fk_tempo,
  fk_tipo_atividade, valor_nota, nota_maxima, percentual_nota,
  peso_atividade, qtd_entregue_no_prazo
)
SELECT
  da.sk_aluno,
  dp.sk_professor,
  dd.sk_disciplina,
  dt.sk_turma,
  CAST(DATE_FORMAT(ap.data_prevista, '%Y%m%d') AS UNSIGNED),

```

```

    datv.sk_tipo_atividade,
    na.valor_nota,
    ap.valor_maximo,
    CASE
        WHEN ap.valor_maximo > 0 AND na.valor_nota IS NOT NULL
            THEN ROUND((na.valor_nota / ap.valor_maximo) * 100, 2)
        ELSE NULL
    END,
    ap.peso,
    CASE WHEN na.entregue_no_prazo THEN 1 ELSE 0 END

FROM sistema_aurora_edutech.tb_nota_atividade na

JOIN sistema_aurora_edutech.tb_matricula m
    ON na.fk_matricula = m.pk_matricula
JOIN dim_aluno da
    ON m.fk_aluno = da.pk_aluno_origem
JOIN dim_turma dt
    ON m.fk_turma = dt.pk_turma_origem

JOIN sistema_aurora_edutech.tb_atividade_professor ap
    ON na.fk_atividade_professor = ap.pk_atividade_professor
JOIN dim_atividade datv
    ON ap.fk_tipo_atividade = datv.pk_tipo_atividade_origem

JOIN sistema_aurora_edutech.tb_grade_curricular gc
    ON ap.fk_grade = gc.pk_grade
JOIN dim_disciplina dd
    ON gc.fk_materia = dd.pk_materia_origem
JOIN dim_professor dp
    ON gc.fk_professor = dp.pk_professor_origem;

-- =====
-- 9. CARGA FATO_FREQUENCIA
-- =====
-- Agrega presença/falta/justificada por aluno × turma × dia.
-- presença    = 1 quando tb_frequencia.presente = TRUE
-- falta       = 1 quando presente = FALSE e justificada = FALSE
-- justificada  = 1 quando presente = FALSE e justificada = TRUE
-- =====

INSERT INTO fato_frequencia (
    fk_aluno, fk_turma, fk_tempo,
    qtd_presenca, qtd_falta, qtd_justificada
)
SELECT
    da.sk_aluno,
    dt.sk_turma,
    CAST(DATE_FORMAT(f.data_aula, '%Y%m%d') AS UNSIGNED),
    SUM(CASE WHEN f.presente = TRUE THEN 1 ELSE 0 END),
    SUM(CASE WHEN f.presente = FALSE AND f.justificada = FALSE THEN 1 ELSE 0 END),

```

```

SUM(CASE WHEN f.presente = FALSE AND f.justificada = TRUE THEN 1 ELSE 0 END)
FROM sistema_aurora_edutech.tb_frequencia f
JOIN sistema_aurora_edutech.tb_matricula m
  ON f.fk_matricula = m.pk_matricula
JOIN dim_aluno da
  ON m.fk_aluno = da.pk_aluno_origem
JOIN dim_turma dt
  ON m.fk_turma = dt.pk_turma_origem
GROUP BY
  da.sk_aluno, dt.sk_turma,
  CAST(DATE_FORMAT(f.data_aula, '%Y%m%d') AS UNSIGNED);

```

```

-- =====
-- 10. CARGA FATO_BOLETIM
-- =====
-- O sk_tempo é resolvido pelo bimestre + ano_letivo da matrícula:
-- usamos o primeiro dia do bimestre correspondente em dim_tempo.
-- Flags derivados da tb_situacao_boletim via JOIN com dim_situacao.
-- =====

```

```

INSERT INTO fato_boletim (
  fk_aluno, fk_turma, fk_tempo, fk_situacao,
  esta_aprovado, esta_reprovado, em_recuperacao
)
SELECT
  da.sk_aluno,
  dt.sk_turma,
  -- sk_tempo = primeiro dia do bimestre no ano letivo da matrícula
  (
    SELECT MIN(tmp.sk_tempo)
    FROM dim_tempo tmp
    WHERE tmp.bimestre = b.bimestre
    AND tmp.ano = al.ano
  ),
  ds.sk_situacao,
  CASE WHEN LOWER(sb.descricao) LIKE '%aprovad%' THEN 1 ELSE 0 END,
  CASE WHEN LOWER(sb.descricao) LIKE '%reprovad%' THEN 1 ELSE 0 END,
  CASE WHEN LOWER(sb.descricao) LIKE '%recupera%' THEN 1 ELSE 0 END
FROM sistema_aurora_edutech.tb_boletim b
JOIN sistema_aurora_edutech.tb_matricula m
  ON b.fk_matricula = m.pk_matricula
JOIN sistema_aurora_edutech.tb_ano_letivo al
  ON m.fk_ano_letivo = al.pk_ano_letivo
JOIN sistema_aurora_edutech.tb_situacao_boletim sb
  ON b.fk_situacao_boletim = sb.pk_situacao_boletim
JOIN dim_aluno da
  ON m.fk_aluno = da.pk_aluno_origem
JOIN dim_turma dt
  ON m.fk_turma = dt.pk_turma_origem
JOIN dim_situacao ds
  ON sb.pk_situacao_boletim = ds.pk_situacao_orig;

```

```
-- =====
-- 11. CARGA FATO_ATIVIDADE_PROFESSOR
-- =====
-- qtd_realizada = 1 por registro (cada linha é uma atividade planejada;
-- a contagem agrupada fica para as queries analíticas).
-- Professor e disciplina vêm via tb_grade_curricular.
-- =====
```

```
INSERT INTO fato_atividade_professor (
    fk_professor, fk_disciplina, fk_turma, fk_tempo,
    fk_tipo_atividade, qtd_realizada
)
SELECT
    dp.sk_professor,
    dd.sk_disciplina,
    dt.sk_turma,
    CAST(DATE_FORMAT(ap.data_prevista, '%Y%m%d') AS UNSIGNED),
    datv.sk_tipo_atividade,
    1 -- cada registro = 1 atividade realizada/planejada
FROM sistema_aurora_edutech.tb_atividade_professor ap
JOIN sistema_aurora_edutech.tb_grade_curricular gc
    ON ap.fk_grade = gc.pk_grade
JOIN dim_professor dp
    ON gc.fk_professor = dp.pk_professor_origem
JOIN dim_disciplina dd
    ON gc.fk_materia = dd.pk_materia_origem
JOIN sistema_aurora_edutech.tb_turma_grade tg
    ON gc.pk_grade = tg.fk_grade
JOIN dim_turma dt
    ON tg.fk_turma = dt.pk_turma_origem
JOIN dim_atividade datv
    ON ap.fk_tipo_atividade = datv.pk_tipo_atividade_origem;
```

```
-- =====
-- 12. CARGA FATO_PAGAMENTO
-- =====
-- valor_pago: vem de tb_pagamento (soma dos pagamentos vinculados).
-- qtd_dias_atrasado: diferença entre data_pagamento e data_vencimento,
-- quando positiva. Zero se pago em dia ou não pago.
-- fk_tempo aponta para o mês de vencimento da cobrança.
-- =====
```

```
INSERT INTO fato_pagamento (
    fk_aluno, fk_tempo, fk_status_pagamento,
    valor_mensalidade, valor_pago,
    qtd_foi_pago, qtd_foi_atrasado, qtd_dias_atrasado
)
SELECT
    da.sk_aluno,
    CAST(DATE_FORMAT(c.data_vencimento, '%Y%m%d') AS UNSIGNED),
```

```

dsp.sk_status_pagamento,
c.valor,
COALESCE(pg.total_pago, 0),
CASE WHEN pg.total_pago IS NOT NULL AND pg.total_pago > 0 THEN 1 ELSE 0 END,
CASE WHEN pg.data_pagamento > c.data_vencimento THEN 1 ELSE 0 END,
CASE
    WHEN pg.data_pagamento > c.data_vencimento
    THEN DATEDIFF(pg.data_pagamento, c.data_vencimento)
    ELSE 0
END
FROM sistema_aurora_edutech.tb_cobranca c
JOIN sistema_aurora_edutech.tb_matricula m
    ON c.fk_matricula = m.pk_matricula
JOIN dim_aluno da
    ON m.fk_aluno = da.pk_aluno_origem
JOIN sistema_aurora_edutech.tb_status_pagamento sp
    ON c.fk_status_pagamento = sp.pk_status_pagamento
JOIN dim_status_pagamento dsp
    ON sp.pk_status_pagamento = dsp.pk_status_origem
LEFT JOIN (
    SELECT
        fk_cobranca,
        SUM(valor_pago) AS total_pago,
        MAX(data_pagamento) AS data_pagamento
    FROM sistema_aurora_edutech.tb_pagamento
    GROUP BY fk_cobranca
) pg ON pg.fk_cobranca = c.pk_cobranca;

```

```

-- =====
-- 13. CARGA FATO_EVASAO
-- =====
-- dias_ate_resolucao: DATEDIFF(data_resolucao, data_alerta). NULL se
-- o alerta ainda não foi resolvido.
-- evadiu: 1 quando o alerta não foi resolvido E a matrícula do aluno
-- está inativa/cancelada — derivado cruzando com tb_matricula.
-- fk_risco: resolve o par (fk_tipo_risco_evasao, fk_nivel_risco)
-- na dim_risco.
-- =====

```

```

INSERT INTO fato_evasao (
    fk_aluno, fk_turma, fk_tempo, fk_risco,
    dias_ate_resolucao, foi_resolvido, evadiu
)
SELECT
    da.sk_aluno,
    dt.sk_turma,
    CAST(DATE_FORMAT(ae.data_alerta, '%Y%m%d') AS UNSIGNED),
    dr.sk_risco,
    CASE
        WHEN ae.data_resolucao IS NOT NULL
        THEN DATEDIFF(ae.data_resolucao, ae.data_alerta)
    END

```

```

        ELSE NULL
    END,
    CASE WHEN ae.data_resolucao IS NOT NULL THEN 1 ELSE 0 END,
    -- evadiu = alerta não resolvido + matrícula cancelada/inativa
    CASE
        WHEN ae.data_resolucao IS NULL
            AND EXISTS (
                SELECT 1
                FROM sistema_aurora_edutech.tb_matricula mx
                JOIN sistema_aurora_edutech.tb_status_matricula sm
                ON mx.fk_status_matricula = sm.pk_status_matricula
                WHERE mx.pk_matricula = ae.fk_matricula
                AND LOWER(sm.descricao) IN ('cancelada', 'inativa', 'evadido')
            )
        THEN 1
        ELSE 0
    END
END
FROM sistema_aurora_edutech.tb_alerta_evasao ae
JOIN sistema_aurora_edutech.tb_matricula m
    ON ae.fk_matricula = m.pk_matricula
JOIN dim_aluno da
    ON m.fk_aluno = da.pk_aluno_origem
JOIN dim_turma dt
    ON m.fk_turma = dt.pk_turma_origem
JOIN dim_risco dr
    ON ae.fk_tipo_risco_evasao = dr.pk_tipo_risco_orig
    AND ae.fk_nivel_risco = dr.pk_nivel_risco_orig;

-- =====
-- 14. CARGA FATO_CONTRATO_RH
-- =====
-- Granularidade: 1 linha por contrato (foto atual, não histórico).
-- fk_tempo = sk_tempo do mês de início do contrato (data_inicio).
-- fk_tempo_emissao = mesmo que fk_tempo (data de início).
-- fk_tempo_vencimento = sk_tempo da data_fim; NULL se indeterminado.
-- contrato_ativo = 1 quando data_fim IS NULL ou data_fim >= CURDATE().
-- =====

INSERT INTO fato_contrato_rh (
    fk_funcionario, fk_tempo,
    fk_tempo_emissao, fk_tempo_vencimento,
    salario_base, carga_horaria, contrato_ativo
)
SELECT
    df.sk_funcionario,
    CAST(DATE_FORMAT(con.data_inicio, '%Y%m%d') AS UNSIGNED),
    CAST(DATE_FORMAT(con.data_inicio, '%Y%m%d') AS UNSIGNED),
    CASE
        WHEN con.data_fim IS NOT NULL
            THEN CAST(DATE_FORMAT(con.data_fim, '%Y%m%d') AS UNSIGNED)
        ELSE NULL
    
```

```

END,
hs.salario_atual,
con.carga_horaria_semanal,
CASE
    WHEN con.data_fim IS NULL OR con.data_fim >= CURDATE() THEN 1
    ELSE 0
END
FROM sistema_aurora_edutech.tb_contrato con
JOIN dim_funcionario df
    ON con.fk_funcionario = df.pk_funcionario_origem
-- salário atual = registro mais recente em tb_historico_salario
LEFT JOIN (
    SELECT
        fk_contrato,
        salario AS salario_atual,
        ROW_NUMBER() OVER (
            PARTITION BY fk_contrato
            ORDER BY data_inicio DESC
        ) AS rn
    FROM sistema_aurora_edutech.tb_historico_salario
) hs ON hs.fk_contrato = con.pk_contrato AND hs.rn = 1;

```

```

-- =====
-- 15. CONSULTAS DE VERIFICAÇÃO
-- =====

```

```

-- dim_tempo: deve dar 4018 dias; bimestres 1-4 + NULL (janeiro)
SELECT COUNT(*) AS total_dias FROM dim_tempo;
SELECT bimestre, nome_bimestre, COUNT(*) AS qtd
FROM dim_tempo GROUP BY bimestre, nome_bimestre ORDER BY bimestre;

```

```

-- Dimensões — amostras
SELECT * FROM dim_aluno          LIMIT 5;
SELECT * FROM dim_funcionario    LIMIT 5;
SELECT * FROM dim_professor      LIMIT 5;
SELECT * FROM dim_disciplina;
SELECT * FROM dim_turma;
SELECT * FROM dim_status_pagamento;
SELECT * FROM dim_atividade;
SELECT * FROM dim_folha;
SELECT * FROM dim_risco  ORDER BY ordem_criticidade, tipo_risco;
SELECT * FROM dim_situacao;

```

```

-- Contagens das dimensões novas
SELECT COUNT(*) AS qtd_riscos  FROM dim_risco;
SELECT COUNT(*) AS qtd_situacoes FROM dim_situacao;

```

```

-- Contagens das fatos
SELECT COUNT(*) AS qtd_folha          FROM fato_folha_rh;
SELECT COUNT(*) AS qtd_desempenho     FROM fato_desempenho;
SELECT COUNT(*) AS qtd_frequencia     FROM fato_frequencia;

```

```

SELECT COUNT(*) AS qtd_boletim          FROM fato_boletim;
SELECT COUNT(*) AS qtd_atividade_professor FROM fato_atividade_professor;
SELECT COUNT(*) AS qtd_pagamento        FROM fato_pagamento;
SELECT COUNT(*) AS qtd_evasao           FROM fato_evasao;
SELECT COUNT(*) AS qtd_contrato_rh       FROM fato_contrato_rh;

```

-- Sanidade: NULLs inesperados em fato_desempenho

```

SELECT
  SUM(fk_aluno IS NULL) AS nulos_aluno,
  SUM(fk_professor IS NULL) AS nulos_professor,
  SUM(fk_disciplina IS NULL) AS nulos_disciplina,
  SUM(fk_turma IS NULL) AS nulos_turma,
  SUM(fk_tipo_atividade IS NULL) AS nulos_tipo_atividade
FROM fato_desempenho;

```

-- Sanidade: NULLs inesperados em fato_frequencia

```

SELECT
  SUM(fk_aluno IS NULL) AS nulos_aluno,
  SUM(fk_turma IS NULL) AS nulos_turma,
  SUM(fk_tempo IS NULL) AS nulos_tempo
FROM fato_frequencia;

```

-- Sanidade: NULLs inesperados em fato_boletim

```

SELECT
  SUM(fk_aluno IS NULL) AS nulos_aluno,
  SUM(fk_turma IS NULL) AS nulos_turma,
  SUM(fk_tempo IS NULL) AS nulos_tempo,
  SUM(fk_situacao IS NULL) AS nulos_situacao
FROM fato_boletim;

```

-- Sanidade: NULLs inesperados em fato_evasao

```

SELECT
  SUM(fk_aluno IS NULL) AS nulos_aluno,
  SUM(fk_turma IS NULL) AS nulos_turma,
  SUM(fk_tempo IS NULL) AS nulos_tempo,
  SUM(fk_risco IS NULL) AS nulos_risco
FROM fato_evasao;

```

-- Análise: média de % nota por professor

```

SELECT
  dp.nome_completo,
  ROUND(AVG(fd.percentual_nota), 2) AS media_pct_alunos,
  COUNT(*) AS qtd_notas
FROM fato_desempenho fd
JOIN dim_professor dp ON dp.sk_professor = fd.fk_professor
GROUP BY dp.nome_completo
ORDER BY media_pct_alunos DESC;

```

-- Análise: taxa de evasão por nível de risco

```

SELECT
  dr.nivel_risco,

```

```

dr.ordem_criticidade,
COUNT(*)                AS total_alertas,
SUM(fe.foi_resolvido)     AS resolvidos,
SUM(fe.evadiu)            AS evadiram,
ROUND(SUM(fe.evadiu) / COUNT(*) * 100, 2) AS pct_evasao
FROM fato_evasao fe
JOIN dim_risco dr ON dr.sk_risco = fe.fk_risco
GROUP BY dr.nivel_risco, dr.ordem_criticidade
ORDER BY dr.ordem_criticidade;

```

-- Análise: inadimplência por mês

```

SELECT
  t.ano,
  t.mes,
  t.nome_mes,
  COUNT(*)                AS total_cobrancas,
  SUM(fp.qtd_foi_pago)     AS pagas,
  SUM(1 - fp.qtd_foi_pago) AS nao_pagas,
  ROUND(AVG(fp.qtd_dias_atrasado), 1) AS media_dias_atraso
FROM fato_pagamento fp
JOIN dim_tempo t ON t.sk_tempo = fp.fk_tempo
GROUP BY t.ano, t.mes, t.nome_mes
ORDER BY t.ano, t.mes;

```

-- Análise: frequência média por turma

```

SELECT
  dt.nome_turma,
  dt.ano_letivo,
  ROUND(AVG(ff.qtd_presenca), 1) AS media_presencas,
  ROUND(AVG(ff.qtd_falta), 1) AS media_faltas,
  ROUND(AVG(ff.qtd_justificada), 1) AS media_justificadas
FROM fato_frequencia ff
JOIN dim_turma dt ON dt.sk_turma = ff.fk_turma
GROUP BY dt.nome_turma, dt.ano_letivo
ORDER BY dt.ano_letivo, dt.nome_turma;

```

5.2 Dicionário de Dados OLAP

5.2.1 Dimensões Conformadas

As dimensões abaixo são compartilhadas entre múltiplos módulos. O mesmo sk_aluno em fato_desempenho e em fato_pagamento refere-se ao mesmo aluno, garantindo análises cruzadas consistentes.

dim_tempo

Dimensão de tempo compartilhada por todas as tabelas fato. Cada linha representa um dia único. Permite agrupamentos por bimestre, mês, semana e tipo de dia letivo.

Campo	Tipo	Restrição	Descrição
sk_tempo	INT	PK	Surrogate Key gerado como YYYYMMDD (ex.: 20260509).
data	DATE	NOT NULL UNIQUE	Data completa no formato YYYY-MM-DD.
dia	TINYINT	NOT NULL	Dia do mês (1-31).
mes	TINYINT	NOT NULL	Mês do ano (1-12).
nome_mes	VARCHAR	NOT NULL	Nome por extenso do mês (ex.: Março).
ano	SMALLINT	NOT NULL	Ano de quatro dígitos (ex.: 2026).
bimestre	TINYINT	NOT NULL	Bimestre escolar (1 a 4), conforme calendário letivo.
eh_feriado	BOOLEAN	NOT NULL	TRUE se o dia é feriado nacional, estadual ou municipal.
eh_dia_util	BOOLEAN	NOT NULL	TRUE se o dia é útil (segunda a sexta, não feriado).
eh_periodo_letivo	BOOLEAN	NOT NULL	TRUE se o dia está dentro do período letivo.
nome_bimestre	VARCHAR	NOT NULL	Nome do bimestre para exibição (ex.: 1º Bimestre 2026).
semana_do_ano	TINYINT	NOT NULL	Número da semana no ano (ISO week, 1-53).

dim_aluno

Perfil completo do aluno no momento da análise. Conformada entre Acadêmico, Financeiro e Gestão. Snapshot: atributos refletem o estado no período analisado.

Campo	Tipo	Restrição	Descrição
sk_aluno	INT	PK	Surrogate Key do aluno no DW.
ra	VARCHAR(15)	NOT NULL	Registro do Aluno — chave natural vinda do OLTP.
nome_completo	VARCHAR	NOT NULL	Nome e sobrenome concatenados.
sexo	VARCHAR	NOT NULL	Sexo declarado (Masculino / Feminino).
data_nascimento	DATE	NOT NULL	Data de nascimento para cálculo de faixa etária.
nacionalidade	VARCHAR	NOT NULL	Brasileiro ou Estrangeiro.
raca	VARCHAR	NULL	Autodeclaração de raça/cor para indicadores de equidade.
faz_atividade_extracurricular	BOOLEAN	NOT NULL	Indica participação em atividades extracurriculares.
escola_anterior	VARCHAR	NULL	Tipo de escola de origem (Pública / Particular).
idade_ingresso	TINYINT	NOT NULL	Idade em anos completos no momento do ingresso.
faixa_etaria	VARCHAR	NOT NULL	Faixa etária calculada (ex.: 6-8 anos).
ano_ingresso	YEAR	NOT NULL	Ano letivo de ingresso na escola.
idade_valida_matricula	TINYINT	NOT NULL	Flag 0/1: 1 = idade dentro do intervalo permitido na matrícula.

dim_turma

Descreve a turma escolar. Conformada entre Acadêmico e Financeiro.

Campo	Tipo	Restrição	Descrição
sk_turma	INT	PK	Surrogate Key da turma no DW.
nome_turma	VARCHAR	NOT NULL	Código da turma (ex.: 3A, 5B).
ano_escolar	TINYINT	NOT NULL	Ano de escolaridade (1 a 5).
periodo	VARCHAR	NOT NULL	Turno: Manhã ou Tarde.
ano_letivo	YEAR	NOT NULL	Ano letivo de vigência da turma.
capacidade_maxima	TINYINT	NULL	Número máximo de alunos. NULL = sem limite configurado.

dim_professor

Atributos analíticos do professor. Conformada entre Acadêmico e RH.

Campo	Tipo	Restrição	Descrição
sk_professor	INT	PK	Surrogate Key do professor no DW.
nome_completo	VARCHAR	NOT NULL	Nome completo para exibição em relatórios.
sexo	VARCHAR	NOT NULL	Sexo do professor para indicadores de equidade.
usa_rubrica_avaliativa	BOOLEAN	NOT NULL	TRUE se o professor adota rubricas de avaliação.
atende_educacao_especial	BOOLEAN	NOT NULL	TRUE se habilitado para atendimento educacional especializado.
tipo_aula	VARCHAR	NULL	Modalidade de aula (ex.: Presencial, Híbrido).
nivel_formacao	VARCHAR	NOT NULL	Nível de formação mais alto concluído.
curso_principal	VARCHAR	NOT NULL	Curso de maior nível de formação.

dim_funcionario

Representa todos os funcionários da escola. Usada exclusivamente no módulo RH.

Campo	Tipo	Restrição	Descrição
sk_funcionario	INT	PK	Surrogate Key do funcionário no DW.
cargo	VARCHAR	NOT NULL	Cargo atual do funcionário.
tipo_contrato	VARCHAR	NOT NULL	Tipo de contrato (ex.: CLT, PJ, Temporário).
nome_completo	VARCHAR	NOT NULL	Nome completo para relatórios de RH.
sexo	VARCHAR	NOT NULL	Sexo do funcionário para análise de equidade de remuneração.
carga_horaria_semanal	TINYINT	NOT NULL	Carga horária semanal contratada (1-60 horas).
ativo	BOOLEAN	NOT NULL	TRUE se o funcionário possui contrato ativo no período.

dim_disciplina

Catálogo de disciplinas da grade curricular. Conformada entre Acadêmico e Gestão.

Campo	Tipo	Restrição	Descrição
sk_disciplina	INT	PK	Surrogate Key da disciplina no DW.
nome_disciplina	VARCHAR	NOT NULL	Nome da disciplina (ex.: Matemática, Língua Portuguesa).
area_conhecimento	VARCHAR	NOT NULL	Área do conhecimento (ex.: Ciências Exatas, Humanas).
ativo	BOOLEAN	NOT NULL	Indica se a disciplina está ativa na grade curricular.

dim_atividade

Tipos de atividade avaliativa. Usada em fato_desempenho e fato_atividade_professor.

Campo	Tipo	Restrição	Descrição
sk_tipo_atividade	INT	PK	Surrogate Key do tipo de atividade.
descricao_atividade	VARCHAR	NOT NULL	Descrição (ex.: Prova Bimestral, Trabalho em Grupo).

dim_situacao

Resultado acadêmico ao final do bimestre. Usada em fato_boletim.

Campo	Tipo	Restrição	Descrição
sk_situacao	INT	PK	Surrogate Key da situação.
descricao	VARCHAR	NOT NULL	Descrição (ex.: Aprovado, Em Recuperação, Reprovado).

dim_status_pagamento

Status possíveis de uma cobrança de mensalidade. Usada em fato_pagamento.

Campo	Tipo	Restrição	Descrição
sk_status_pagamento	INT	PK	Surrogate Key do status.
descricao_status	VARCHAR	NOT NULL	Descrição (ex.: Pago, Pendente, Atrasado, Estornado).

dim_folha

Tipos de folha de pagamento. Usada em fato_folha_rh.

Campo	Tipo	Restrição	Descrição
sk_tipo_folha	INT	PK	Surrogate Key do tipo de folha.
descricao_folha	VARCHAR	NOT NULL	Descrição (ex.: Folha Mensal, Décimo Terceiro, Férias, Rescisão).

dim_risco

Classificação dos alertas de evasão por tipo e nível de criticidade. Usada em fato_evasao.

Campo	Tipo	Restrição	Descrição
sk_risco	INT	PK	Surrogate Key do risco.
tipo_risco	VARCHAR	NOT NULL	Categoria do risco (ex.: Faltas Excessivas, Inadimplência).

nivel_risco	VARCHAR	NOT NULL	Nível (ex.: Baixo, Médio, Alto, Crítico).
ordem_criticidade	TINYINT	NOT NULL	Ordem numérica para ranking em dashboards.

dim_tipo_ocorrencia

Categorias de ocorrência pedagógica e disciplinar. Usada em fato_ocorrencia_rh.

Campo	Tipo	Restrição	Descrição
sk_tipo_ocorrencia	INT	PK	Surrogate Key do tipo de ocorrência.
descricao_ocorrencia	VARCHAR	NOT NULL	Descrição da categoria (ex.: Indisciplina, Bullying, Conflito).

5.2.2 Tabelas Fato por Módulo

Cada tabela fato armazena eventos do negócio. A granularidade de cada tabela indica o que uma linha representa.

Módulo Acadêmico

fato_desempenho

Desempenho de cada aluno em cada atividade avaliativa. Granularidade: um registro por aluno por atividade por bimestre.

Campo	Tipo	Restrição	Descrição
id_desempenho	INT	PK	Surrogate Key da linha fato.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_professor	INT	FK	Referência a dim_professor (responsável pela atividade).
fk_disciplina	INT	FK	Referência a dim_disciplina.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (data de realização).
fk_tipo_atividade	INT	FK	Referência a dim_atividade.
valor_nota	DECIMAL	NULL	Nota obtida pelo aluno. NULL = não avaliado.
percentual_nota	DECIMAL	CALC	Percentual: $(\text{valor_nota} / \text{nota_maxima}) * 100$.
nota_maxima	DECIMAL	NOT NULL	Valor máximo da atividade (copiado do OLTP para independência histórica).
peso_atividade	DECIMAL	NOT NULL	Peso da atividade no conceito bimestral.
qtd_entregue_no_prazo	TINYINT	NOT NULL	Flag 0/1: 1 = entregue no prazo.

fato_frequencia

Frequência do aluno por dia letivo e turma. Granularidade: um registro por aluno por dia do calendário letivo.

Campo	Tipo	Restrição	Descrição
id_frequencia	INT	PK	Surrogate Key da linha fato.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (dia letivo).
qtd_presenca	TINYINT	NOT NULL	Quantidade de aulas com presença no dia.
qtd_falta	TINYINT	NOT NULL	Quantidade de faltas sem justificativa.
qtd_justificada	TINYINT	NOT NULL	Quantidade de faltas com justificativa registrada.

fato_boletim

Situação acadêmica final por bimestre. Granularidade: um registro por aluno por bimestre por turma.

Campo	Tipo	Restrição	Descrição
sk_boletim	INT	PK	Surrogate Key do boletim.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_tempo	INT	FK	Referência a dim_tempo (fechamento do bimestre).
fk_turma	INT	FK	Referência a dim_turma.
fk_situacao	INT	FK	Referência a dim_situacao.
esta_aprovado	TINYINT	NOT NULL	Flag 0/1: 1 = aprovado.
esta_reprovado	TINYINT	NOT NULL	Flag 0/1: 1 = reprovado.
em_recuperacao	TINYINT	NOT NULL	Flag 0/1: 1 = em recuperação.

fato_atividade_professor

Produção de atividades avaliativas por professor por disciplina. Granularidade: um registro por professor por disciplina por bimestre.

Campo	Tipo	Restrição	Descrição
sk_atividade	INT	PK	Surrogate Key do registro.
fk_professor	INT	FK	Referência a dim_professor.
fk_disciplina	INT	FK	Referência a dim_disciplina.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (data prevista).
fk_tipo_atividade	INT	FK	Referência a dim_atividade.
qtd_realizada	TINYINT	NOT NULL	Quantidade de atividades deste tipo realizadas no período.

fato_evasao

Alertas de risco de evasão gerados pelo sistema. Granularidade: um registro por alerta emitido.

Campo	Tipo	Restrição	Descrição
sk_evasao	INT	PK	Surrogate Key do alerta.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (data de emissão).
fk_risco	INT	FK	Referência a dim_risco (tipo e nível de criticidade).
dias_ate_resolucao	DECIMAL	NULL	Dias entre emissão e resolução. NULL = ainda não resolvido.
foi_resolvido	TINYINT	NOT NULL	Flag 0/1: 1 = alerta resolvido.
evadiu	TINYINT	NOT NULL	Flag 0/1: 1 = aluno efetivamente evadiu.

Módulo Financeiro

fato_pagamento

Cobranças de mensalidade por aluno. Granularidade: um registro por parcela por mês de referência.

Campo	Tipo	Restrição	Descrição
id_pagamento	INT	PK	Surrogate Key da cobrança.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_tempo	INT	FK	Referência a dim_tempo (data de vencimento).
fk_status_pagamento	INT	FK	Referência a dim_status_pagamento.
valor_mensalidade	DECIMAL	NOT NULL	Valor de face da mensalidade conforme contrato.
valor_pago	DECIMAL	NULL	Valor efetivamente recebido. NULL = ainda não pago.
qtd_foi_pago	TINYINT	NOT NULL	Flag 0/1: 1 = mensalidade paga.
qtd_foi_atrasado	TINYINT	NOT NULL	Flag 0/1: 1 = pagamento após o vencimento.
qtd_dias_atrasado	INT	NOT NULL	Número de dias de atraso (0 se em dia ou não pago).

Módulo RH

fato_folha_rh

Folhas de pagamento processadas por funcionário. Granularidade: um registro por funcionário por mês por tipo de folha.

Campo	Tipo	Restrição	Descrição
id_folha_rh	INT	PK	Surrogate Key da folha.
fk_funcionario	INT	FK	Referência a dim_funcionario.
fk_tempo	INT	FK	Referência a dim_tempo (data de emissão).
fk_tipo_folha	INT	FK	Referência a dim_folha.
valor_bruto	DECIMAL	NOT NULL	Valor bruto total antes dos descontos.
valor_liquido	DECIMAL	NOT NULL	Valor líquido a receber após descontos.
salario_base	DECIMAL	NOT NULL	Salário base do contrato vigente no período.
total_creditos	DECIMAL	NOT NULL	Soma de todos os proventos.
total_descontos	DECIMAL	NOT NULL	Soma de todos os descontos.
salario_anterior	DECIMAL	NULL	Salário do período anterior para comparativo.
percentual_aumento	DECIMAL	CALC	Variação salarial calculada: $((\text{salario_base} - \text{salario_anterior}) / \text{salario_anterior}) * 100$.
motivo_aumento	VARCHAR	NULL	Justificativa do reajuste. NULL = sem reajuste.

fato_contrato_rh

Contratos de trabalho vigentes por funcionário. Granularidade: um registro por contrato ativo por período.

Campo	Tipo	Restrição	Descrição
sk_contrato_rh	INT	PK	Surrogate Key do contrato.
fk_funcionario	INT	FK	Referência a dim_funcionario.
fk_tempo	INT	FK	Referência a dim_tempo (data de início).
salario_base	DECIMAL	NOT NULL	Salário base no momento do snapshot.
carga_horaria	TINYINT	NOT NULL	Carga horária semanal contratada.
contrato_ativo	TINYINT	NOT NULL	Flag 0/1: 1 = contrato ativo no período.
fk_tempo_emissao	INT	FK	Referência a dim_tempo (data de assinatura).
fk_tempo_vencimento	INT	FK NULL	Referência a dim_tempo (término previsto). NULL = prazo indeterminado.

Módulo Gestão

fato_ocorrencia_rh

Ocorrências pedagógicas e disciplinares. Granularidade: um registro por ocorrência registrada.

Campo	Tipo	Restrição	Descrição
id_ocorrencia	INT	PK	Surrogate Key da ocorrência.
fk_aluno	INT	FK	Referência a dim_aluno (aluno envolvido).
fk_professor	INT	FK	Referência a dim_professor (professor que registrou).
fk_disciplina	INT	FK	Referência a dim_disciplina (aula em que ocorreu).
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (data da ocorrência).
fk_tipo_ocorrencia	INT	FK	Referência a dim_tipo_ocorrencia.
qtd_ocorrencia	TINYINT	NOT NULL	Quantidade de ocorrências do mesmo tipo no evento.
peso_gravidade	TINYINT	NOT NULL	Nível de gravidade (1 = leve, crescente até crítico).

5.2 Dicionário de Dados OLAP

Convenções

Sigla	Significado
PK	Chave Primária da tabela no modelo analítico.
SK	Surrogate Key — chave artificial gerada no DW, independente da PK do sistema OLTP.
FK	Foreign Key — referência ao SK de uma dimensão.
CALC	Campo calculado no processo ETL a partir de outros campos. Não vem diretamente do OLTP.
Flag 0/1	Campo TINYINT binário: 0 = Não, 1 = Sim. Permite contagens com SUM() sem CASE WHEN.
NULL	Campo opcional — pode ser nulo em cenários válidos de negócio.
Snapshot	Dados gravados no estado do momento da análise, sem depender do estado atual do OLTP.

Dimensões Conformadas

dim_tempo

[DIMENSÃO] Dimensão de tempo compartilhada por todas as tabelas fato. Cada linha representa um único dia. Permite agrupamentos por bimestre, mês, semana e classificação do dia letivo.

Campo	Tipo	Restrição	Descrição
sk_tempo	INT	PK	Surrogate Key do dia no DW. Gerado como YYYYMMDD (ex.: 20250203).
data	DATE	-	Data completa no formato YYYY-MM-DD.

dia	TINYINT	-	Dia do mês (1 a 31).
mes	TINYINT	-	Mês do ano (1 a 12).
nome_mes	VARCHAR	-	Nome por extenso do mês (ex.: Março) para exibição em eixos de gráficos.
ano	SMALLINT	-	Ano de quatro dígitos (ex.: 2025).
bimestre	TINYINT	-	Bimestre escolar correspondente (1 a 4), conforme o calendário letivo.
eh_feriado	BOOLEAN	-	TRUE se o dia é feriado nacional, estadual ou municipal.
eh_dia_util	BOOLEAN	-	TRUE se o dia é útil (segunda a sexta, exceto feriados).
eh_periodo_letivo	BOOLEAN	-	TRUE se o dia está dentro do período letivo definido no calendário escolar.
nome_bimestre	VARCHAR	-	Descrição do bimestre para exibição em relatórios (ex.: 1º Bimestre 2025).
semana_do_ano	TINYINT	-	Número da semana ISO no ano (1 a 53).

dim_turma

[DIMENSÃO] Dimensão que descreve a turma escolar. Compartilhada entre os módulos Acadêmico e Financeiro.

Campo	Tipo	Restrição	Descrição
sk_turma	INT	PK	Surrogate Key da turma no DW.
nome_turma	VARCHAR	-	Código ou nome da turma (ex.: 1A, 3B).
ano_escolar	TINYINT	-	Ano de escolaridade da turma (1 a 5).
periodo	VARCHAR	-	Turno de funcionamento da turma: manhã ou tarde.
ano_letivo	YEAR	-	Ano letivo de vigência da turma (ex.: 2025).
capacidade_maxima	TINYINT	-	Número máximo de alunos permitidos. NULL indica sem limite configurado.

dim_aluno

[DIMENSÃO] Perfil completo do aluno no momento da análise. Dimensão conformada entre Acadêmico, Financeiro e Gestão. Armazenada como snapshot: reflete o estado do aluno no período analisado.

Campo	Tipo	Restrição	Descrição
sk_aluno	INT	PK	Surrogate Key do aluno no DW, independente da PK do OLTP.
ra	VARCHAR(15)	-	Registro do Aluno — chave natural vinda do sistema transacional.
nome_completo	VARCHAR	-	Nome e sobrenome concatenados para exibição em relatórios.
sexo	VARCHAR	-	Sexo declarado pelo aluno (Masculino / Feminino).
data_nascimento	DATE	-	Data de nascimento utilizada para cálculo de faixa etária e validação de idade.
nacionalidade	VARCHAR	-	Nacionalidade do aluno: Brasileiro ou

			Estrangeiro.
raca	VARCHAR	-	Autodeclaração de raça/cor conforme classificação IBGE, para indicadores de equidade.
faz_atividade_extracurricular	BOOLEAN	-	Indica se o aluno participa de atividades extracurriculares no período analisado.
escola_anterior	VARCHAR	-	Tipo de escola de origem (ex.: Pública, Particular) para análise de adaptação.
idade_ingresso	TINYINT	-	Idade do aluno em anos completos no momento do ingresso na escola.
faixa_etaria	VARCHAR	-	Faixa etária calculada no ETL (ex.: 6-8 anos) para agrupamentos em relatórios.
ano_ingresso	YEAR	-	Ano letivo em que o aluno ingressou na escola.
idade_valida_matricula	TINYINT	-	Flag 0/1: indica se a idade estava dentro do intervalo permitido no ato da matrícula.

dim_professor

[DIMENSÃO] Atributos analíticos do professor. Dimensão conformada entre os módulos Acadêmico e RH. Permite cruzar métricas de desempenho dos alunos com o perfil docente.

Campo	Tipo	Restrição	Descrição
sk_professor	INT	PK	Surrogate Key do professor no DW.
nome_completo	VARCHAR	-	Nome completo para exibição em relatórios e dashboards.
sexo	VARCHAR	-	Sexo do professor para análise de equidade de gênero no corpo docente.
usa_rubrica_avaliativa	BOOLEAN	-	TRUE se o professor adota rubricas de avaliação. Correlacionável com desempenho dos alunos.
atende_educacao_especial	BOOLEAN	-	TRUE se o professor possui habilitação para atendimento educacional especializado.
tipo_aula	VARCHAR	-	Modalidade de aula predominante (ex.: Presencial, Híbrido).
nivel_formacao	VARCHAR	-	Nível de formação mais alto concluído pelo professor (ex.: Mestrado, Licenciatura).
curso_principal	VARCHAR	-	Curso de maior nível de formação para análise do perfil docente.

dim_funcionario

[DIMENSÃO] Representa todos os funcionários da escola. Usada exclusivamente no módulo RH para análise de folha, contratos e histórico salarial.

Campo	Tipo	Restrição	Descrição
sk_funcionario	INT	PK	Surrogate Key do funcionário no DW.

cargo	VARCHAR	-	Cargo atual do funcionário, vindo do histórico de cargos mais recente.
tipo_contrato	VARCHAR	-	Tipo de contrato de trabalho vigente (ex.: CLT, PJ, Temporário).
nome_completo	VARCHAR	-	Nome completo para exibição em relatórios de RH.
sexo	VARCHAR	-	Sexo do funcionário para análise de equidade de gênero e remuneração.
carga_horaria_semanal	TINYINT	-	Carga horária semanal contratada em horas (1 a 60).
ativo	BOOLEAN	-	TRUE se o funcionário possui contrato ativo no período de referência.

dim_atividade

[DIMENSÃO] Catálogo dos tipos de atividade avaliativa utilizados pelos professores. Usada em fato_desempenho e fato_atividade_professor.

Campo	Tipo	Restrição	Descrição
sk_tipo_atividade	INT	PK	Surrogate Key do tipo de atividade no DW.
descricao_atividade	VARCHAR	-	Descrição do tipo de atividade (ex.: Prova Bimestral, Trabalho em Grupo, Exercício).

Módulo Acadêmico

dim_situacao

[DIMENSÃO] Resultado acadêmico do aluno ao final de cada bimestre. Usada em fato_boletim para classificar a situação de aprovação.

Campo	Tipo	Restrição	Descrição
sk_situacao	INT	PK	Surrogate Key da situação acadêmica no DW.
descricao	VARCHAR	-	Descrição da situação (ex.: Aprovado, Reprovado, Em Recuperação, Em Andamento).

dim_disciplina

[DIMENSÃO] Catálogo de disciplinas da grade curricular. Conformada entre os módulos Acadêmico e Gestão para análise de desempenho por área do conhecimento.

Campo	Tipo	Restrição	Descrição
sk_disciplina	INT	PK	Surrogate Key da disciplina no DW.
nome_disciplina	VARCHAR	-	Nome da disciplina (ex.: Matemática, Língua Portuguesa, Ciências).
area_conhecimento	VARCHAR	-	Área do conhecimento à qual a disciplina

			pertence (ex.: Ciências Exatas, Linguagens).
ativo	BOOLEAN	-	Indica se a disciplina está ativa na grade curricular no período de referência.

fato_boletim

[FATO] Situação acadêmica final do aluno ao término de cada bimestre. Granularidade: um registro por aluno por bimestre por turma.

Campo	Tipo	Restrição	Descrição
sk_boletim	INT	PK	Surrogate Key do registro de boletim no DW.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_tempo	INT	FK	Referência a dim_tempo (data de fechamento do bimestre).
fk_turma	INT	FK	Referência a dim_turma.
fk_situacao	INT	FK	Referência a dim_situacao (resultado acadêmico do bimestre).
esta_aprovado	TINYINT	-	Flag 0/1: 1 = aluno aprovado no bimestre.
esta_reprovado	TINYINT	-	Flag 0/1: 1 = aluno reprovado no bimestre.
em_recuperacao	TINYINT	-	Flag 0/1: 1 = aluno em recuperação no bimestre.

fato_frequencia

[FATO] Frequência do aluno por dia letivo e turma. Granularidade: um registro por aluno por dia do calendário letivo.

Campo	Tipo	Restrição	Descrição
id_frequencia	INT	PK	Surrogate Key do registro de frequência no DW.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (dia letivo registrado).
qtd_presenca	TINYINT	-	Quantidade de aulas em que o aluno esteve presente naquele dia.
qtd_falta	TINYINT	-	Quantidade de faltas sem justificativa registrada.
qtd_justificada	TINYINT	-	Quantidade de faltas com justificativa registrada pelo responsável.

fato_desempenho

[FATO] Desempenho individual de cada aluno em cada atividade avaliativa. Granularidade: um registro por aluno por atividade por bimestre.

Campo	Tipo	Restrição	Descrição
id_desempenho	INT	PK	Surrogate Key da linha fato no DW.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_professor	INT	FK	Referência a dim_professor (responsável pela atividade avaliativa).

fk_disciplina	INT	FK	Referência a dim_disciplina.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (data de realização da atividade).
fk_tipo_atividade	INT	FK	Referência a dim_atividade (tipo de avaliação aplicada).
valor_nota	DECIMAL	-	Nota obtida pelo aluno na atividade. NULL indica não avaliado.
percentual_nota	DECIMAL	CALC	Percentual da nota em relação ao valor máximo: $(\text{valor_nota} / \text{nota_maxima}) * 100$.
nota_maxima	DECIMAL	-	Valor máximo possível na atividade, copiado do OLTP para preservar o histórico.
peso_atividade	DECIMAL	-	Peso da atividade no cálculo do conceito bimestral.
qtd_entregue_no_prazo	TINYINT	-	Flag 0/1: 1 = atividade entregue dentro do prazo previsto.

fato_atividade_professor

[FATO] Produção de atividades avaliativas por professor por disciplina. Granularidade: um registro por professor por disciplina por bimestre.

Campo	Tipo	Restrição	Descrição
sk_atividade	INT	PK	Surrogate Key do registro no DW.
fk_professor	INT	FK	Referência a dim_professor.
fk_disciplina	INT	FK	Referência a dim_disciplina.
fk_turma	INT	FK	Referência a dim_turma.
fk_tempo	INT	FK	Referência a dim_tempo (data prevista da atividade).
fk_tipo_atividade	INT	FK	Referência a dim_atividade.
qtd_realizada	TINYINT	-	Quantidade de atividades deste tipo realizadas no período analisado.

Módulo Financeiro

dim_status_pagamento

[DIMENSÃO] Status possíveis de uma cobrança de mensalidade. Usada em fato_pagamento para segmentar análises de inadimplência.

Campo	Tipo	Restrição	Descrição
sk_status_pagamento	INT	PK	Surrogate Key do status de pagamento no DW.
descricao_status	VARCHAR	-	Descrição do status (ex.: Pago, Pendente, Atrasado, Isento, Cancelado).

fato_pagamento

[FATO] Cobranças de mensalidade geradas para os alunos. Granularidade: um registro por parcela por mês de referência. Alimenta indicadores de inadimplência e receita.

Campo	Tipo	Restrição	Descrição
id_pagamento	INT	PK	Surrogate Key da cobrança no DW.
fk_aluno	INT	FK	Referência a dim_aluno.
fk_tempo	INT	FK	Referência a dim_tempo (data de vencimento da parcela).
fk_status_pagamento	INT	FK	Referência a dim_status_pagamento.
valor_mensalidade	DECIMAL	-	Valor de face da mensalidade conforme o contrato financeiro.
valor_pago	DECIMAL	-	Valor efetivamente recebido. NULL quando ainda não houve pagamento.
qtd_foi_pago	TINYINT	-	Flag 0/1: 1 = mensalidade paga. Facilita agregações com SUM().
qtd_foi_atrasado	TINYINT	-	Flag 0/1: 1 = pagamento realizado após a data de vencimento.
qtd_dias_atrasado	INT	-	Número de dias de atraso. Zero se pago em dia ou ainda não pago.

Módulo RH

dim_folha

[DIMENSÃO] Tipos de folha de pagamento processados no módulo RH. Usada em fato_folha_rh para segmentar análises por modalidade de pagamento.

Campo	Tipo	Restrição	Descrição
sk_tipo_folha	INT	PK	Surrogate Key do tipo de folha no DW.
descricao_folha	VARCHAR	-	Descrição do tipo (ex.: Folha Mensal, Décimo Terceiro, Férias, Rescisão, Adiantamento).

fato_folha_rh

[FATO] Folhas de pagamento processadas por funcionário. Granularidade: um registro por funcionário por mês por tipo de folha.

Campo	Tipo	Restrição	Descrição
id_folha_rh	INT	PK	Surrogate Key da folha no DW.
fk_funcionario	INT	FK	Referência a dim_funcionario.
fk_tempo	INT	FK	Referência a dim_tempo (data de emissão da folha).
fk_tipo_folha	INT	FK	Referência a dim_folha (modalidade da folha processada).
valor_bruto	DECIMAL	-	Valor bruto total da folha antes de qualquer

			desconto.
valor_liquido	DECIMAL	-	Valor líquido a receber pelo funcionário após todos os descontos.
salario_base	DECIMAL	-	Salário base do contrato vigente no período de referência.
total_creditos	DECIMAL	-	Soma de todas as verbas de crédito (proventos, horas extras, adicionais).
total_descontos	DECIMAL	-	Soma de todas as verbas de débito (INSS, IRRF, vale-transporte, etc.).
salario_anterior	DECIMAL	-	Salário base do período imediatamente anterior, para cálculo do reajuste.
percentual_aumento	DECIMAL	CALC	Variação salarial calculada: $((\text{salario_base} - \text{salario_anterior}) / \text{salario_anterior}) \times 100$.
motivo_aumento	VARCHAR	-	Justificativa do reajuste salarial (ex.: Dissídio, Promoção). NULL se não houve reajuste.

fato_contrato_rh

[FATO] Contratos de trabalho vigentes por funcionário ao longo do tempo. Granularidade: um registro por contrato ativo por período. Permite análise de turnover e evolução salarial.

Campo	Tipo	Restrição	Descrição
sk_contrato_rh	INT	PK	Surrogate Key do contrato no DW.
fk_funcionario	INT	FK	Referência a dim_funcionario.
fk_tempo	INT	FK	Referência a dim_tempo (data de início do contrato).
salario_base	DECIMAL	-	Salário base vigente no momento do snapshot do contrato.
carga_horaria	TINYINT	-	Carga horária semanal contratada em horas (1 a 60).
contrato_ativo	TINYINT	-	Flag 0/1: 1 = contrato ativo no período de referência.
fk_tempo_emissao	INT	FK	Referência a dim_tempo (data de assinatura do contrato).
fk_tempo_vencimento	INT	FK	Referência a dim_tempo (término previsto). NULL para prazo indeterminado.

Módulo Gestão

dim_risco

[DIMENSÃO] Classifica os alertas de evasão escolar por tipo e nível de criticidade. Usada em fato_evasao para priorizar intervenções pedagógicas.

Campo	Tipo	Restrição	Descrição
sk_risco	INT	PK	Surrogate Key do risco no DW.
tipo_risco	VARCHAR	-	Categoria do fator de risco (ex.: Faltas)

			Excessivas, Inadimplência, Baixo Desempenho).
nivel_risco	VARCHAR	-	Nível de criticidade do risco (ex.: Baixo, Médio, Alto, Crítico).
ordem_criticidade	TINYINT	-	Valor numérico para ordenação em dashboards (crescente = mais crítico).

fato_evasao

[FATO] Alertas de risco de evasão escolar gerados pelo sistema. Granularidade: um registro por alerta emitido. Permite análise de tempo de resolução e eficácia das intervenções.

Campo	Tipo	Restrição	Descrição
sk_evasao	INT	PK	Surrogate Key do alerta de evasão no DW.
fk_aluno	INT	FK	Referência a dim_aluno (aluno em situação de risco).
fk_turma	INT	FK	Referência a dim_turma (turma do aluno no período do alerta).
fk_tempo	INT	FK	Referência a dim_tempo (data de emissão do alerta).
fk_risco	INT	FK	Referência a dim_risco (tipo e nível de criticidade do risco identificado).
dias_ate_resolucao	DECIMAL	-	Número de dias entre a emissão e a resolução do alerta. NULL se ainda não resolvido.
foi_resolvido	TINYINT	-	Flag 0/1: 1 = alerta foi resolvido antes da evasão.
evadiu	TINYINT	-	Flag 0/1: 1 = aluno efetivamente evadiu após o registro do alerta.

5. Problemas Identificados e Correções Implementadas

A seguir, cada ponto levantado pela auditoria é descrito com seu estado anterior e a solução adotada na versão v06.

5.1 Chaves primárias compostas em tabelas N:M

Antes: As tabelas tb_responsavel_aluno, tb_verba_folha e tb_nota_atividade usavam IDs artificiais (pk_) como chave primária, em vez das chaves estrangeiras compostas, violando o modelo relacional para tabelas associativas.

Depois (v06): Implementadas PKs compostas das FKs nativas em todas as tabelas N:M. A tentativa de inserir um par duplicado gerou Erro 1062 no teste T07, confirmando a proteção.

5.2 Ausência de UNIQUE em tb_email

Antes: Sem a constraint UNIQUE no campo email, o mesmo endereço podia ser vinculado a múltiplas pessoas ou repetido indefinidamente para a mesma pessoa,

criando Dirty Data, falha de segurança na recuperação de senha e disparos duplicados de comunicação.

Depois (v06): Adicionada constraint uq_email. O teste T02 confirmou o bloqueio de e-mail duplicado com Erro 1062. O trigger trg_email_validar_insert/update impede que uma mesma pessoa tenha mais de um e-mail principal (T31).

5.3 Ausência de UNIQUE em tb_telefone (ddd + numero)

Antes: Sem a UNIQUE KEY composta de ddd+numero, o mesmo telefone podia ser cadastrado múltiplas vezes para a mesma pessoa ou para pessoas diferentes, comprometendo o módulo de Alerta de Evasão (RN005) e gerando inconsistência no BI.

Depois (v06): Adicionada constraint uq_ddd_telefone. O teste T03 confirmou o bloqueio. O trigger garante no máximo um telefone principal por pessoa (T33).

5.4 Campo calculado valor_cobrado armazenado fisicamente em tb_pagamento

Antes: O campo valor_cobrado armazenava o resultado de juros + multas de forma física, violando a 3FN e tornando os dados históricos inconsistentes caso as regras financeiras (RN004 e RN010) fossem alteradas.

Depois (v06): O campo foi removido. O teste T54 confirmou que apenas valor_original existe na tabela — o cálculo ocorre em tempo de execução. A integridade da fonte única da verdade foi restaurada.

5.5 Campo nome_completo composto em tb_pessoa (violação de 1FN)

Antes: O campo nome_completo misturava nome e sobrenome em uma única coluna, impedindo buscas exatas e forçando o uso de operadores custosos como LIKE '%...%', o que compromete performance e indicadores de BI.

Depois (v06): O campo foi mantido na v06 mas documentado como ponto de melhoria futura. A violação foi formalmente registrada no dicionário de dados. Refatoração prevista para versão posterior sem impacto operacional imediato.

5.6 Prefixo 'to' em vez de 'tb_' em tabelas

Antes: Tabelas como to_calendario_letivo e to_responsavel usavam o prefixo incorreto 'to', violando a regra de nomenclatura tb_ exigida e impedindo a execução automatizada do script DDL.

Depois (v06): Todos os prefixos foram corrigidos para tb_. O teste T53 confirmou que tb_tipo_verba e tb_verba_folha existem corretamente no schema v06.

5.7 CHECK com CURRENT_DATE em tb_pessoa (bug médio)

Antes: Uma constraint CHECK usava CURRENT_DATE, expressão inválida no DDL estático do MySQL 8.0, tornando a constraint inoperante e potencialmente bloqueando execuções do script.

Depois (v06): A constraint foi removida. O teste T51 verificou via information_schema que nenhum CHECK com curdate existe em tb_pessoa.

5.8 CHECK incorreto em tb_matricula (bug crítico)

Antes: O CHECK em tb_matricula referenciava a coluna com lógica incorreta, podendo bloquear inserções válidas ou permitir datas inválidas sem aviso.

Depois (v06): O CHECK foi corrigido para referenciar data_matricula adequadamente. O teste T50 confirmou a correção via information_schema.

5.9. DEFAULT como string literal 'true'/'false' (bug menor)

Antes: Alguns campos usavam a string literal 'true' como valor DEFAULT, incompatível com o tipo BOOLEAN/TINYINT do MySQL, causando comportamento inesperado na inserção de registros.

Depois (v06): Corrigido para os valores nativos 0 e 1. O teste T52 confirmou que nenhuma coluna possui DEFAULT de string literal no schema v06.

10. Índice UNIQUE duplicado em tb_grade_curricular

Antes: Havia um índice UNIQUE duplicado com data_inicio na tabela tb_grade_curricular, gerando redundância de índice e potencial conflito em operações de DDL.

Depois (v06): O índice duplicado foi removido. O teste T55 confirmou que apenas um UNIQUE index com data_inicio existe — nenhum duplicado.

11. Links externos ausentes no relatório PDF

Antes: O relatório entregue não continha os links externos referenciados no corpo da mensagem, caracterizando descumprimento de integridade documental, com penalização aplicada na nota.

Depois (v06): O relatório de testes v06 foi gerado com referências completas, data de emissão explícita (09/05/2026) e rastreabilidade de ambiente. A documentação foi atualizada.

12. Ausência de triggers para regras de negócio

Antes: Nenhuma regra de negócio era protegida por triggers. O banco dependia exclusivamente de validação na camada de aplicação, expondo-o a inconsistências em acessos diretos ao banco de dados.

Depois (v06): 20 triggers implementados em 10 grupos (INSERT + UPDATE). 56 testes cobriram todos os cenários com 100% de aprovação. As regras críticas como RN015 (capacidade de turma) e unicidade de principal agora são garantidas no próprio banco.

3. Detalhamento por Bateria de Testes

Os 56 testes foram organizados em 8 baterias. Todos aprovados — nenhuma falha registrada.

- **Bateria 1 — Unicidade e Duplicatas (T01–T12):** 12 testes cobrindo CPF, e-mail, telefone, notas e folha de pagamento. Resultado: 12/12 aprovados.
- **Bateria 2 — Triggers de Contrato / RH (T13–T17b):** 6 testes validando funcionário ativo, sobreposição de períodos e consistência status × data_fim. Resultado: 6/6 aprovados.
- **Bateria 3 — Histórico de Cargo e Salário (T18–T23):** 6 testes sobre cargo ativo, datas dentro do contrato e sobreposição de períodos salariais. Resultado: 6/6 aprovados.
- **Bateria 4 — Capacidade da Turma / RN015 (T24–T27):** 4 testes cobrindo vagas ativas, matrículas trancadas e turmas sem limite definido. Resultado: 4/4 aprovados.
- **Bateria 5 — Ano Letivo e Dados Principais (T28–T36):** 9 testes para ano letivo único ativo, e-mail/telefone/responsável principal e validação cruzada de turma. Resultado: 9/9 aprovados.
- **Bateria 6 — Integridade Referencial e CHECK de Domínio (T37–T45):** 9 testes de FKs, ON DELETE RESTRICT e domínio de valores (bimestre, dia de vencimento, horário, alerta). Resultado: 9/9 aprovados.
- **Bateria 7 — Fluxos Completos entre Módulos (T46–T49):** 4 testes de ponta a ponta nos módulos RH, Acadêmico, Financeiro e Alerta de Evasão. Resultado: 4/4 aprovados.
- **Bateria 8 — Pontos do Documento de Auditoria (T50–T55):** 6 testes de regressão cobrindo especificamente cada bug apontado pelo professor. Resultado: 6/6 aprovados.

4. Triggers Implementados — Regras de Negócio Protegidas

Antes da versão v06, nenhuma regra de negócio era protegida por triggers. A v06 implementou 20 triggers organizados em 10 grupos, cada um com um disparo de INSERT e um de UPDATE:

- **Grupo 1 — tb_contrato:** funcionário ativo, sobreposição de períodos, consistência entre status e data_fim.
- **Grupo 2 — tb_historico_cargo:** cargo ativo, datas dentro do intervalo do contrato, sobreposição de períodos de cargo.
- **Grupo 3 — tb_historico_salario:** datas dentro do contrato, sobreposição de períodos salariais.

- **Grupo 4 — tb_matricula:** capacidade máxima da turma (RN015) — matrículas trancadas não ocupam vaga.
- **Grupo 5 — tb_ano_letivo:** unicidade de ano letivo ativo — apenas um ativo por vez.
- **Grupo 6 — tb_email:** unicidade de e-mail principal por pessoa.
- **Grupo 7 — tb_telefone:** unicidade de telefone principal por pessoa.
- **Grupo 8 — tb_responsavel_aluno:** unicidade de responsável principal por aluno.
- **Grupo 9 — tb_presenca_aluno:** horário de aula pertence à turma da matrícula.
- **Grupo 10 — tb_nota_atividade:** atividade pertence à turma da matrícula informada.

5. Integridade Referencial e Constraints de Domínio

A versão anterior apresentava ausências críticas de constraints que comprometiam a confiabilidade dos dados. Todas foram corrigidas na v06:

- **UNIQUE em tb_email:** antes ausente, permitindo Dirty Data. Corrigido com uq_email — bloqueio confirmado pelo Erro 1062 no teste T02.
- **UNIQUE em tb_telefone (ddd+numero):** antes ausente, permitindo duplicatas livres. Corrigido com uq_ddd_telefone — confirmado no teste T03.
- **PK composta em N:M:** antes substituída por IDs artificiais. Corrigida com PKs compostas nativas — Erro 1062 em duplicata confirmado no teste T07.
- **ON DELETE RESTRICT:** antes parcialmente implementado. Agora presente em todas as 52 FKs — confirmado nos testes T38 e T39.
- **CHECK em tb_matricula:** antes referenciava coluna incorreta (bug crítico). Corrigido e verificado via information_schema no teste T50.
- **CHECK com CURRENT_DATE em tb_pessoa:** antes inválido no MySQL 8.0 (bug médio). Removido e confirmado sem curdate no teste T51.
- **DEFAULT literal string:** antes incompatível com BOOLEAN/TINYINT (bug menor). Corrigido para 0/1 nativo — confirmado no teste T52.

Referências:

DE MORAES, Cristine do C. S. Bueno; CAMOLESI JÚNIOR, Luiz. Modelagem de Regras de Negócios Integrada ao Projeto de Software Orientado a Processos de Negócios. In: SIMPÓSIO BRASILEIRO DE SISTEMAS DE INFORMAÇÃO (SBSI), 2. , 2005, Florianópolis. [...]. Porto Alegre: Sociedade Brasileira de Computação, 2005 . p. 111-118. ISSN 3086-4836. DOI: <https://doi.org/10.5753/sbsi.2005.14974>. Acesso em: 15 mar. 2026.

IWAMOTO, P. Y. S.; LINTZMAYER, C. N.; TAIT, T. F. C.; HUZITA, E. H. M. Estudo de técnicas de modelagem de regras de negócio. [S.l.: s.n., s.d.]. Acesso em: 16 mar. 2026.

ALENQUER, Pablo Lopes. **Regras de negócio para análise em ambientes OLAP**. 2002. Dissertação (Mestrado) — Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2002. Acesso em: 18 mar. 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de banco de dados. Tradução de vários tradutores. Revisão técnica de Luis Ricardo de Figueiredo*. São Paulo: Pearson Addison Wesley, 2005. Acesso em: 18 mar. 2026.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo, SP: Pearson, 2018. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 02 abr 2026.

PROENÇA, Fernando Roberto. **BD1 - Dicionário de Dados**. [S.l.]: YouTube, 2020. 1 vídeo. Disponível em: <https://youtu.be/VD1FKJ5hHYs>. Acesso em: 20 mar. 2026.

PROENÇA, Fernando Roberto. **BD1 - Modelo Relacional do Banco de Dados**. [S.l.]: YouTube, 2020. 1 vídeo. Disponível em: <https://youtu.be/3ZV6kpoFBPs>. Acesso em: 20 mar. 2026.

